

Hardness Over Hope: Policy-as-Code and Deterministic Governance in LLM Agent Orchestration

Confinamiento de varianza en agentes LLM mediante un plano de control determinista y plástico

Borrador 3.1 (v2 de la tesis), 2026-09-02 Autor: Ariel Edgardo Levy Estado: borrador de trabajo. Los dos teoremas están verificados por máquina; toda afirmación empírica lleva su tamaño de muestra y su piso de ruido, y §1.3 declara el alcance. Destino: arXiv cs.LG (primario), cs.AI (cross-list).

Esta es la única versión mantenida. La redacción inglesa quedó congelada en `historico/paper-en-congelado.md`, anterior a este borrador y superada por él. El borrador 2.0 quedó en `historico/paper-es-borrador-2.0-2026-08-31.md`.

Las diferencias con los borradores anteriores están en el Apéndice C.

Resumen

Un arnés de agentes es la estructura de control que envuelve a un LLM. Decide qué llamada viene después, qué buscar, cuántas vueltas dar, cuándo verificar, cuándo reintentar y cuándo parar. Cuando esas decisiones las toma el propio modelo, o cuando el arnés verifica llamando al mismo LLM o a otro, el arnés hereda la aleatoriedad de lo que quería controlar. El guard que le pregunta a un modelo si la respuesta de un modelo es correcta es el caso típico: apila una segunda llamada estocástica sobre la primera. Apilar instancias del componente sube una probabilidad sin producir una propiedad.

En este trabajo interponemos entre el modelo y el flujo un motor determinista y plástico. El LLM sólo dice qué hay en el material; su salida se tipa como proposiciones con procedencia en una base de creencias. Reglas en código deciden sobre esa base, y el resultado es una garantía que se verifica repitiéndola: la misma base produce la misma decisión. El motor aprende con los pesos del LLM congelados. Lo que aprende son políticas, tablas de decisión sobre qué exige un request y qué puede hacer cada brazo (así llamamos a cada paradigma que el motor puede correr), consolidadas offline con guarda anti-regresión, firmadas y ejecutadas como código (§3.5). Todo se mide sobre una ejecución real: un banco que corre doce paradigmas sobre 78 tareas de análisis forense de hechos, sin juez LLM, y que es la fuente de episodios de la que el sistema aprende.

Cuatro contribuciones.

1. Una condición estructural sobre la varianza. Un agente confina su varianza cuando ninguna ramificación de su trayectoria depende del LLM, y entonces toda discrepancia entre corridas tiene un

nodo responsable (Proposición 4). El confinamiento no pide un modelo mejor; pide mover decisiones al código, y en el banco mover una sola decisión se cobró a la vez en utilidad y en reproducibilidad: al menos entre el 12% y el 28% de las celdas cambian de resultado a temperatura cero, y sacarle al LLM una sola decisión lleva un paradigma de 0,33 a 0,89 y hace coincidir sus réplicas (§6.1).

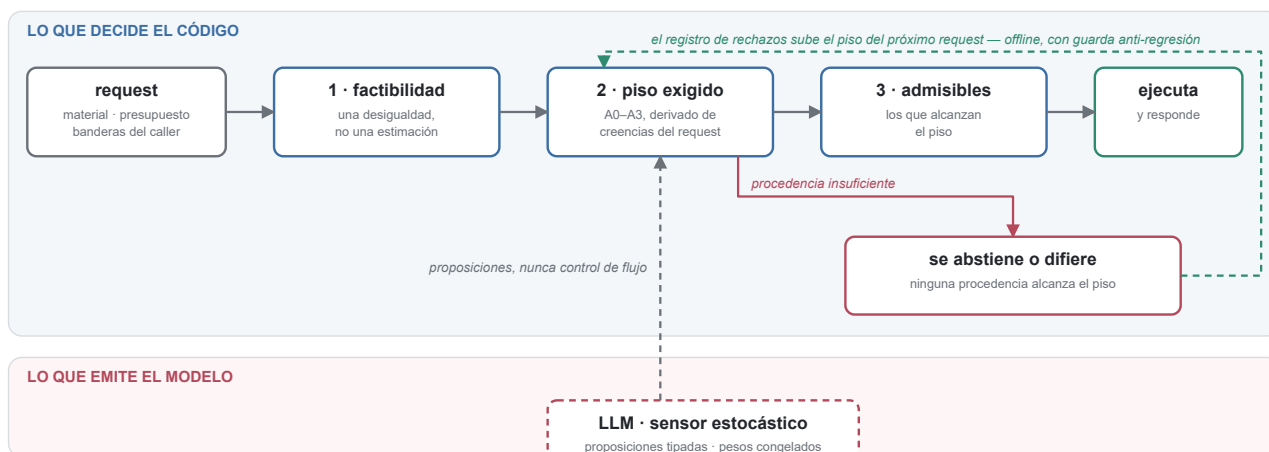
2. Una interfaz aprendible que no depende de nombres. Un paradigma es un paquete de capacidades declaradas desde su código, y la pregunta exige capacidades. Sobre esa interfaz se puede predecir un paradigma que nunca corrió, cosa que ninguna política sobre identidades de paradigma puede hacer: mejor que la dificultad de la tarea sola, sin cruzar su nulo ($p = 0,058$) y por debajo de la identidad aditiva (MAE 0,229 contra 0,225). Sugestivo, y más brazos lo sentencian (§6.3).
3. Una frontera declarada entre lo que madura solo y lo que se diseña. La política crece desde el registro sin que nadie toque una regla, θ pasó de gobernar cero tareas a 36 de 78 reproducible en cada ciclo, y una guarda frenó el único paso que no se separaba del ruido (§6.6). Los sentidos nuevos, los medidores, los agregan personas, y cada uno entró por una refutación preregistrada con la decisión reproducida 26 de 26 (§6.5).
4. Dónde está el premio de elegir. Entre brazos con las mismas capacidades no hay premio de calidad que se separe del ruido; el premio vive en los ejes donde las capacidades difieren. El desempate por costo entre brazos capaces ahorra 58% de tokens a utilidad igual al mejor fijo, leave-one-task-out (§6.4.1). Y el acuerdo entre paradigmas predice la corrección con precisión de al menos 0,98 desde cuatro coincidencias, y mueve credencia, nunca procedencia (§6.4.2).

Las cuatro descansan en una exigencia sobre la política que §6.2.5 mide. Una política, acá, es una tabla que mira una clave, una tupla de ejes medidos sobre el request y el material, y devuelve una decisión. Esa clave tiene que ser `COMPUTED`, el nivel de procedencia que significa calculada por código desde el request y el material sin pasar por el modelo. Si un eje de la clave lo emite el LLM, la misma pregunta puede caer en filas distintas de la tabla en dos réplicas, y la decisión hereda la varianza que el motor existía para confinar. Determinismo y aprendizaje no compiten; el primero define sobre qué aprende el segundo. El banco enseñó además que entre brazos con las mismas capacidades la brecha de oráculo apenas se separa del piso de ruido, y que el premio sobre el catálogo entero lo cobrará una clave que vea capacidades (§6.2).

Palabras clave: confinamiento de varianza, agentes LLM, plano de control, procedencia, predicción selectiva, aprendizaje plástico, capacidades, reproducibilidad, trazabilidad

El contrato de garantía, y qué pasa cuando la evidencia no alcanza

la abstención es una salida, no un fallo - y el rechazo sube el piso para el próximo request, sin tocar un peso



La garantía no es «mismo prompt $\Rightarrow$ misma respuesta», que ningún LLM puede dar. Es «misma base de creencias $\Rightarrow$ misma decisión», que se verifica repitiéndola.

Figura 1. El contrato de garantía, nodo por nodo. El request entra con tres cosas que declara quien llama: el material, el presupuesto y las banderas de riesgo. El nodo 1, factibilidad, es una desigualdad y no una estimación: cada paradigma declara cuántas unidades lee y cuántas llamadas emite, y los que no entran en el presupuesto se podan sin gastar un token. El nodo 2 deriva el piso de garantía exigido, A0 a A3, desde creencias sobre el request y no desde su texto. El nodo 3 deja pasar sólo a los paradigmas admisibles, los que alcanzan ese piso. Recién el nodo final ejecuta al elegido y responde, y es el único que gasta tokens. Dos cosas que el resto del paper desarrolla en prosa están acá dibujadas. La rama de rechazo: cuando ninguna procedencia alcanza el piso exigido, el sistema se abstiene o difiere, y eso es una salida y no un fallo. Y el lazo que vuelve desde el rechazo, que es la plasticidad: el registro de rechazos sube el piso del próximo request, offline y con guarda anti-regresión, sin que nadie toque un peso. El carril de abajo es el LLM, un sensor estocástico con pesos congelados. La única flecha que cruza la frontera entre carriles lleva proposiciones tipadas hacia arriba, nunca control de flujo hacia abajo. La garantía que la figura dibuja es «misma base de creencias, misma decisión», y se verifica repitiéndola; «mismo prompt, misma respuesta» no la puede dar ningún LLM, y el motor no la promete.

1. Introducción

1.1 Tres propiedades que un sistema de producción exige

Un *arnés* de agentes es la estructura de control que envuelve al modelo: una llamada única, un bucle de razonamiento, una descomposición, un grafo de verificar-replanificar. Se elige una vez en tiempo de diseño y se congela en el código, y sobre él se sostienen sistemas agentivos que firman números, disparan acciones y contestan a usuarios. Las demás capas de un sistema de producción dan tres propiedades por sentadas. Una base de datos ejecuta la misma consulta igual cada vez, un log dice de dónde salió cada registro, y un validador rechaza la entrada que no cumple el esquema. En un agente esas tres propiedades hay que construirlas, porque su componente central no tiene ninguna.

1. El flujo de control vive en el código. Los modelos son aleatorios por naturaleza, y lo siguen siendo a pesar del trabajo extenso en darles más determinismo: la misma llamada a temperatura cero, con el mismo prompt y la misma semilla, devuelve una distribución de respuestas y no una respuesta. La única forma de sacar esa aleatoriedad de una decisión es que la decisión la tome el código. Lo que un arnés decide es cuánto del sistema depende de la aleatoriedad del modelo. Delegarle ramificaciones (qué buscar después, cuántas vueltas dar, cuándo parar) vuelve aleatoria la trayectoria misma. Es una decisión de arquitectura, y en el banco ejecutado su precio está medido sobre modelos de última generación, `gpt-5.6-luna` y `gpt-5.6-terra` de OpenAI: al menos entre el 12% y el 28% de las celdas (una celda es un par tarea × paradigma; una réplica, una corrida repetida de la misma celda) cambian de resultado entre réplicas a temperatura cero con la misma huella (§6.1.2). El glosario de §3.1 fija los términos del banco.
2. Cada valor emitido exhibe de dónde salió. Una respuesta correcta y una inventada llegan con la misma cara, así que la fuente tiene que viajar con el valor. Eso es lo que vuelve explicable una decisión: se puede mostrar de qué unidad del material salió cada número y qué regla lo dejó pasar. Un sistema hecho sólo de llamadas al LLM no puede ofrecerlo, porque la explicación que el modelo da de su propia salida es otra emisión, con la misma cara que la primera. Con una base de creencias

tipada se puede exigir que un número esté implicado por evidencia de cierto nivel; el sustituto es la confianza en que «el modelo suele acertar».

3. El sistema puede callarse. En el modo más difícil del corpus, las cadenas acopladas medidas con `terra` (§1.3.2 dice qué modelo corre cada medición), el paradigma que gana lo hace con 8 correctas y 1 abstención sobre 9 celdas, sin una sola respuesta equivocada; y en el mismo modo sobre `luna` los brazos (así llama el banco a cada paradigma que corre) que fallan se abstienen más de lo que inventan (§6.1.3). El banco puntúa abstención y error con el mismo `0,000`, que es correcto para medir utilidad y ciego justo sobre el eje que producción necesita.

La tesis es que las tres se consiguen en el mismo lugar: moviendo decisiones de flujo de control desde el modelo hacia el código, sobre señales del entorno que son contables y verificables. Este trabajo las construye, las mide y dice qué cuestan.

«Aprender» tiene acá un sentido más estrecho que el habitual en cs.LG, y hay que fijarlo antes de seguir. Aprender significa que el sistema madura: cambia lo que hace con la experiencia, cada cambio es un artefacto que una persona puede leer y comparar con el anterior, y ningún cambio rompe la garantía de la decisión. Lo que se acumula son creencias, proposiciones y tablas, nunca pesos; lo que las gobierna son reglas y medidores (un medidor computa una propiedad de la tarea desde el request o desde el material, sin llamar al modelo) que se escriben en código y no aprenden.

Con eso quedan fijadas tres palabras más. Determinismo son esas reglas. Plasticidad es que las creencias, las proposiciones y las tablas se acumulen solas, desde lo que el sistema sensa y registra, sin que una persona intervenga en el diseño para cada cambio. Explicabilidad es que el sistema entregue, por decisión, la clave que vio, la creencia que la disparó y la versión firmada de la tabla que la produjo. §3.5 dice qué es fijo y qué es plástico y por qué la frontera está donde está; §6.6 mide si el sistema, con esa definición, maduró sobre el registro de la campaña.

1.1.1 La instrucción llega hasta la probabilidad, y una norma pide una propiedad

Hasta ahora el comportamiento determinista se buscó instruyendo al modelo: «citá siempre», «si no sabés, decilo», «no inventes números». Los modelos siguen instrucciones cada vez mejor y cada generación se acerca más. Pero una instrucción es un prior sobre la distribución de salidas, no una restricción sobre la muestra: seguirla mejor sube la probabilidad, y el límite de esa sucesión queda del lado abierto.

En un chat la brecha da igual. Una norma pide otra cosa: el Reglamento de IA de la Unión Europea exige trazabilidad y supervisión humana sobre sistemas de alto riesgo; el artículo 22 del RGPD condiciona las decisiones automatizadas sobre personas; una auditoría exige exhibir por qué se afirmó algo. Los tres piden una propiedad del sistema, y eso se consigue con una restricción, no con un pedido. Este trabajo reemplaza la instrucción por una restricción fuera del modelo.

1.1.2 Una expectativa de la literatura, y qué mide de verdad

La motivación habitual para trabajar sobre arneses es que elegir entre patrones o paradigmas es una decisión importante, porque el mismo modelo rinde distinto según la estructura de control que lo envuelve. La selección oráculo por tarea supera al mejor paradigma fijo por 17,1pp sobre seis paradigmas, cuatro modelos frontera y diez benchmarks [Select-then-Solve, arXiv:2604.06753]. El mismo trabajo mide que el premio no se cobra (un ruteador sobre embeddings recupera un cuarto de la brecha, el auto-ruteo zero-shot recupera valor *negativo*), y la conclusión habitual es que hacen falta mejores selectores.

El trabajo del banco reordena esa expectativa, y lo hace incorporando un nivel intermedio entre la pregunta y el paradigma: las capacidades. Los paradigmas son paquetes distintos de capacidades, y el premio alcanzable aparece sobre los ejes donde las capacidades difieren: costo, cobertura, abstención. §6.4.1 mide el primero: 58% de ahorro con una señal computada, a utilidad igual al mejor fijo, evaluado dejando una tarea afuera por vez (leave-one-task-out).

Sobre la calidad, el eje que los contendientes comparten, el premio apenas se separa del ruido. Los contendientes son los tres paradigmas cuya utilidad queda pegada a la del mejor fijo. La brecha de oráculo (lo que ganaría quien eligiera por tarea el mejor paradigma, sobre el mejor paradigma fijo) entre los tres es $+0,058$, y apenas se separa de su piso de ruido, que es la brecha que un oráculo mostraría por puro azar, porque un máximo sobre estimaciones ruidosas está sesgado hacia arriba ($+0,028$ neto contra el piso calibrado, $+0,011$ contra su p95). §6.3 dice por qué: los tres tienen las mismas capacidades y son el mismo brazo para decidir. La interacción tarea×paradigma es grande (39% de la varianza explicada, descontado el ruido) y vive sobre todo entre los paradigmas que nadie elegiría (§6.2). Sobre los ocho brazos y sobre el held-out el oráculo sí muestra premio neto, $+0,07$ a $+0,15$, y lo que ese premio pide es una clave que lo vea: ninguna señal por identidad de paradigma lo predice (§6.2.4, §6.5). El banco existe para que el sistema aprenda qué exige cada request y qué le puede dar cada brazo, y ahí es donde el premio se cobra.

1.2 Cuatro afirmaciones que preceden al problema de aprendizaje

El banco es fuente de episodios, no torneo. Un banco que corre doce paradigmas sobre las mismas tareas produce dos cosas: un ranking, y un registro de qué hizo cada brazo sobre cada pregunta. El ranking es lo que se lee primero y lo que menos vale: sobre calidad los brazos capaces empatan (§6.2) y el mejor fijo cambia de mundo a mundo (§6.2.8). El registro es lo que alimenta al sistema, y lo que se extrae de él es el motivo del comportamiento, no un ganador. Qué capacidad faltaba donde un brazo falló, qué eje de la pregunta separaba dos celdas que la región confundía, qué señal contable predice que una trayectoria se va a desbocar. Todo lo que este paper llama aprendizaje es eso, y por eso una refutación es un episodio y no un fracaso (§6.5).

La factibilidad es aritmética, y es gratis. Antes de preguntar cuál topología es *mejor*, se puede preguntar cuál puede *correr*. Esa pregunta se responde con cantidades que la tarea ya declara. Una política aprendida que gasta episodios descubriendo que map-reduce pierde en tareas de 500 unidades está aprendiendo aritmética por el camino difícil; el tope era computable antes del primer token.

La selección paga sólo bajo una condición precisa. Un ruteador que debe elegir siempre no tiene ningún grado de libertad sobre su tasa de falsos positivos y por lo tanto paga cada error de ruteo. Formalizamos cuándo la selección le gana a un fallback fijo y mostramos que el punto de operación óptimo generalmente implica abstenerse en la mayoría de las solicitudes. Eso es un objetivo distinto, no un ruteador más débil.

Mucho de lo que se le atribuye a la topología es atribuible a la superficie. Entre el paradigma más caro y el más barato del plantel hay un factor $10\times$ de costo y $0,17$ de utilidad: en calidad se separan por centésimas y en lo que cuestan, por órdenes de magnitud (§5.2). Y lo que gobierna esa diferencia es cuánto material arrastra cada uno al prompt, no la estructura de control. El costo es de 98,6% a 100,0% de entrada en todos ellos. Un benchmark que no reporta la calidad de su superficie de herramientas compara un retriever envuelto de doce maneras, no topologías.

1.3 Alcance de las afirmaciones

Establecido: la teoría (§4), verificada por máquina contra distribuciones sintéticas de respuesta conocida; una compuerta de factibilidad (§3.3) validada sobre cuatro escalas de corpus; un generador de corpus cuyo ground truth se re-deriva de forma independiente a partir de los documentos (§3.5).

Medido: todo §6, sobre una campaña de 78 tareas, 12 paradigmas y 3 réplicas que da 2.511 filas y 123,3M tokens (nueve brazos corrieron 67 tareas y tres, `rewoo`, `handoff` y `graph_traverse`, las 78; cero errores de infraestructura, sin juez LLM y con el corrector auditado), más un held-out corrido por el mismo camino de código y una réplica del hallazgo de consenso sobre una segunda familia de modelo. Todo panel comparativo es el rectángulo mecánico del registro, 64 tareas × 8 brazos: las tareas que esos ocho brazos corrieron todas con tres réplicas (§5.3). Cada comparación declara su piso de ruido.

Dos mediciones corren sobre otro modelo. Los tres episodios del ciclo (§6.5) son anteriores a la campaña y corrieron sobre `gpt-5.4-nano`; §6.5.1 declara qué de ellos transfiere a la campaña y qué no. La celda de cadenas acopladas de §6.1.3 y §6.1.4 corrió sobre `gpt-5.6-terra`, porque sobre `nano` daba cero en toda la grilla; §6.1.3 declara los números de `luna` al lado.

1.3.1 El dominio

Lo que se evalúa es capacidad de RAG sobre análisis forense de hechos: un sistema que recibe preguntas del tipo que hace un auditor, un analista de cumplimiento o un investigador sobre un conjunto documental. Qué cuenta figura a nombre de quién, quién le reporta a quién subiendo N escalones, si existe registro de una transferencia, si una designación sigue vigente, si alguien debía escalar y no lo hizo.

El corpus es heterogéneo por construcción, y sus once modos son los modos de falla de ese dominio, no una muestra de dificultad: hecho único verificable, enumeración independiente, cadena acoplada, agregación con cobertura exhaustiva, horizonte desconocido, acción irreversible, vigencia, plantel declarado, ausencia, presuposición falsa y escritura compartida. Ausencia y presuposición falsa son las dos que un benchmark de extracción no suele tener y las que un sistema forense necesita: la respuesta correcta es que el hecho no está, o que la pregunta da por cierto algo que es falso.

Sobre eso se apilan tres anchos (5, 20 y 60 unidades) y entidades con variantes de superficie, anáfora y señuelos: el 36,5% de las menciones son invisibles a una búsqueda del nombre completo y el 100% de los saltos de cadena exige resolver una variante.

La maquinaria no depende del dominio. Sensor estocástico, motor determinista, procedencia tipada, compuerta aritmética y abstención son propiedades de cómo se decide, no de qué se pregunta; §3.5 las instancia sobre otras tres superficies (datos, acciones, gobierno) con el mismo enunciado sobre otro tipo de proposición. Lo que este registro establece es que la separación funciona y qué cuesta en un dominio; que el número transfiera a otro es una predicción, no un resultado.

Fuera de alcance: un selector de calidad validado; ningún resultado sobre benchmarks públicos; ninguna medición fuera del dominio descrito arriba; y un clasificador que recupere los ejes de la ontología desde un request real. Que la ontología separe a los brazos está medido sobre el registro (§6.3.5). Que se la pueda detectar afuera del corpus es la medición que sigue.

1.3.2 Mapa de mediciones

Cada sección empírica corre sobre un corpus, un modelo y un panel, y no son los mismos en todas. La tabla dice cuál es cuál para que ningún número se lea fuera de su rectángulo.

sección	corpus	modelo	panel	qué transfiere entre secciones
§5.2, §6.1, §6.2.1 a §6.2.5, §6.3, §6.4.1, §6.4.2	gold_h1 , 78 tareas, entidades reales	gpt-5.6-luna	rectángulo 64 × 8 para toda comparación entre brazos; registro entero para cobertura y aporte	es la campaña, y de acá salen los números del resumen
§6.1.3, §6.1.4	gold_h1 , modo de cadenas acopladas, 3 tareas	gpt-5.6-terra	9 celdas por brazo	el mecanismo de la intervención; sobre luna los números van al lado
§6.2.6 a §6.2.8	gold_holdout , 26 tareas, semilla nueva	gpt-5.6-luna	rectángulo 24 × 8	el veredicto fuera de muestra del oráculo
§6.4.2	réplica del consenso	gpt-5.6-terra	200 celdas, réplica 0	si el consenso es de los paradigmas o del modelo
§6.4.3, primera parte	gold_transfer , 26 tareas	gpt-5.4-nano	90 celdas, cinco brazos	el mecanismo del recall; las magnitudes no
§6.5	tres mundos de 26 tareas, semillas 47, 61 y 73	gpt-5.4-nano	390 filas cada uno	el mecanismo de cada refutación y la reproducibilidad; las magnitudes no
§6.6	gold_h1	gpt-5.6-luna	616 episodios en seis lotes	la trayectoria de θ
§3.4.2, tabla del dial	registro anterior a la campaña	gpt-5.4-nano	cinco brazos	el porcentaje que A3 se lleva; los absolutos no

1.4 Organización del paper

§2 ubica el trabajo en cuatro tramos cortos: el linaje clásico y el supuesto que cada línea hace sobre su fuente de creencias, los vecinos de selección de paradigmas, los mecanismos con antecedente, y la gobernanza simbólica; el trabajo relacionado extendido está en el Apéndice D. §3 es el motor, abierto con un request real de punta a punta: preliminares y glosario, el plantel, la compuerta aritmética, creencias, procedencia y dial, y qué es plástico y cómo se consolida. §4 es la teoría mínima que el argumento usa: el confinamiento de varianza, la clave de la política, el valor de seleccionar como identidad contable, y el alcance; los resultados formales auxiliares están en el Apéndice B.

§5 es el banco y el método, dicho una vez: medir sin juez, el plantel en una tabla, los paneles y el piso de ruido, las preguntas de investigación. §6 son los resultados en el orden del argumento: la varianza de trayectoria, por qué rutear por nombre no tiene premio, la interfaz de capacidades y ejes, lo que sí paga, el ciclo que reparó el vocabulario, y la trayectoria de maduración de θ . §7 discute qué cambia para cómo se evalúan agentes, la ventana frontera y el impacto amplio. §8 son las amenazas a la validez. §9 concluye y ordena lo que sigue. El Apéndice C lista las diferencias con los borradores anteriores.

2. Trabajo relacionado

Cuatro tramos, en el orden en que el argumento los necesita. Las líneas que el cuerpo sólo cita, búsqueda de workflows, modelos de costo, deferral y agentes que aprenden sin pesos, y la tabla de posicionamiento, están completas en el Apéndice D.

2.1 El linaje clásico, y qué supuesto rompe un LLM estocástico

Las piezas del motor no son nuevas. Cada una viene de una línea con décadas de trabajo, y cada línea asume algo sobre su fuente de creencias que un LLM viola. Lo que sigue es el catálogo de supuestos que hubo que reemplazar.

línea	qué aporta	qué asume sobre la fuente	qué rompe un LLM
Revisión de creencias, AGM (Alchourrón, Gärdenfors y Makinson, 1985)	qué se retracta cuando entra evidencia contradictoria	el postulado de éxito: la creencia entrante se acepta	una fuente que puede equivocarse vuelve inadmisible aceptar por defecto; de ahí la compuerta de admisión
Mantenimiento de verdad con justificaciones: Doyle (1979), ATMS de de Kleer (1986)	cada creencia exhibe la justificación que la sostiene	que la justificación existe y es recuperable	fabrica justificaciones plausibles y sintácticamente bien formadas
Procedencia de primera clase: Buneman, Khanna y Tan (2001); semianillos de Green, Karvounarakis y Tannen (2007)	de dónde vino un valor, separado de cuánto se le cree	que la procedencia se deriva de la operación, no se declara	no hay operación: hay una emisión, y su procedencia no está en la salida
Arquitectura BDI (Rao y Georgeff, 1995)	deliberación separada de ejecución	sensores que no alucinan, un sensor roto se detecta por inconsistencia	alucina consistentemente, así que la inconsistencia no lo delata
Políticas como datos, aprendidas offline: Soar (Laird, Newell y Rosenbloom, 1987), ACT-R	la política es un artefacto inspeccionable, no un peso	que las condiciones de las reglas son observables	las condiciones interesantes las emite el LLM, y entonces la clave de la política hereda su varianza (§6.2.5)
Plasticidad hebbiana (Hebb, 1949)	una asociación entre pares tiene contenido propio sobre sus marginales	que los eventos asociados son eventos, no descripciones de eventos	la traza es lo que el LLM pidió, no lo que ocurrió
Opción de rechazo (Chow, 1970) y predicción selectiva	cubrir menos a cambio de errar menos	que hay una puntuación de confianza calibrada	la confianza declarada por el modelo no está calibrada, y §6.4.2 tuvo que construir una
Argumentación abstracta (Dung, 1995)	una conclusión vale si su prueba sobrevive a los ataques disponibles	el marco se define sobre una relación de ataque dada; instanciarlo exige construirla	el espacio de respuestas plausibles y falsas no es construible, así que la admisión se decide por procedencia y no por supervivencia

La lectura crítica, en una línea por columna. Las tres primeras líneas aportan la maquinaria de creencias y las tres suponen sinceridad, existencia de justificación o derivabilidad de procedencia, y las tres suposiciones caen a la vez cuando la fuente puede inventar con forma correcta. BDI aporta la separación que este trabajo hereda y su detección de sensores rotos no funciona sobre un LLM que alucina de forma consistente. Soar aporta la política como dato y su supuesto de condiciones observables es lo que §6.2.5 mide que falla. Chow aporta la abstención y su supuesto de confianza calibrada es el que §6.4.2 reemplaza por una curva medida.

Qué se hizo con cada supuesto roto, línea por línea. Donde AGM asume que la creencia entrante se acepta, el motor pone una compuerta de admisión: una proposición entra a la base sólo con su

procedencia declarada, y la procedencia decide qué puede gobernar. Donde el mantenimiento de verdad asume que la justificación existe, el motor no le pide al modelo que justifique; exige que el código pueda exhibir de qué unidad del material salió el valor, y si no puede, el valor no se emite (Teorema 2). Donde la procedencia clásica se deriva de la operación que produjo el dato, acá el dato llega como una emisión del modelo, sin operación de la que derivar nada, así que la procedencia se declara en el punto de entrada y se verifica contra el índice. Donde BDI detecta un sensor roto por inconsistencia, acá el sensor alucina de forma consistente, y la detección se reemplaza por tipar la salida antes de usarla y por el acuerdo entre estructuras de control distintas (§6.4.2). Donde Soar asume condiciones observables, las condiciones de la política pasan a exigirse `COMPUTED`, porque una clave elicitada convierte la tabla de decisión en una variable aleatoria (§6.2.5). Donde Hebb asocia eventos, el motor asocia lo que ocurrió, contado por el arnés, y no lo que el modelo dijo que pedía. Donde Chow asume una confianza calibrada, la confianza deja de leerse del modelo y pasa a calibrarse contra el registro, y §6.4.2 construye esa curva. Y donde Dung necesita el espacio de ataques, la admisión se decide por procedencia y no por supervivencia a ataques que nadie puede enumerar.

2.2 Selección de paradigma en tiempo de inferencia

Select-then-Solve entrena un ruteador liviano sobre embeddings para elegir un paradigma por tarea [arXiv:2604.06753]. FlowBank construye un portafolio offline y selecciona por consulta [arXiv:2606.11290]. TRACE-Router rutea a granularidad de traza de tarea y no de solicitud [arXiv:2607.22465]. Uno-Orchestra aprende una política conjunta de descomposición y despacho [arXiv:2605.05007].

Todos operan a cobertura uno: toda tarea recibe un paradigma. §4.3 sostiene que ésa es la restricción vinculante, más que la capacidad del modelo, y ninguno reporta una curva de riesgo-cobertura. Dos detalles de Select-then-Solve, verificados contra el paper completo: su oráculo es un máximo empírico por tarea, y sus propias limitaciones anotan que la muestra es fija, sin re-muestreo entre seeds. Es el caso para el que el piso de ruido de §6.2.3 existe, y este trabajo lo aplica sobre su propio registro. Y la maquinaria de deferral existe al lado, sin haber cruzado: eDAct difiere decisiones individuales a un modelo más grande sobre un umbral calibrado de incertidumbre [arXiv:2604.07036]; el ruteo por descomposición de incertidumbre unifica abstención y ruteo con garantías distribution-free, para clasificadores [arXiv:2605.07805]; y el meta-ruteo composicional entrena un ruteador interpretable sobre features textuales y nombra un confidence gate con fallback a ruteo estático como trabajo futuro sin evaluar [arXiv:2608.00106]. Nadie aplica abstención tasada como curva riesgo-cobertura a la selección entre topologías de control (buscado 2026-08-26 y 2026-09-01); la ventana se está cerrando a la vista.

Rutear entre paradigmas de recuperación por request existe. Adaptive-RAG entrena un clasificador de complejidad que manda cada consulta a no recuperar, a un paso de RAG o a varios pasos [arXiv:2403.14403]; Self-RAG decide por request cuándo recuperar y cuándo abstenerse de citar [arXiv:2310.11511]; FLARE recupera activamente cuando la generación pierde confianza [arXiv:2305.06983]; Self-Route deja que el modelo elija entre RAG y ventana larga por auto-reflexión [arXiv:2407.16833], que es la comparación de §7.2 hecha y medida, con un selector elicitado; y AutoMix encadena modelos con auto-verificación como detector [arXiv:2310.12963]. Ninguno reporta la curva riesgo-cobertura de su selector, ninguno declara capacidades del brazo, y todos los selectores son elicitados o entrenados sobre el texto. Lo que este trabajo agrega es exigir que la clave del ruteo sea `COMPUTED` (§4.2), medir el premio contra un piso de sesgo del máximo, y poder no elegir.

La cascada de §B.5 tiene una línea propia, anterior y sobre modelos en vez de paradigmas, y hay que nombrarla. FrugalGPT encadena modelos de menor a mayor costo con un puntaje de aceptación por escalón y reporta ahorros grandes a igual calidad [arXiv:2305.05176]; el ruteo híbrido entrena un clasificador de dificultad para mandar la consulta al modelo chico o al grande [arXiv:2404.14618]; RouterBench fija el banco para comparar ruteadores de modelos [arXiv:2403.12031] y RouteLLM aprende el ruteador desde datos de preferencia [arXiv:2406.18665]. Las cuatro deciden entre modelos con el mismo control de flujo. §B.5 agrega la partición por verificabilidad, cuándo la cascada precede a la selección porque existe un detector barato, y la observación de que un benchmark de coincidencia exacta trae ese detector por construcción (§8.2). Y el detector de FrugalGPT es un puntaje aprendido sobre la respuesta, es decir, elicitado; acá el detector tiene que ser `COMPUTED` u `OBSERVED` para disparar la cascada, por la misma razón que §6.2.5 le exige a la clave de la política.

2.3 Capacidades declaradas contra descripciones de habilidades

El vecino de §6.3 del lado de la ingeniería es el ruteo por habilidades: agentes que publican una tarjeta con lo que saben hacer y un orquestador que despacha por esa tarjeta, como en el protocolo Agent2Agent y sus antecesores en sistemas multiagente. La diferencia es de procedencia, y es la misma de todo el paper. Una tarjeta de habilidades es texto que el agente o su autor escriben sobre sí mismos, se lee con un modelo o con un matcher semántico, y nada la verifica contra la conducta. Una capacidad de §6.3 es un booleano declarado desde el código del brazo, con el sitio donde se ve y con una auditoría que contrasta lo declarado contra lo corrido. La primera es `ELICITED` sobre el propio agente, que es la clase de evidencia que los contratos de delegación de §2.5 midieron que selecciona a los peores; la segunda es `COMPUTED`. Y la tabla de exigencias va del eje de la pregunta a la capacidad, nunca al nombre del agente, así que predice sobre un brazo que todavía no existe. Búsqueda fechada 2026-09-01: no se encontró trabajo que declare capacidades de control de flujo desde el código y las someta a leave-one-arm-out.

2.4 Tres mecanismos con antecedente

Tres mecanismos que este trabajo usa tienen antecedente directo, cada uno en un trabajo distinto (leídos completos el 2026-08-28), y se acreditan.

trabajo	qué establece
EnvProbe, <i>Ask the World Before Acting: Budgeted Environment Probing for World-Model Calibration</i> (arXiv 2606.31422)	un operador de sondeo con presupuesto cuyo único propósito es reparar una tabla de creencias estructurada. Es la sonda de §3.4, mecanismo por mecanismo
Kintsugi, <i>Learning Policies by Repairing Executable Knowledge Bases</i> (arXiv 2605.09487)	ediciones a un artefacto ejecutable tipado, filtradas por un verificador, con las fallas diagnosticadas y localizadas en ediciones candidatas. Es la consolidación de §3.5 con su guarda de promoción
ProvenanceGuard, <i>Safeguarding LLM Agents from Misalignment through Provenance Analysis</i> (arXiv 2607.01236)	la desalineación como si una llamada propuesta está sostenida por evidencia trazable en el contexto. Es el piso de procedencia de §3.4.1 sobre las acciones

Y el área está lo bastante poblada como para tener survey: *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents* (arXiv 2606.04990).

Sondeo con presupuesto sobre un estado de creencias tipado, ediciones de política filtradas por verificador, y pisos de procedencia sobre acciones están establecidos por esos tres trabajos, y este paper los usa sin reclamarlos.

Lo que queda es una conjunción, enunciada por lo que excluye: una capa de decisión que (a) elige qué topología de control de flujo correr, por request, de un catálogo de ellas, con una clave `COMPUTED` (ninguno de los tres rutea entre topologías, y los que sí lo hacen en RAG, Adaptive-RAG y Self-Route, lo hacen con una clave elicitada o entrenada sobre el texto, §2.2); (b) puede abstenerse, con la curva riesgo–cobertura reportada en vez de la utilidad de lo que eligió contestar; y (c) poda por aritmética sobre el presupuesto declarado antes de cualquier inferencia. Sacando cualquiera de las tres, el resto queda cubierto por el trabajo de arriba.

Y uno de los cuatro nos apoya, desde un lugar al que no llegamos. *Trace2Policy: From Expert Behavior Traces to Self-Evolving Decision Agents* (arXiv 2606.10457) reporta un despliegue en producción de 22 días sobre 3.349 casos resueltos, y encuentra que a lo largo de cinco escalas de modelo, la varianza atribuible a la versión de la regla supera a la atribuible a la elección de modelo. Independiente, a escala de producción, y lo más parecido a corroboración externa que tiene esta línea: la estructura decide más que el modelo.

2.5 Gobernanza simbólica sobre inferencia probabilística

Es la vecindad más cercana a §3.4 y hay cuatro trabajos dentro. Los dos primeros se leyeron completos el 2026-08-26; la tabla dice qué aporta cada uno y qué de §3.4 queda afuera de su alcance.

trabajo	qué aporta	qué agrega este trabajo sobre eso
SCL / Soft Symbolic Control [arXiv:2511.17673]	gobernanza sobre inferencia probabilística, partida en <i>Regulation</i> (un metaprompt persistente) y <i>Control</i> (un runtime determinista sobre el historial del turno: llamadas duplicadas, conteo de errores, profundidad de ciclos), más clasificación de riesgo por acción	la garantía graduada por solicitud y derivada de creencias sobre el contenido, el espacio de planes acotado por nivel, y la calibración medida de la confianza
MINERVA / HADD, Jaime y Errecalde (2026) [Zenodo 10.5281/zenodo.20003407]	el antecedente de vocabulario más cercano: el LLM confinado a sensor tipado, la cognición determinista sobre la base de creencias («mismo estado → misma acción»), una compuerta de admisión en la frontera de percepción, y el encuadre neurosimbólico de Kautz (Tipo 2, simbólico que envuelve a lo neuronal) [Kautz, 2022]. Es una arquitectura declarada para dominios regulados	la medición de cada invariante con su costo, la procedencia como jerarquía con semántica de admisibilidad, la garantía graduada por solicitud, y la plasticidad de §3.5
Motores de creencias: Nous [arXiv:2606.22030], MemIR [arXiv:2605.25869], Eywa [arXiv:2605.30771], HEP [arXiv:2607.09195]	acotar la confiabilidad por procedencia del canal; tipar la memoria para impedir colapso de fuentes; promover hechos sólo tras validadores contra evidencia inmutable; hacer auditable la evolución de hipótesis	la jerarquía de procedencia que filtra la promoción, y la partición proponer/puntuar de §3.5 como mecanismo para el jardín de senderos que se bifurcan que [arXiv:2607.01507] diagnostica
Contratos de delegación e identidad atestiguada [arXiv:2603.18043]	el vecino más cercano del lado del ruteo, y empírico: rutear sobre calidad auto-reportada selecciona a los peores delegados y rinde peor que al azar (0,55 contra 0,68); el remedio son contratos que acotan autoridad más identidad reclamada contra atestiguada, y el brazo atestiguado llega a ruteo casi óptimo	la procedencia como orden sobre tipos de evidencia que una regla lee, el piso sobre acciones irreversibles, y la abstención tasada como curva riesgo-cobertura

Dos observaciones que salen de la tabla. La primera corrobora la disciplina del LLM desde una dirección adversarial: la razón de este trabajo para rechazar la auto-evaluación es epistémica (una afirmación sobre su propia suficiencia no supera `ELICITED`) y la de los contratos es que un delegado tiene incentivo a inflar; las dos llegan a la misma prohibición, y `reclamada / atestiguada` es `ELICITED / OBSERVED` restringido a una proposición.

La segunda achica lo que se puede afirmar: «procedencia + ruteo + contratos» ya está ocupado como frase, así que la conjunción se enuncia por lo que excluye. Queda sin antecedente el conjunto de: un retículo de procedencia sobre evidencia, un piso que filtra acciones irreversibles con él, la abstención

tasada como curva riesgo-cobertura medida, y el mismo cálculo sobre factibilidad, control y contenido. Cualquier término suelto tiene antecedentes.

3. El motor

3.0 Un request, de punta a punta

Antes de las definiciones, una decisión real del registro de la campaña, para que se vea qué hace cada pieza. La tarea `d1-002-w48` pregunta: «*On what date did Lucia Vallejos transfer the settlement account to the successor account? Report the date.*» Declara 60 unidades de material, 483 mil tokens en total, un presupuesto de 40 mil tokens, respuesta de cardinalidad singular, cobertura suficiente, sin acción irreversible y sin oráculo barato. La respuesta correcta es que no hay ninguna transferencia registrada: la pregunta presupone algo falso.

Primero decide la aritmética, sin gastar un token. Tres de los doce brazos quedan podados porque necesitan el material entero en una llamada o una llamada por unidad más un reduce que no entra en el presupuesto: `direct`, `extract_compute` y `streaming_scan`. Nueve pueden correr.

Después se arma la clave, la tupla de ejes con la que la política indexa su tabla (§4.2). Cuatro de sus cinco ejes son `COMPUTED` desde lo que el request declara o desde el material: muchas unidades, sin oráculo barato, sin continuidad de clave entre unidades, y sin un literal entrecomillado en la pregunta. El quinto, el acoplamiento entre unidades, lo emite el LLM sobre una muestra del material, lleva procedencia `ELICITED`, y es el eje del que §4.2 se ocupa. La región, el valor que toma la clave, es `many/no_oracle/loose/flat/no_lit`, la más poblada del corpus, con 24 tareas.

Después el dial, el nivel de garantía del request, de A0 a A3 (§3.4). La tarea no declara acción irreversible ni escritura compartida, el caller no pidió más, y no hay piso aprendido para la región: el nivel efectivo es A1, el estándar de producción, y no excluye ningún patrón.

Después la política. θ , la tabla consolidada de §3.5, en su versión 6 y firmada, tiene 24 episodios por brazo en esa región; el mejor par confiado es `dag_strategy` con `0,833` de utilidad media contra `0,775` del fallback `react` (el brazo por defecto cuando θ no gobierna), un margen de `0,058` que supera el umbral, así que θ gobierna esa región y elige `dag_strategy`. Con el objetivo de costo en vez del de calidad, la señal computada elegiría al más barato que empata en la región, `rewoo`.

Y lo que pasó. Los nueve brazos factibles contestaron que no hay transferencia registrada y sacaron utilidad `1,000`; lo que los separa es el costo, de 14.641 tokens en `rewoo` a 427.975 en `react`. El artefacto de la decisión lleva la acción, el brazo, la región, los tres patrones excluidos y por qué, el nivel del dial, la versión y la firma de θ , y un digest; con eso y la base de creencias registrada, la decisión se re-deriva sin volver a llamar al modelo. Las secciones que siguen definen cada una de esas piezas y §6 mide qué cuesta cada una.

3.0.1 La figura, y qué decide cada carril

Cómo se decide un request

la frontera entre los dos carriles es la única decisión de diseño que el registro muestra que importa



Una decisión que cruza al carril de abajo se lleva el determinismo con ella: medido, el ancla de una caminata elegida por el modelo dio u=1,000 / 0,000 / 0,000 con la misma huella.

Figura 2. Cómo se decide un request, nodo por nodo. En el carril de arriba decide el código. Los sensores (1) computan ejes de la pregunta y del material en forma cerrada, sin modelo. Las creencias tipadas (2) guardan cada proposición con su procedencia, de menor a mayor ASSUMED , ELICITED , OBSERVED , COMPUTED . El portón de factibilidad (3) es aritmética pura: qué paradigmas entran en el presupuesto. Las capacidades exigidas (4) traducen la ontología de la pregunta a capacidades, no a nombres de paradigma. Los brazos candidatos (5) son los que tienen todas las capacidades exigidas. El dial de garantía (6) lo declara el caller y jamás se infiere del texto. Elegir o abstenerse (7) toma, entre los candidatos que empatan, el más barato. Y EXPLAIN (8) registra qué se creyó, con qué procedencia y qué se pudo. En el carril de abajo el modelo contesta sólo dos preguntas: qué dice esta unidad, y hacia dónde sigue el rastro. Su salida se tipa antes de usarse. El código lleva el bucle: cuántas vueltas, qué índice, cuándo parar.

La figura es el argumento del paper en una imagen, y la frontera entre los dos carriles es la única decisión de diseño que el registro muestra que importa.

Toda decisión que cruza al carril de abajo se lleva el determinismo con ella, y eso está medido (§6.1.4). La guarda que cierra el ciclo, tipar la salida del LLM antes de usarla, salió de un caso concreto: el modelo emitía 'M. Arrieta settlement account' y esa cola arrastraba la consulta a clasificarse como prosa, mandándola al índice equivocado. La regla era correcta y la entrada estaba sucia.

lo que decide el código	lo que emite el modelo
cuántas vueltas dar (del largo declarado en la pregunta)	qué dice esta unidad
qué índice usar (léxico para entidad nombrada, híbrido para prosa)	hacia dónde sigue el rastro
qué unidad es el ancla, y cuál el término de la cadena	qué hecho aporta lo leído
si hay evidencia suficiente para responder, o si hay que abstenerse	la redacción de la respuesta

3.1 Preliminares y glosario

Una tarea t aporta una pregunta, un conjunto de *unidades* (documentos), un presupuesto declarado de tokens, y flags de irreversibilidad y escritura de estado compartido. Un paradigma $p \in P$ es una estructura de control que puede llamar herramientas y debe emitir una respuesta. La calidad $q(t, p) \in [0, 1]$ es F1 de conjuntos contra un oráculo exacto. El costo $c(t, p)$ es el total de tokens sobre cada llamada que el paradigma hace.

El mejor paradigma fijo es $p^* = \operatorname{argmax}_p E_t[u(t, p)]$. El oráculo es $E_t[\max_p u(t, p)]$, y la brecha del oráculo es su diferencia. La utilidad es calidad neta de una preferencia de costo:

$$u(t, p) = q(t, p) - \lambda \cdot (c(t, p) / \min_{p'} c(t, p') - 1)$$

Normalizar por el paradigma más barato en esa tarea hace a λ interpretable, la calidad que uno está dispuesto a cambiar por un múltiplo extra del costo mínimo, y $\lambda=0$ recupera calidad pura exactamente. No se elige ningún λ : la calidad y el costo crudos se guardan sin modificar y el trade-off se aplica en tiempo de análisis, de modo que los resultados se reportan como función de λ y no bajo un supuesto sobre λ . Convención de §6: toda u reportada usa $\lambda = 0$, calidad pura con el costo en su propia columna, salvo donde se indique otro λ .

Dos términos más que §6.2 usa. Sobre un panel de tareas y brazos, la utilidad se descompone como $u(t, p) = \mu + \alpha(t) + \beta(p) + \gamma(t, p) + \epsilon$, donde α es la dificultad de la tarea, β la calidad del brazo, γ la interacción y ϵ el ruido entre réplicas de la misma celda. Una celda es un par tarea $\times$ paradigma y no se usa con otro sentido; las once clases de falla del corpus (§1.3.1) se llaman modos. Y una región es un elemento de la partición del espacio de features que la política usa como clave (§4.2, §3.5).

El glosario que el resto del paper usa. La procedencia de una creencia es cómo se obtuvo, en cuatro niveles ordenados: **COMPUTED** (una función pura del request y del material, credencia 1,0), **OBSERVED** (medido ejecutando una sonda sobre el material), **ELICITED** (el modelo lo afirmó, sujeto a calibración) y **ASSUMED**. La credencia es cuánto se le cree, y es un campo distinto. Una sonda es una acción de la política que lee una unidad del material para medir una propiedad de la tarea antes de decidir, con un costo declarado, y devuelve una creencia **OBSERVED** o queda sin resolver. Un episodio es una decisión registrada con su clave, su brazo, su utilidad y su costo (Definición en §3.5).

Y los términos del banco. Un brazo es un paradigma visto desde el banco, una columna del registro. Una réplica es una corrida repetida de la misma celda con la misma huella; la campaña corre tres por celda. El rectángulo es el panel de 64 tareas $\times$ 8 brazos sobre el que se hace toda comparación entre brazos: las tareas que esos ocho corrieron todas con tres réplicas (§5.3). Los contendientes son los tres brazos cuya utilidad media queda a menos de cinco centésimas del mejor fijo, **react**, **dag_strategy** y **reflection** (§6.2.3). El piso de ruido es la brecha de oráculo que brazos idénticos mostrarían por azar, porque el máximo de estimaciones ruidosas está sesgado hacia arriba; se estima con pseudo-brazos (§6.2.3). pass^k es la probabilidad de que las k réplicas de una celda acierten todas (§6.1.2). Un medidor es una función que computa un eje de la clave desde el request o desde el material, sin llamar al modelo. El dial es el nivel de garantía de un request, de A0 a A3 (§3.4).

3.2 Los doce paradigmas, en una línea cada uno

Un paradigma es una estructura de control, no una variante de prompt. Los tres que gobiernan los resultados tienen origen publicado: ReAct, el bucle abierto de razonar y llamar herramientas [arXiv:2210.03629]; Reflexion, borrador, crítica y revisión [arXiv:2303.11366]; ReWOO, plan de dataflow previo y una sola llamada de resolución [arXiv:2305.18323]. Los demás son implementaciones de este banco contra un modo de falla declarado.

paradigma	qué hace, operativamente	quién decide la próxima llamada
direct	una llamada con todo el material adentro	nadie; una sola llamada
react	bucle abierto: el modelo elige la herramienta, lee el resultado, sigue o contesta	el modelo, cada vuelta
reflection	como react , más un ciclo de crítica y revisión de la respuesta	el modelo
dag_strategy	descompone en sub-preguntas, las ejecuta sobre un pizarrón compartido, verifica y replanifica	el modelo planifica; el código fija topes de iteración y replanificación
rewoo	planifica todas las llamadas de herramientas de una vez, las ejecuta sin modelo, y resuelve con una llamada	el código, tras el plan
handoff	agentes con alcance propio sobre partes del material; el código autoriza una transferencia cuando la referencia aparece literal en otro alcance	el código
supervisor	un orquestador despacha sub-agentes con ventana recortada de ocho unidades, uno por vez, tras ver lo que volvió	el modelo, entre despachos
gist_reader	una tabla determinista de resúmenes cortos de todas las unidades, y lecturas completas dirigidas desde ella	el modelo elige qué leer; la tabla es del código
pointer_chase	ancla por búsqueda y sigue una cadena una unidad por salto, con el largo fijado por la pregunta (Algoritmo 2)	el código; el modelo sólo dice hacia dónde sigue el rastro
graph_traverse	construye un índice de entidades y relaciones y lo recorre	el código, sobre el índice
extract_compute	extrae registros estructurados de cada unidad y computa la respuesta sobre ellos	el código
streaming_scan	recorre las unidades en secuencia con un estado acotado que se arrastra	el código

Un brazo de prompting, `cot` , quedó fuera del plantel por control nulo (abajo), y `plan_execute` y `map_reduce` se retiraron por dominados antes de la campaña; sus registros se conservan y no se corren.

El plantel son doce paradigmas, y se agrupan por de qué es función su costo (que es lo que predice quién sobrevive cuando el material crece) y no por su nombre de familia:

ley de costo	paradigmas	qué los define
estructural	<code>direct</code> , <code>rewoo</code> , <code>graph_traverse</code> , <code>extract_compute</code> , <code>streaming_scan</code>	número fijo de llamadas, pase lo que pase
por vueltas	<code>react</code> , <code>reflection</code> , <code>dag_strategy</code> , <code>supervisor</code> , <code>gist_reader</code> , <code>pointer_chase</code>	el costo escala con cuántas veces vuelven al modelo
por alcance	<code>handoff</code>	el costo escala con cuánto material hay

Cinco existen para cotejar contra grillas publicadas. Los otros siete entran cada uno contra un modo de falla concreto: `rewoo` porque planifica todo de antemano y su costo no depende del alcance; `dag_strategy` porque una comparación que omite la topología más elaborada disponible está sesgada a favor de las simples; `handoff` y `supervisor` porque reparten alcance entre sub-agentes de dos maneras distintas; `gist_reader`, `pointer_chase` y `graph_traverse` porque cada uno interpone una representación intermedia más chica que el material (un resumen, un puntero, un índice de entidades) y §6.1.1 mide qué cuesta eso.

La ingeniería de prompts no es un paradigma y no está en el plantel de decisión. Un brazo que sólo cambia el fraseo quedó como control nulo: en toda celda medida da la misma utilidad que la llamada directa y nunca cuesta menos. Los paradigmas se distinguen por estructura de control de flujo, jamás por redacción, y esa regla es lo que hace que las mejoras de §6.1.4 sean transferibles en vez de anecdóticas.

3.3 La compuerta aritmética de factibilidad

Algoritmo 1. La compuerta, entera. No hay ninguna llamada al modelo y no hay ningún parámetro aprendido: es una desigualdad sobre cantidades que la tarea ya declara.

ALGORITMO 1 Portón de factibilidad

Entrada : tarea t con unidades U , presupuesto B tokens, plantel P

Salida : $P' \subseteq P$, los que PUEDEN correr

```
1   $N \leftarrow |U|$                                 ▷ cardinalidad
2   $C \leftarrow \sum_{u \in U} \text{tokens}(u)$           ▷ contenido total
3   $A \leftarrow B - \text{reserva\_de\_conversación}(t)$   ▷ lo disponible de verdad
4   $P' \leftarrow \emptyset$ 
5  for cada  $p \in P$  do
6    según ley_de_costo( $p$ ):
7      alcance      :  $\text{coste} \leftarrow C$ 
8      estructural  :  $\text{coste} \leftarrow C / \text{ramas}(p)$ 
9      vueltas      :  $\text{coste} \leftarrow \text{unidad\_máxima}(U) \cdot \text{techo\_de\_llamadas}(p)$ 
10   if  $\text{coste} \leq A$  then  $P' \leftarrow P' \cup \{p\}$ 
11  return  $P'$ 
```

La línea 6 es la que separa dos modos de falla que se confunden de rutina: a un paradigma que reparte por unidad lo acota N y no C , y a uno que lee todo lo acota C y no N . Un corpus con pocas unidades enormes poda a los primeros; uno con muchas unidades chicas, a los segundos. Sin la línea 6 los dos casos se reportan como «no alcanzó el contexto».

Los corpus de esta sección son la escalera de escala, no el corpus de medición. `gold_v2`, `gold_wide` y `gold_deep` existen para barrer cuatro órdenes de magnitud de material (16k a 1,27M tokens) y mostrar que la compuerta de factibilidad se comporta como su aritmética predice en los cuatro. Los resultados de §6 corren sobre otro corpus, con entidades reales y variantes de superficie. La factibilidad no depende del modelo ni del corpus: es una desigualdad sobre cantidades declaradas, y por eso se puede validar en un lugar y aplicar en otro.

3.3.1 El chequeo

Que p pueda correr t depende de cantidades que t ya declara. Con n unidades, contenido C tokens, presupuesto B , y una asignación $A = 0,6 \cdot B$ que deja lugar para la conversación:

paradigma	restricción vinculante	infactible cuando
Direct, CoT	un prompt con todas las unidades	$C > A$
Map-Reduce	una llamada por unidad, más un reduce sobre todos los parciales	$n > 80$ o $n \cdot f > A$
DAG	sub-preguntas $\times$ iteraciones $\times$ replanificaciones	llamadas proyectadas > 200
ReAct, Reflection	tope propio de iteraciones; lee selectivamente	nunca

donde f es el tamaño proyectado de un hallazgo. Sin llamada al modelo, sin estadística, sin aprendizaje.

3.3.2 Separa dos modos de falla que se confunden de rutina

Aplicado a 168 tareas sobre cinco corpus del mismo generador, cada tarea con su propio presupuesto declarado:

corpus	tokens/unidad	unidades/tarea	contenido máx	Direct, CoT	Map-Reduce
gold_v2	232	60	16k	39/39	39/39
gold_v3	2.147	60	152k	12/39	39/39
gold_wide	264	500	135k	20/32	18/32
gold_deep	7.161	60	483k	2/26	26/26
gold_xl	2.499	500	1.272k	8/32	18/32

Tareas factibles sobre el total. Los otros cuatro paradigmas son factibles en 168/168.

Agregado sobre paradigmas, el espacio de planes admisibles se contrae monótonamente con la escala, 273/273 celdas en gold_v2, 186/224 en gold_wide, 134/182 en gold_deep, 162/224 en gold_xl: de 100% admisible a 72%, íntegramente por aritmética y antes de gastar un token. Un selector que opere sin esta capa tendría que aprender que ese cuarto del espacio es inalcanzable, un episodio fallado a la vez.

El par decisivo es gold_wide contra gold_deep. gold_deep lleva 3,6× más contenido, y Map-Reduce pasa de 18/32 factibles a 26/26 mientras Direct y CoT se derrumban a 2/26. El tamaño total no predice la factibilidad de Map-Reduce; la cardinalidad sí. Con 483k tokens en 60 unidades corre, porque nunca las tiene juntas. Con 135k en 500 unidades no: 501 llamadas, y un reduce que concatena 500 hallazgos. Tratar su límite como un límite de contexto lleva a descartarlo exactamente donde funciona.

El mismo tope de acumulación aplica al blackboard del DAG, que renderiza cada hallazgo dentro de cada prompt de sub-agente.

Cuán fuerte muerde la restricción de leer-todo es fácil de subestimar. En gold_deep una tarea de una sola unidad es infactible para Direct, porque un documento son 8.075 tokens contra una asignación de 4.800. La restricción real es que la pieza más chica que se puede direccionar ya no entra, no que el corpus sea grande, y ninguna cantidad de lectura selectiva cambia eso para un paradigma cuyo único movimiento es leer.

Una segunda lectura. Los cuatro paradigmas selectivos son factibles en las 168 tareas. Es una propiedad real, acotan sus propias iteraciones y leen a demanda, pero también acota lo que esta capa puede hacer. La factibilidad restringe sólo a los paradigmas que retienen material en contexto o se abren por unidad; no ofrece ninguna protección contra un paradigma selectivo gastando a través de un corpus de 1,27M tokens. Esa protección tiene que venir de un presupuesto, no de aritmética sobre el corpus, y §6.1 muestra por qué hace falta: los selectivos son justamente los que varían cincuenta veces en costo.

3.3.3 Por qué esto va delante del problema de aprendizaje

La factibilidad es determinista, gratis, y aguas arriba de todo lo demás. Poda el espacio de planes antes de cualquier selección, aprendida o no. Para producción, saber el largo y declinar es lo que separa un sistema acotado de uno desbocado.

Medido. Sobre cuatro celdas de gold_deep, Direct quedó podado en tres y corrió en una, donde sacó 1,000. Registradas como respuestas equivocadas, esas tres dejarían su media en 0,250, de mejor a peor del

plantel, por un artefacto de registro. Registradas como infactibles, el enunciado es el correcto: *el mejor donde puede correr, no disponible donde no*. La misma capa que protege al estudio de una conclusión falsa es la que un sistema en producción necesita para declinar en lugar de fallar.

3.4 Creencias, procedencia y garantía por solicitud

El determinismo pleno no está disponible con un LLM, y perseguirlo excluyendo al modelo de la decisión descarta información que el modelo tiene. La inversión tiene el vocabulario que §2.5 acredita y el linaje que §2.1 tabula: el LLM sólo emite proposiciones tipadas, una capa simbólica determinista decide sobre la base de creencias, y la garantía toma la única forma que puede tener:

La misma base de creencias da la misma decisión, con la base registrada. Es la estabilidad de la decisión respecto de esa base. «El mismo prompt da la misma respuesta» es falso con un LLM y no se promete.

Lo que esta sección agrega es la epistemología de la creencia misma. En los antecedentes la procedencia es un campo de trazabilidad; en este trabajo carga el peso de la decisión como jerarquía tipada: `COMPUTED` (una función pura, credencia 1,0) > `OBSERVED` (medido ejecutando una sonda) > `ELICITED` (el modelo lo afirmó, sujeto a calibración) > `ASSUMED`. Una regla puede exigir una procedencia mínima, así que una acción irreversible puede restringirse a evidencia computada y observada: *la opinión de un modelo de que una acción es segura no es evidencia admisible para tomarla*.

La garantía es entonces una propiedad de la solicitud, no del sistema. Un modo global hace que todo el tráfico pague por la solicitud más estricta. Cuatro niveles graduán el piso de procedencia, si θ puede aprender online, si la corrida es reproducible desde caché, y qué patrones son admisibles, el nivel certificado excluye topologías cuyo flujo de control es no acotado, porque sus modos de falla no son enumerables, aunque a menudo sean mejores. El piso se deriva de creencias sobre la solicitud: quien llama puede pedir más y nunca menos. La derivación misma aprende, offline: una categoría de solicitudes cuyas afirmaciones elicítadas son rechazadas repetidamente por la compuerta es una categoría cuyo piso sube, las estadísticas de rechazo son evidencia sobre la clase de solicitud, y consumirlas cierra el bucle sin ajustar jamás nada adentro de una solicitud.

Tres propiedades lo vuelven seguro de correr, y las tres están impuestas en código.

El evento es tipado, no parseado. Un rechazo lleva su motivo como valor, la procedencia que se tenía contra la que la regla exigía, de modo que la estadística cuenta el evento. Contarlo emparejando subcadenas de la explicación mediría el fraseo, y el fraseo es prosa que se reescribe. Sólo cuentan los rechazos por PROCEDENCIA insuficiente: una creencia rechazada por credencia baja, o por tener el valor equivocado, es el sistema funcionando, y no dice nada sobre el régimen de evidencia de la clase.

La guarda es replicación, no utilidad. El registro se parte por tarea; una mitad propone las regiones cuyo piso debería subir, y el piso se instala sólo si la otra mitad (solicitudes que la propuesta nunca vio) dice lo mismo de manera independiente. La utilidad sería el criterio equivocado. Subir un piso hace que el sistema exija evidencia medida donde habría actuado sobre una afirmación, lo que cuesta tokens y sólo puede bajar la utilidad medida en el corto plazo. Un piso de gobierno puntuado por la utilidad que produce es un piso que nunca sube.

El aumento es acotado y monótono. Se detiene en accountable y nunca llega a certified, porque certified además restringe qué patrones pueden correr y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad, un piso que aprende no puede quedar habilitado a descalificar una topología. Y nunca baja solo: la ausencia de rechazos después de que un piso sube es justamente lo que ese piso se instaló para producir, así que leer esa ausencia como motivo para bajarlo sería una oscilación puesta en el diseño.

Los pisos aprendidos viajan en el bundle de política firmado, así que nada puede elevar el nivel de una solicitud salvo por el mismo camino de promoción que recorre θ . Verificado: una región cuyos rechazos replican a través de la partición recibe su piso; una región que califica sólo en la mitad que la propuso, no.

Credencia y tamaño de efecto no deben confundirse. Un margen aprendido es una creencia *certera* sobre un efecto *grande*, credencia 1,0, valor 0,9, y codificar la magnitud como credencia reporta un hecho computado como incierto, destruyendo la distinción para la cual existe el motor.

3.4.1 Quién fija el dial

La pregunta parece de gobernanza y es de diseño. Si el nivel de garantía lo elige el llamador, un llamador apurado lo baja; si lo elige el sistema, el llamador no puede pedir más rigor del que el sistema cree necesario. Ninguna de las dos.

$$\text{nivel efectivo} = \max(\text{pedido}, \text{piso de creencias}, \text{piso aprendido})$$

fuelle	quién la produce	qué puede hacer
pedido	el llamador, en el request	subir, nunca bajar
piso de creencias	<code>required_floor(Γ)</code> sobre las creencias <code>COMPUTED</code> del request	subir, y no se puede desactivar
piso aprendido	<code>θ.floors[región]</code> , dentro del bundle firmado	subir, y sólo si está promovido

Proposición 1. Sea f una composición de niveles que satisface dos condiciones: (i) cada fuente sólo puede endurecer, es decir $f(x_1, \dots, x_n) \geq x_i$ para toda i , y (ii) no endurece sin motivo, es decir $f(x, \dots, x) = x$. Entonces $f = \max$.

Demostración. Por (i), $f(x_1, \dots, x_n) \geq \max x_i$. Sea $m = \max x_i$; por monotonía en cada argumento, que (i) y (ii) implican sobre un orden total finito, $f(x_1, \dots, x_n) \leq f(m, \dots, m) = m$. Luego $f = \max$. ■

Sin (ii) hay otras composiciones que sólo endurecen, por ejemplo la constante `CERTIFIED`; (ii) excluye las que endurecen aunque nadie lo pida. Con `min` o con un promedio, agregar una fuente podría ablandar el resultado, y entonces una fuente nueva sería un riesgo en vez de una garantía.

Por qué el llamador sube y no baja. Conoce cosas que el sistema no: que este request va a un informe regulatorio, que el resultado se publica, que hay un auditor mirando. Nada de eso está en el material. Lo que no puede es pedir menos, porque el piso sale de propiedades del request mismo, `irreversible` eleva a A3, `shared_writes` a A2, y las dos entran como creencias `COMPUTED` declaradas por el caller, nunca inferidas del texto. Un llamador que pudiera bajar el piso podría declarar una acción irreversible y después pedir tratarla como exploratoria, que es la combinación que el piso existe para impedir.

Una cuarta fuente, que es una degradación y no un nivel. A2 admite creencias `ELICITED`, pero sólo una vez que la calibración se ganó; admitirlas antes anula el propósito del nivel. Así que la resolución no baja el nivel: endurece el piso de procedencia dentro del nivel. La misma idea del `max`, aplicada al otro eje.

El dial se evalúa marginalizando sobre sus cuatro posiciones, porque reportar métricas a un dial fijo reporta una política y no un sistema. Marginalizar produjo un hallazgo sobre el propio dial:

Tres de las cuatro posiciones son indistinguibles. A0, A1 y A2 declaran `admissible_patterns = None`, así que el dial no restringe el catálogo hasta A3. Dos de sus tres transiciones no hacen nada en esa dimensión, y toda la diferencia se paga en un solo escalón.

Lo que sí distingue A1 de A2 vive en otros ejes (θ firmada, log de creencias, profundidad de composición, y el piso de procedencia), así que el dial no es inerte ahí: es inerte en la dimensión que esa tabla mide. Decir cuál es cuál es el punto de marginalizar.

3.4.2 La cota nativa del ratchet

El piso de garantía aprendido sólo sube, y lo que hay que acotarle es cuánto daño acumulado puede hacer antes de detenerse, no cuánto oscila. Una cota de varianza no sirve: una secuencia monótona y acotada tiene varianza que tiende a cero por construcción, así que se cumpliría vacuamente. Lo que se acota es un conteo.

Los niveles son `EXPLORATORY < STANDARD < ACCOUNTABLE < CERTIFIED`, y el techo aprendido es el tercero: `CERTIFIED` queda fuera de alcance a propósito, porque ese nivel restringe qué patrones son admisibles y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad.

Proposición 2 (daño total acotado). Sobre R regiones, el número total de eventos de endurecimiento en toda la vida del sistema es $\leq 2R$, sea cual sea la cantidad de ciclos de consolidación.

Demostración. Monotonía: el piso de cada región es una secuencia no decreciente en un conjunto finito y totalmente ordenado, así que cambia a lo sumo tantas veces como niveles haya por encima de su base. No hace falta nada probabilístico. ■

Proposición 3 (la guarda de replicación es fuerte lejos del umbral y débil cerca). Con q la tasa verdadera de rechazo de la región, ocho tareas por split, umbral de cuatro rechazos para que un split proponga (es decir, $\text{Binomial}(8, q) \geq 4$), y dos splits de tareas disjuntos:

q	un split	ambos	$\approx$
0,10	0,0050	0,000025	1 en 39.613
0,25	0,1138	0,01295	1 en 77
0,40	0,4059	0,16477	1 en 6
0,45	0,5230	0,27358	1 en 4

Importa menos de lo que parece, por dos razones estructurales. Cerca del umbral un falso positivo es casi indistinguible de un verdadero (una región cuyo q real es 0,45 efectivamente rechaza casi la mitad de las veces). Y la Proposición 2 acota el daño acumulado pase lo que pase.

La monotonía que vuelve vacua una cota de varianza es la que acota el daño de la propia tasa de falsos positivos: la propiedad que rompe una cota es la que la hace innecesaria.

Lo que cuesta está medido, y corrige cómo se lee la Proposición 2. La medición es del registro `nano` anterior a la campaña, sobre un catálogo de cinco brazos (`react` , `dag_strategy` , `rewoo` , `gist_reader` , `map_reduce`) y los corpus de esa etapa; el porcentaje de catálogo y de utilidad que A3 se lleva es lo que transfiere, los valores absolutos no.

nivel	brazos admisibles	cobertura	u (mejor fijo)
A0 · A1 · A2	5	100%	0,6101
A3	2	40%	0,4221

El ratchet es gratis hasta A2 y cuesta todo de una vez en A3: la única transición con precio se lleva 60% del catálogo y 31% de la utilidad. «A lo sumo dos subidas» invita a imaginar un daño que se acumula despacio; lo medido es lo contrario, una sola transición tiene precio, y ahí es abrupto. Las otras dos son gratis porque no hacen nada. Y el promedio esconde a quien paga: una región pierde $-0,5000$ mientras la media del corpus es $0,0000$.

3.5 Consolidación de la política de control

El aprendizaje es offline y copy-on-write. Los episodios se reproducen en orden de sorpresa y no cronológico, lo que elimina el sesgo de recencia que la actualización online tiene por construcción; las estadísticas se reducen en escala y las entradas sin uso se podan; y una etapa de abstracción busca particiones de features que separen paradigmas mejor que el binning actual.

Dos disciplinas hacen esto seguro. Una política candidata se instala sólo si no regresa sobre episodios retenidos. Y el registro se parte en tres por tarea, una parte propone particiones, una las puntúa, una la toca sólo el guardia de promoción, porque buscar muchas particiones contra un solo holdout es la forma en que detectar una verdad se vuelve confabularla. Una partición descubierta entra con cero episodios, por debajo del piso de confianza, y no puede gobernar una decisión hasta haber ganado evidencia: un sueño es una hipótesis, no un hecho.

Verificado: dado un registro donde la utilidad es independiente de todo atributo, ninguna partición sobrevive la validación.

Dos términos. Un episodio es una tupla (κ, p, u, c) asentada en el registro: la clave del request al momento de decidir (Definición 5.6), el brazo que corrió, la utilidad que obtuvo y su costo, más la procedencia de cada eje de la clave. La política consolidada es la función θ de la Definición 5 materializada como tabla de condiciones sobre ejes de la clave, con un episodio mínimo por celda antes de que una fila pueda gobernar, firmada y versionada. Aprender es, en este paper, producir una θ nueva desde episodios y dejarla entrar sólo si la guarda la deja.

La política de control es código: una tabla de condiciones sobre features, versionada, firmada, con guarda de promoción y evaluada de forma determinista. Ése es el patrón que el título llama *policy-as-code*, y es el que este trabajo usa: la política se ejecuta, se firma, se compara y se revierte como cualquier otro artefacto de código. El patrón no exige un motor externo; la capa de decisión lo implementa sola y no depende de ninguno.

Cómo se llena una fila, en cinco pasos. (1) Los episodios se ordenan por sorpresa, la distancia entre la utilidad obtenida y la que la política vigente esperaba para esa clave, y se reproducen en ese orden. (2) Por cada par (región, brazo) se acumulan utilidad media, costo medio, conteo y tasa de victoria; un par no puede gobernar una decisión hasta tener un mínimo de episodios, hoy ocho. (3) Una etapa de abstracción propone particiones nuevas de los ejes existentes que separen brazos mejor que la partición vigente, sobre la parte del registro que propone; cada candidata se puntúa sobre la parte que puntúa, y sobrevive sólo si su ventaja se sostiene ahí. (4) Un paso de homeostasis reduce en escala las estadísticas viejas y poda las entradas sin uso, para que el registro no crezca sin límite ni fije un ganador para siempre. (5) El bundle candidato se compara con el vigente sobre los episodios retenidos y se instala sólo si no regresa; se firma, y el piso de garantía por región (§3.4) viaja adentro. Los cinco pasos están en `app/consolidation.py` y `app/policy.py` y sus tests; ninguno llama al modelo.

3.5.1 El sistema aprende con los pesos del LLM congelados

El LLM tiene los pesos congelados. No aprende de este despliegue, no guarda nada entre requests, y dos llamadas idénticas no se enteran una de la otra. Y sin embargo el sistema en su conjunto cambia de comportamiento con la experiencia. Ésa es la propiedad, y sale de un mecanismo de tres pasos:

1. Cada decisión deja su huella epistémica. Al registro va la creencia que la disparó, su procedencia, el paradigma elegido y lo que salió. No es un log de texto: es la base de creencias creciendo, con la misma estructura tipada que gobierna una decisión en vivo.
2. La consolidación reproduce ese registro offline (en orden de sorpresa, no cronológico) y emite una política: una tabla de condiciones sobre features, versionada y firmada, que se ejecuta como código determinista.
3. La política entra sólo si no regresa sobre episodios retenidos, y una partición nueva arranca por debajo del piso de confianza: no puede gobernar hasta haber ganado evidencia.

Qué es fijo y qué es plástico, y por qué la frontera está ahí. El diseño tiene dos capas, y la tesis del paper es la frontera entre las dos:

capa	quién la cambia	ejemplos	por qué
reglas y medidores	una persona, con versión y test	qué puede medir el sistema (los medidores: cardinalidad, continuidad, literal, la sonda de acoplamiento), el retículo de procedencia y qué nivel exige cada acción, la compuerta aritmética, la composición por <code>max</code> , la guarda de promoción, la partición proponer/puntuar/promover	son lo que se audita y lo que se promete; una regla o un sentido que el sistema pudiera darse solo sería uno que nadie puede prometer
lo que se acumula	el sistema, desde lo que sensa y registra, sin intervención	las creencias, que entran cuando un medidor las emite, con su procedencia, y de las que la vigente reemplaza a la anterior sin borrarla; las proposiciones mismas, que no están listadas de antemano; la calibración de credencia por proposición; qué brazo admite cada región y con qué evidencia; las particiones de la clave descubiertas sobre los ejes que hay; los pisos de garantía por región	son lo que la experiencia enseña; nadie las define al principio, se acumulan, y cada versión de lo consolidado queda firmada y diffeable

Nada de la segunda fila lo define nadie por adelantado: la base de creencias es una lista que sólo crece, cada proposición aparece la primera vez que un LLM la emite, y la política y las particiones se llenan desde el registro. Lo que sí exige código es un medidor nuevo, un eje que el sistema no sabía medir. Eso es lo que los tres episodios de §6.5 agregaron, continuidad y literal, y por eso los ejecutaron personas: darle un sentido nuevo al sistema es diseño, no plasticidad, y la frontera está puesta ahí a propósito. Lo que el sistema hace solo es lo de la segunda fila, y §6.6 mide si lo hizo.

Así que el aprendizaje vive en la base de creencias y en la política que se destila de ella, nunca en pesos. Y de ahí sale lo que hace a esta forma valer la pena, porque son dos cosas que normalmente se pagan una con la otra:

Un sistema plástico suele ser opaco, y uno auditable suele ser fijo. Un artefacto entrenado extremo a extremo aprende y no se puede leer; una tabla de reglas escrita a mano se lee y no aprende. Poner el aprendizaje en la base de creencias en vez de en pesos da las dos: lo aprendido es un artefacto legible, diffeable, versionado y revertible, y su instalación pasa por una guarda. Se puede preguntar *qué* aprendió el sistema y contestarlo mostrando dos versiones de una tabla.

La plasticidad es una propiedad del diseño, y hereda su forma de una genealogía que §2.1 hace explícita: consolidar episodios en reglas ejecutables es lo que los sistemas de producción hicieron con *chunking* (Soar, ACT-R), y que un par de decisiones tenga contenido propio por encima de sus marginales es plasticidad hebbiana llevada a decisiones en vez de a neuronas. Lo que se agrega es sostenerla cuando quien emite las creencias puede inventar con forma correcta.

3.5.2 Sobre qué superficies

Lo que el motor aprende son políticas, que consolidan offline en un artefacto firmado y versionado y después se ejecutan como código determinista.

superficie	qué se aprende
ruteo	qué paradigma admite una región de features
herramientas	qué índice sirve a una consulta según su clase (§6.1.4)
pares de herramientas	qué asociación <code>tool_i → tool_j</code> predice, que un histograma marginal no captura
handoff	cómo repartir el alcance entre sub-agentes
comportamiento	el piso de garantía sube donde el registro muestra que las aserciones del modelo se rechazan
credencia	calibración por proposición en vez de un interruptor global de confianza

Las dos últimas son las que cambian lo que el sistema *promete*, no lo que elige: un modelo puede estar bien calibrado sobre cardinalidad y ser inútil sobre acoplamiento, y una política de confianza global no puede representar eso. Qué predictores medidos alimentan hoy a cada superficie, y con qué procedencia, está en §6.4.3.

3.5.3 De dónde viene esta forma

El linaje está en §2.1, con la tabla de qué supone cada línea sobre su fuente de creencias y qué de eso rompe un LLM estocástico. Lo que importa para el argumento es la consecuencia de diseño: la procedencia no se deriva, se declara y se verifica, y esa inversión es la que permite que las ocho líneas sigan funcionando cuando la fuente puede inventar con forma correcta.

3.5.4 Una consecuencia afirmable

Más inferencia no compra más ajuste. La consolidación es replay sobre el registro, así que el costo de aprender es cero llamadas: lo que la cuota compra son *episodios*, y el ajuste es gratis sobre los que haya. Eso separa dos decisiones que se suelen tomar juntas, cuánto medir lo gobierna la potencia estadística, cuánto entrenar no lo gobierna nada.

Y es observable mientras una campaña corre. Sobre un registro `nano` anterior a la campaña, con el vocabulario de región de entonces, 516 filas daban 141 episodios sobre 6 regiones, 57 pares (`región`, `paradigma`) con evidencia y 27 con `n ≥ 3`, leído entre dos tandas, sin costo adicional; el registro de la campaña tiene hoy 14 regiones bajo el vocabulario vigente. La potencia estadística la fija el corpus, no la cuota: un par junta un episodio *por tarea* de su región, así que cruzar el piso de evidencia exige esa cantidad de tareas. Gastar más agrega tareas, y sólo cuentan si caen en la región justa.

4. Teoría mínima

Lo que esta sección enuncia es lo que el argumento usa. §4.1 define la propiedad que da nombre al trabajo. §4.2 enuncia la única condición formal que la tesis necesita: sobre qué clase de clave puede aprender una política sin perder la garantía. §4.3 es el instrumento con el que §6.2 midió que rutear por nombre no tiene premio, una identidad contable que no afirma que rutear convenga. §4.4 dice sobre qué

se afirma todo esto. Los resultados formales auxiliares, los corolarios del Teorema 1 y el umbral de imposibilidad, la dominancia de la cascada y el soundness del ensamblador, están en el Apéndice B con sus demostraciones; el cuerpo los cita donde los usa. Los enunciados se rotulan como teoremas y proposiciones por consistencia con los tests que los verifican, no porque su demostración sea difícil.

4.1 Confinamiento de varianza

El título de este trabajo nombra una propiedad, así que la propiedad necesita definición antes que evidencia.

Definición 1 (Trayectoria). Sea q un request y M el material declarado. Una *trayectoria* $T(q, M)$ es la secuencia ordenada de nodos visitados por un agente: cada nodo es un par (unidad leída, llamada emitida).

Definición 2 (Punto de ramificación delegado). Un nodo $n_i \in T$ es un *punto de ramificación delegado* si la identidad de n_{i+1} es función de la salida del LLM. Se escribe $d(T) = |\{n_i \in T : n_i \text{ es delegado}\}|$.

La distinción es entre qué se extrae de un nodo y cuál es el nodo siguiente. Un agente que le pregunta al modelo «¿qué dice esta unidad?» no delega ramificación; uno que le pregunta «¿dónde busco ahora?» sí.

Definición 3 (Confinamiento de varianza). La varianza de un agente está *confinada* sobre (q, M) si $d(T(q, M)) = 0$: la secuencia de nodos visitados es una función determinista de q y de M , y la salida del LLM sólo determina el contenido atribuido a cada nodo.

Definición 3b (Ramificación delegada de dominio tipado). Un punto de ramificación delegado n_i es de *dominio tipado* si la salida del LLM se proyecta, antes de usarse, sobre un conjunto finito de candidatos C_i que el código computa desde el índice y desde (q, M) , y $n_{i+1} \in C_i$ siempre. Es de *dominio abierto* si n_{i+1} puede ser cualquier cosa que el LLM emita. El Algoritmo 2 tiene $d(T) = n$ ramificaciones delegadas, una por salto, todas de dominio tipado: la línea 10 proyecta la salida del LLM a una entidad, la línea 12 la resuelve contra los candidatos que devuelve el índice, y la línea 13 abstiene si el conjunto queda vacío.

No cumple la Definición 3, y el paper no afirma que la cumpla. Lo que las cuatro correcciones de §6.1.4 hicieron fue sacar una ramificación del LLM (el ancla, línea 4) y volver tipadas las n que quedan. La Definición 3 es binaria y no puede expresar eso; esta definición lo expresa, y lo que sigue sin formalizar es cuánta varianza compra pasar de dominio abierto a tipado. §6.1.6 lo mide sobre ocho brazos y no lo prueba.

Proposición 4 (Localización). Si la varianza está confinada, entonces para dos ejecuciones cualesquiera T_1 y T_2 sobre el mismo (q, M) se cumple $T_1 = T_2$ como secuencia de nodos, y toda discrepancia entre sus salidas es atribuible a un nodo identificable.

Demostración. Por inducción sobre la longitud. El nodo inicial es función de q y M por hipótesis. Dado n_i idéntico en ambas ejecuciones, n_{i+1} es función de $(q, M, n_1 \dots n_i)$ y no de la salida del LLM, porque $d(T) = 0$ excluye la dependencia; luego n_{i+1} coincide. Las dos secuencias son iguales. Una discrepancia de salida exige entonces que algún nodo haya recibido contenido distinto, y ese nodo es el testigo. ■

Lo que la proposición compra. Sin confinamiento, dos réplicas que difieren pueden diferir porque leyeron material distinto, y no hay forma de localizar la causa: la discrepancia se reparte sobre una historia entera.

Con él, la discrepancia siempre tiene un nodo responsable, y una discrepancia localizable es depurable, auditable y corregible, mientras una repartida no.

Lo que no afirma. El confinamiento no reduce la varianza del LLM ni mejora la calidad de la respuesta: reordena dónde puede manifestarse. La afirmación empírica de §6.1 (que confinar sube la utilidad *además* de estabilizar) es un resultado de ese corpus y no una consecuencia de la Proposición 4.

Observación 1 (Consecuencia observable). Para k réplicas de una misma celda, $\text{pass}^k = P(\text{todas las } k \text{ réplicas aciertan})$. Bajo confinamiento la única fuente de discrepancia entre réplicas es el contenido extraído en nodos idénticos; sin confinamiento se suma la divergencia de trayectorias. No es una proposición, porque no acota ninguna cantidad: es la razón por la que §6.1.2 reporta pass^k al lado de $\text{pass}@1$, y por la que la diferencia entre los dos se lee como varianza que vive dentro de una celda. Con $k = 3$ la probabilidad de detectar una celda cuya réplica se da vuelta con probabilidad q es $1 - q^3 = (1 - q)^3$, que vale 0,27 para $q = 0,10$ y 0,49 para $q = 0,20$; toda fracción de celdas inestables que este paper reporta es una cota inferior.

4.2 La clave de la política y su procedencia

Todo lo que §3.5 llama aprender se ejecuta como una tabla: la política mira una clave y devuelve una decisión. Esta sección dice de qué depende que esa tabla conserve la garantía de §4.1, y es la condición que une las cuatro contribuciones.

Definición 4 (Clave de la política). Sea ϕ una función que lleva un request q y su material M a una tupla de ejes, $\kappa = \phi(q, M, \sigma)$, donde σ es la salida del LLM sobre (q, M) . La *clave* es κ , y una *región* es una celda de la partición del espacio de claves. Un eje de κ es **COMPUTED** si es función de (q, M) solamente, y **ELICITED** si depende de σ .

Definición 5 (Política). Una *política* θ es una función determinista de la clave a una decisión, $\theta(\kappa) \in D$, donde D incluye a los brazos del catálogo, la abstención y la sonda.

Proposición 5 (La procedencia de la clave hereda o rompe el confinamiento). Sea θ una política y $\kappa = \phi(q, M, \sigma)$ su clave.

1. Si todos los ejes de κ son **COMPUTED**, entonces $\theta(\phi(q, M))$ es función determinista de (q, M) , y la decisión de qué brazo correr no es un punto de ramificación delegado en el sentido de la Definición 2. La trayectoria completa, decisión incluida, conserva la Proposición 4.
2. Si algún eje de κ es **ELICITED**, entonces para (q, M) fijos la decisión $\theta(\kappa)$ es una variable aleatoria sobre la distribución de σ , aun cuando $d(T) = \theta$ para cada brazo del catálogo. Dos réplicas del mismo request pueden ejecutar brazos distintos, y la discrepancia entre sus salidas no tiene nodo responsable dentro de ninguna trayectoria: está antes de todas.

Demostración. (1) es composición de funciones: ϕ restringida a ejes **COMPUTED** es función de (q, M) , θ es función de κ , así que $\theta \circ \phi$ es función de (q, M) , y la Definición 2 excluye que sea delegada porque no depende de σ . La inducción de la Proposición 4 arranca entonces un nodo antes, en la elección del brazo, y sigue igual. (2) Si un eje depende de σ y σ no es constante sobre (q, M) , entonces κ no es constante y $\theta(\kappa)$ tampoco tiene por qué serlo; la elección del brazo es entonces una ramificación cuyo siguiente nodo es función de la salida del LLM, es decir, delegada por la Definición 2, y la Proposición 4 pierde su hipótesis en el primer paso. ■

Lo que la proposición compra. Da la forma exacta de la exigencia que §6.2.5 mide: el vocabulario de región tenía un eje elicitado, el acoplamiento, y ese eje cambió de valor en el 27% de las tareas según qué modelo las sensó, así que la política buscaba en filas distintas de la tabla para el mismo request. No es un defecto de calibración. Es (2). Y explica por qué cada medidor que el ciclo de §6.5 agregó tuvo que ser `COMPUTED`: era la única forma de agregar información a la clave sin volver a (2).

Lo que no afirma. No dice que una clave `COMPUTED` sea informativa. Una clave puede ser determinista e inútil, y §6.5.2 mide exactamente ese caso: la región no veía el horizonte. La proposición separa dos preguntas que el diseño suele mezclar, si la tabla es una tabla y si la tabla sabe algo. La primera es esta sección; la segunda es §6.3 y §6.5.

4.3 El Teorema del Valor de Selección

Es una identidad contable, no un resultado empírico, y se usa como instrumento. No afirma que rutear convenga: descompone exactamente el valor de rutear en términos que se pueden medir por separado, y por eso permite que una evaluación de ruteo sea falsable. §B.4 desarrolla la distinción. Un teorema presentado como evidencia del sistema sería el error que §2.5 le audita a otros.

Sea p^* el fallback y $p_1 \dots p_k$ los especialistas. Sea $\Delta_j(t) = u(t, p_j) - u(t, p^*)$. Para cada brazo, partir el espacio de tareas en tres, la tercera parte no es un tecnicismo, y §4.3.1 muestra qué cuesta colapsarla:

$$\begin{array}{lll} S_+^j = \{ \Delta_j > 0 \} & \text{ganancia estricta} & \pi_j = \Pr[S_+^j] \\ S_0^j = \{ \Delta_j = 0 \} & \text{empate} & \tau_j = \Pr[S_0^j] \\ S_-^j = \{ \Delta_j < 0 \} & \text{pérdida estricta} & \nu_j = \Pr[S_-^j] \end{array}$$

y sean, condicionadas a lo efectivamente ruteado.

$$\begin{array}{ll} \alpha_j = \Pr[\text{rutear a } p_j \mid S_+^j] & G_j = E[\Delta_j \mid \text{rutear a } p_j, S_+^j] \\ \beta_j = \Pr[\text{rutear a } p_j \mid S_-^j] & L_j = E[-\Delta_j \mid \text{rutear a } p_j, S_-^j] \end{array}$$

Teorema 1. Para todo $k \geq 1$.

$$V(r) - V(p^*) = \sum_j (\pi_j \cdot \alpha_j \cdot G_j - \nu_j \cdot \beta_j \cdot L_j)$$

exactamente, sin ningún supuesto de independencia. Por lo tanto la selección le gana al mejor paradigma fijo si y sólo si $\sum_j \pi_j \alpha_j G_j > \sum_j \nu_j \beta_j L_j$.

Demostración. $V(r) - V(p^*) = E[\Delta_{\{r(t)\}}(t) \cdot 1\{r(t) \neq p^*\}]$. Descomponiendo por destino queda $\sum_j \Pr[\text{rutear a } p_j] \cdot E[\Delta_j \mid \text{rutear a } p_j]$, y partiendo cada esperanza condicional según el signo de Δ_j : la parte de S_+^j pesa $\pi_j \alpha_j$ con media G_j , la de S_-^j pesa $\nu_j \beta_j$ con media $-L_j$, y S_0^j aporta exactamente cero porque ahí $\Delta_j = 0$. ■

Dos decisiones lo vuelven exacto y no aproximado. Condicionar G y L a los subconjuntos *ruteados* elimina todo supuesto de independencia entre dónde vive la ganancia y dónde el ruteador elige ir. Y descomponer por destino en vez de por un único «especialista» lo hace valer para un catálogo: con k brazos, equivocarse tiene un *destino*, y mandar una tarea a un brazo apenas peor no es el mismo evento que mandarla al peor de doce.

Corolario 1 (precisión sobre cobertura). Cuando p^* es casi óptimo en regiones amplias, G es chico y L grande, así que la condición exige $\beta \rightarrow 0$ incluso a costa de α . El recall no es el objetivo.

4.3.1 Un empate no es un misruteo

S se define con desigualdad *estricta*, así que los empates quedan afuera. Definir β sobre el complemento de S_+ le cobraría al ruteador haber ruteado sobre un empate, un acto que no cuesta nada. Construido: un ruteador que rutea sólo donde gana o empata, y nunca donde pierde, registra $\beta = 0,714$ bajo esa definición sin haber hecho un solo daño.

El producto $\beta \cdot L$ es correcto con cualquiera de las dos definiciones, porque L absorbe el cero. Pero β sola deja de ser la *tasa* de misruteo, y el Corolario 2 usa β y L por separado. Definir β sobre S_- restituye la lectura esperada y deja el Teorema 1 intacto: los empates aportan cero a los dos lados.

No es un caso de borde en ningún catálogo donde varios paradigmas resuelven la misma tarea. En la auditoría del catálogo anterior a la campaña, sobre 96 tareas de seis corpus del registro *nano* y *gpt-5-chat*, un brazo era el más barato al empatar en 46 de ellas.

4.4 Lo que la teoría no supone, y por qué eso es la afirmación

Todo lo de §4.1 a §4.3, y lo del Apéndice B, está enunciado sobre un catálogo de brazos, una utilidad, una base de creencias y un retículo de procedencia. Ninguno de los cinco resultados menciona recuperación, documentos ni respuesta a preguntas. Es la afirmación. Lo que se describe es una capa de decisión sobre *acciones que el sistema puede tomar*, y el ruteo de paradigmas sobre un corpus documental es la instancia que pudimos medir sin juez.

La distinción sobre la que el motor realmente gira es qué se sabe y cuándo, no qué clase de tarea es. Una regla sólo puede gobernar si se la puede evaluar al momento de decidir; un valor sólo se puede emitir si una creencia vigente lo lleva a la procedencia exigida; un nivel sólo se puede subir. Las tres son propiedades de la decisión, no del dominio.

La misma maquinaria, enunciada sobre cuatro superficies. Los teoremas de arriba son la forma general; las columnas son lo que instanciarlos exige.

superficie	el LLM emite	la regla decide	el registro guarda
contenido	números con procedencia	emitir o negarse, por ranura (§B.6)	qué creencia llenó qué ranura
datos	una consulta propuesta, su grano, su resolución temporal	admitir la consulta o exigir elicitación	la consulta, y contra qué se la chequeó
acciones	una llamada a herramienta y sus precondiciones	el piso de procedencia sobre lo irreversible (§3.4.1)	un ledger de idempotencia
gobierno	una edición candidata de la política	la guarda de promoción sobre episodios held-out	el diff entre dos bundles firmados

La selección entre topologías de control es la primera columna instanciada sobre un catálogo de recuperación. Es el caso medido, no el alcance de lo que se afirma.

Y la rama no probada de la teoría y la superficie no medida son el mismo lugar. §B.5 parte el problema sobre v , la disponibilidad de un detector barato, y §1.3 registra que todos los corpus de este registro caen del lado $v = 1$ por construcción, porque el gold es lo que vuelve la calificación libre de juez y el gold es un detector. Así que la rama $v = 0$ (la que necesita un router) no la puede alcanzar ningún benchmark que califique por exact-match.

$v = 0$ vive, predominantemente, en la superficie de acciones. Chequear *si un archivo se escribió* es barato; chequear *si éste era el reembolso correcto* no lo es, y no hay clave de respuestas que lo abarate. El dominio que este registro no mide es el dominio donde la partición central de la teoría por fin tiene dos lados.

La ambición se enuncia en vez de matizarse. La teoría es general por construcción y está verificada como tal (sobre distribuciones sintéticas con respuesta conocida, no sobre corpus). Las mediciones son sólo de recuperación, y §8 dice exactamente qué herramientas existieron y cuáles nunca. §4 se afirma para agentes en general; §6–§8, para extracción de respuesta exacta sobre documentos. La brecha entre las dos es el alcance real, y está declarada en vez de estrechada.

5. El banco y el método de medición

La disciplina de medición se dice una vez acá y se cita después. Corpus y modos están en §1.3.1.

5.1 Medir sin juez, y el corrector

Cada tarea trae un oráculo de conjunto, así que la calidad es F1 de conjuntos tras normalización. §5.1.2 da la razón: el efecto es de un dígito de puntos porcentuales y la varianza del juez es del mismo orden, así que un juez agregaría, además de ruido, un sesgo. El sesgo de verbosidad favorece las respuestas largas que producen los paradigmas caros, que es precisamente la comparación bajo prueba.

Un oráculo vacío es una pregunta legítima e importante, *listá todos los X* donde no hay ningún X, y testea si un paradigma inventa items. Se califica exigiendo un enunciado explícito de vacuidad; el silencio saca cero, porque un paradigma que no devolvió nada porque se cayó no debe puntuar igual que uno que buscó y reportó no haber encontrado nada.

5.1.1 Validez del corrector

El pitfall más caro de un banco de agentes es que sus ceros sean del corrector y no del sistema: la auditoría de OpenAI sobre SWE-bench Verified encontró que el 59,4% de las 138 tareas que su modelo no resolvía de forma consistente tenían defectos materiales en los tests o en el enunciado, que las volvían muy difíciles o imposibles también para una persona [OpenAI, 2026]. El denominador son esas 138 tareas y no las 500 del benchmark, y el defecto es del dataset y no del arnés de ejecución; la lección es la misma. Acá el corrector se audita con una prueba automática que barre toda respuesta con utilidad cero y marca las que traen señal de defecto de emparejamiento (el oráculo presente en la respuesta sin sobrantes, un rechazo no reconocido, un ítem parcial no acreditado, un número equivalente con otro formato). Sobre 567 ceros, ninguno presenta señal fuerte. La auditoría no prueba que el corrector sea correcto: prueba que esas cuatro clases de defecto no están.

La campaña, y es la fuente de los números de esta sección salvo donde se declara otro modelo. Doce paradigmas sobre 78 tareas con 3 réplicas: nueve brazos corrieron 67 tareas y tres corrieron las 78, 2.511 filas, 123,3M tokens, cero errores de infraestructura, sobre un corpus con entidades reales, un modelo, mismas condiciones, sin juez LLM y con el corrector auditado. Toda $\lambda = 0$ (calidad pura, con el costo en su propia columna). Todo panel comparativo es el rectángulo mecánico de `bench.panel` sobre el registro vigente, 64 tareas $\times$ 8 brazos, sin exclusiones a mano. Sobre el corrector. El registro se re-puntuó una vez (252 filas cambiaron, en los modos de plantel declarado y presuposición); toda tabla de §5 y §6 está recomputada sobre el registro re-puntuado, y la auditoría de los 567 ceros de §5.1.1 es posterior a él. Las filas no llevan estampada la versión del corrector; es una deuda declarada.

5.1.2 Por qué no un juez LLM: lo que la literatura mide

Deliberadamente no usamos juez LLM. Un estudio a gran escala sobre 21 modelos y 541.000 juicios reporta confiabilidad sin validez, y que el acuerdo crudo sobreestima la capacidad discriminativa [arXiv:2606.19544]. Las métricas estilo RAGAS exhiben sesgo de posición, de verbosidad y de auto-preferencia, y la mitigación recomendada es corridas repetidas con inspección de dispersión. Como los tamaños de efecto medidos son de un dígito de puntos porcentuales y la varianza del juez es del mismo orden, y como el sesgo de verbosidad favorecería sistemáticamente a los paradigmas caros cuyo valor está justamente en cuestión, un juez introduciría un sesgo alineado con la hipótesis. §5.1 explica la alternativa.

Sobre la varianza a temperatura cero, que §6.1.2 mide y §8.2 discute: Ouyang y colegas [arXiv:2308.02828] y Atil y colegas [arXiv:2408.04667] miden la inestabilidad entre corridas idénticas, y la fuente que §8.2 describe sin nombre, la falta de invariancia por lote en el servidor, la explica He [Thinking Machines, 2025]. Bouthillier y colegas fijan la disciplina de reportar la varianza de todas las fuentes del procedimiento en benchmarks de aprendizaje, muestreo de datos, inicialización e hiperparámetros [arXiv:2103.03098], que es lo que `pass^k` y el piso por celda hacen acá sobre la única fuente que este montaje controla, la réplica. Y `pass^k` lo introdujo τ -bench [arXiv:2406.12045] para agentes multi-turno; τ^2 -bench lo continúa.

Sobre atribución de fallos, MemFail aísla los fallos de sistemas de memoria en modos de resumen, almacenamiento, recuperación y razonamiento, y puede atribuir un error a uno de ellos sólo porque las operaciones intermedias quedan registradas [arXiv:2605.26667], convergente con el requisito de §6.1.3, aunque su atribución corre sobre un juez LLM donde la ésta corre sobre trazas deterministas de

herramientas. Su titular es también el de este trabajo en miniatura: escalar las memorias recuperadas o la fuerza del modelo rinde poco y a veces degrada, dependiendo de la tarea.

5.2 Los doce paradigmas, en una tabla

Las secciones que siguen miden a cada brazo por un lado distinto (cobertura, degradación con el ancho, fiabilidad, latencia, decisiones delegadas) y cada una tiene su tabla. Ésta las junta, porque cinco tablas que nadie cruza son menos útiles que una que declara sus denominadores.

brazo	aplica	u	u × aplica	pass^3	tok/celda	USD/1k celdas	serie	ley de costo
react	100%	0,850	0,850	0,734	108.137	22	1,35 s	vueltas
dag_strategy	100%	0,830	0,830	0,703	105.293	22	2,87 s	vueltas
reflection	100%	0,808	0,808	0,688	133.574	27	1,76 s	vueltas
rewoo	100%	0,678	0,678	0,531	10.840	2	0,77 s	estructural
supervisor	100%	0,591	0,591	0,406	64.079	13	2,66 s	vueltas
gist_reader	100%	0,584	0,584	0,516	23.075	5	0,91 s	vueltas
handoff	96%	0,604	0,581	0,438	120.500	27	2,04 s	alcance
pointer_chase	96%	0,515	0,492	0,359	9.787	2	1,38 s	vueltas
graph_traverse	52%	0,511	0,266	—	21.356	4	—	estructural
streaming_scan	12%	0,750	0,090	—	26.380	5	—	estructural
extract_compute	12%	0,583	0,070	—	26.184	5	—	estructural
direct	6%	0,917	0,055	—	22.822	5	—	estructural

Los denominadores son tres. `aplica`, `u`, `u × aplica` y `tok/celda` salen del registro entero (2.511 filas, cero `infra_error`): `aplica` es qué fracción de las celdas ofrecidas a ese brazo pasa la compuerta aritmética de factibilidad, y `u` promedia sólo las que pasan. `pass^3`, `serie` y `USD` son del cierre de campaña y no se recomputaron tras dos correcciones posteriores del registro (`handoff`, por la contabilidad de la superficie compartida; `graph_traverse`, por una re-corrida de sus celdas anchas); las filas de esos dos brazos dicen el valor vigente en las cuatro primeras columnas. `pass^3` y `serie` salen del rectángulo de 64 tareas × 8 brazos (el 82% de las medidas) porque exigen que todos los brazos hayan corrido las mismas tareas con tres réplicas cada una; los cuatro brazos sin valor son los que la factibilidad poda en casi todas las celdas. Toda `u` usa $\lambda = 0$: calidad pura, con el costo en su propia columna.

Cuatro cosas que sólo se ven con las columnas juntas:

1. `u` y `u × aplica` son dos números y ninguno reemplaza al otro. `direct` es el mejor del plantel donde su mecanismo corre (0,917) y aporta 0,055 sobre el corpus porque corre en el 6% de las celdas. No falla en el 94% restante: no corre, y eso lo decide la aritmética antes del primer token.
2. `pass^3` siempre está por debajo de `u`, y la distancia no es proporcional. `react` pierde 0,116 y `supervisor` 0,185. Esa diferencia es varianza que vive *dentro* de una celda, invisible para cualquier piso de ruido calculado entre brazos.

3. La columna de dólares no es la de tokens reescalada. Entrada y salida se cobran 6× distinto, y los brazos se diferencian justo en esa proporción: dos brazos pueden costar lo mismo en dólares con miles de tokens de diferencia por celda, según cuánto de cada uno sea salida.
4. El margen está en la última columna, no en la primera. Entre `react` y `rewoo` hay 0,172 de utilidad y un factor 10× de costo y 1,8× de latencia serial. En utilidad los brazos se separan por centésimas; en lo que cuestan, por órdenes de magnitud.

El `pass^3` de esta tabla y el de §6.1.2 son rectángulos distintos y no se deben cruzar. Acá sale de las 64 tareas comunes a los ocho brazos; allá, del subconjunto donde los cuatro brazos comparados corrieron sus tres réplicas, que es más chico y más fácil. De ahí que `react` figure con 0,734 en esta tabla y 0,797 en §6.1.2.

`pointer_chase` figura en esta tabla con su número de campaña. Las cuatro correcciones de §6.1.4 (que lo llevan de 0,33 a 0,89 en la celda de cadenas acopladas) son posteriores a esta corrida y tocan 3 de las 78 tareas, así que su efecto sobre el agregado del corpus está dentro del ruido y no se propagó a esta tabla.

5.2.1 Un paradigma tiene dos números y colapsarlos esconde el caso que importa

Los dos números por brazo, `u` donde aplica y `u × aplica`, son las columnas segunda y tercera de la tabla de §5.2; la figura los cruza.

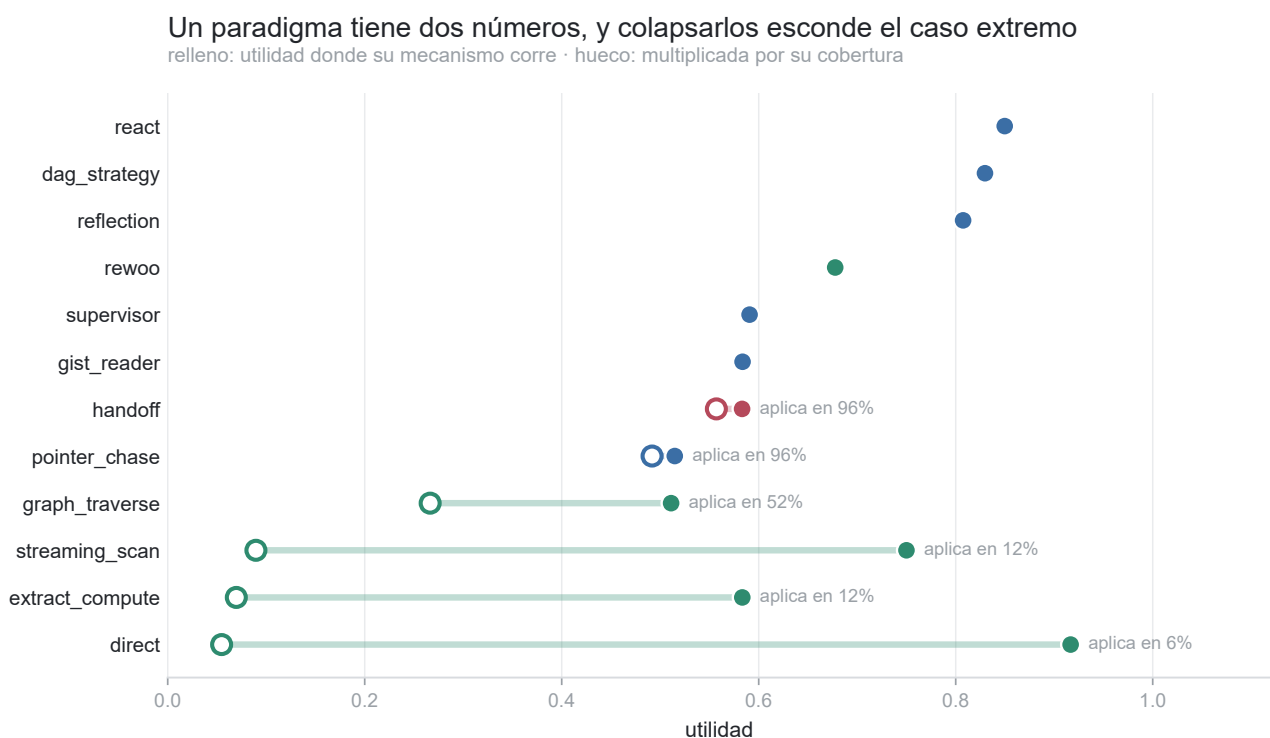


Figura 3. Bueno donde aplica, contra lo que aporta sobre el corpus. Una barra por brazo, ordenadas de `direct` a `react`. La parte rellena es `u`, la utilidad media sobre las celdas donde el mecanismo del brazo corre; la parte hueca es `u × aplica`, esa misma utilidad multiplicada por la fracción de celdas en que la compuerta de factibilidad lo dejó correr. Los cuatro brazos de la izquierda tienen la barra rellena alta y la

hueca casi vacía, porque aplican en el 6%, 12%, 12% y 52% de las celdas; los de la derecha aplican en el 96% o el 100% y las dos partes casi coinciden.

Un paradigma tiene dos números y colapsarlos esconde el caso que importa. `direct` es el mejor del plantel donde su mecanismo corre (0,917), y su mecanismo corre en el 6% de las celdas (12 filas de 201; sobre doce filas «el mejor del plantel» es una afirmación de otra clase que sobre doscientas), porque la aritmética de factibilidad lo poda en cuanto el material no entra. Sobre el corpus aporta 0,055. Reportar un solo número obliga a elegir cuál de las dos afirmaciones falsear.

La cobertura es una propiedad de la intersección entre el mecanismo del brazo y la distribución de tareas, no del brazo solo, y por eso se decide antes de gastar: `direct`, `streaming_scan` y `extract_compute` no fallan en el 88–94% restante, no corren.

5.2.2 La degradación con el ancho separa lo que la utilidad media junta

Casi todos se degradan cuando el material crece. `rewoo` no.

un panel por brazo; los demás, en gris de fondo

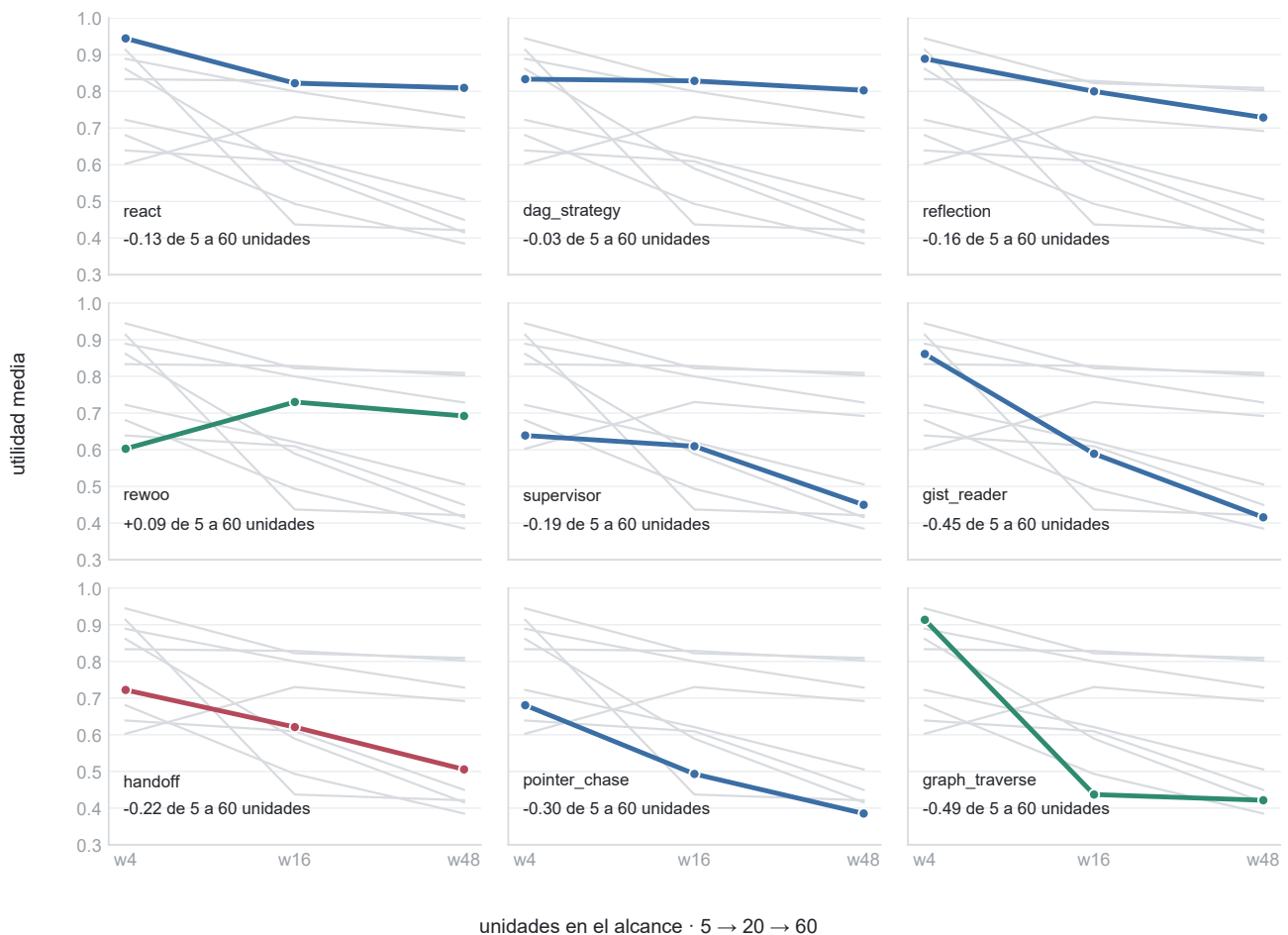


Figura 4. Cómo se degrada cada brazo cuando el material crece. Un panel por brazo, nueve en total, con el mismo eje horizontal (5, 20 y 60 unidades en el alcance) y el mismo eje vertical (utilidad media). En cada panel la línea del brazo va en color y las de los otros ocho quedan en gris de fondo, para que se vea dónde cae cada uno respecto del resto. El número de cada panel es la diferencia de utilidad entre 5 y 60 unidades: negativa en ocho brazos, positiva sólo en `rewoo`.

El eje son los tres anchos declarados (5, 20 y 60 unidades). Las tareas sin sufijo de ancho quedan fuera: agrupan celdas de 1, 8, 9 y 60 unidades, así que no son el extremo angosto de nada y meterlas convertía el eje en algo que no está ordenado.

Sobre todas las filas factibles del registro (36 a 63 filas por celda de la tabla; recomputado el 2026-09-01):

brazo	w4 (5 u.)	w16 (20 u.)	w48 (60 u.)	Δ
react	0,94	0,82	0,81	-0,13
dag_strategy	0,83	0,83	0,80	-0,03
reflection	0,89	0,80	0,73	-0,16
rewoo	0,60	0,73	0,69	+0,09
handoff	0,75	0,62	0,51	-0,24
supervisor	0,64	0,61	0,45	-0,19
gist_reader	0,86	0,59	0,42	-0,45
pointer_chase	0,68	0,49	0,39	-0,30
graph_traverse	0,91	0,44	0,42	-0,49

Todos se degradan menos uno. `graph_traverse` pierde 0,49 y `gist_reader` 0,45 al pasar de 5 a 60 unidades, el gist de 180 caracteres y el índice de entidades dejan de discriminar cuando hay sesenta candidatos. `rewoo` es la única excepción, y sube: +0,09. Es el único brazo cuyo costo no es función del alcance.

Un promedio sobre anchos que no separara los tres declarados habría acercado a `gist_reader` y a `rewoo`, y habría escondido que uno se desploma exactamente donde el otro se sostiene.

5.2.3 Tres clases de costo, y no son las del catálogo

Ajustando `log(costo)` contra `log(alcance)` por brazo, y preguntando por separado qué explica mejor el costo (el alcance o la cantidad de vueltas) aparece una taxonomía que corta transversal a la de control de flujo:

clase	brazos	qué la define
alcance	handoff	$R^2 = 0,77$ contra el alcance, exponente 0,73. El costo lo fija cuánto material arrastra cada llamada
vueltas	react, reflection, dag_strategy, supervisor, gist_reader, pointer_chase	el costo lo fija cuántas veces itera, y eso es endógeno: gasta hasta que algo lo detiene
estructural	rewoo, direct, graph_traverse, extract_compute, streaming_scan	ni una ni la otra: el costo está fijado por la forma del patrón

El dato que obliga a separar estas clases de los techos declarados: `handoff` tiene un techo de 12 llamadas y gasta 120.500 tokens; `dag_strategy` tiene uno de 160 y gasta 105.293. Del mismo orden, con un factor 13 de diferencia en el techo.

Contar llamadas para acotar esfuerzo es contar envases para acotar peso.

Y una consecuencia operativa: la clase vueltas es la única sobre la que una regla de parada puede actuar. Medido en el mismo registro, el 46,5% de las búsquedas de `react` no traen ninguna unidad nueva, con rachas de hasta 14.

5.2.4 Dónde está el margen

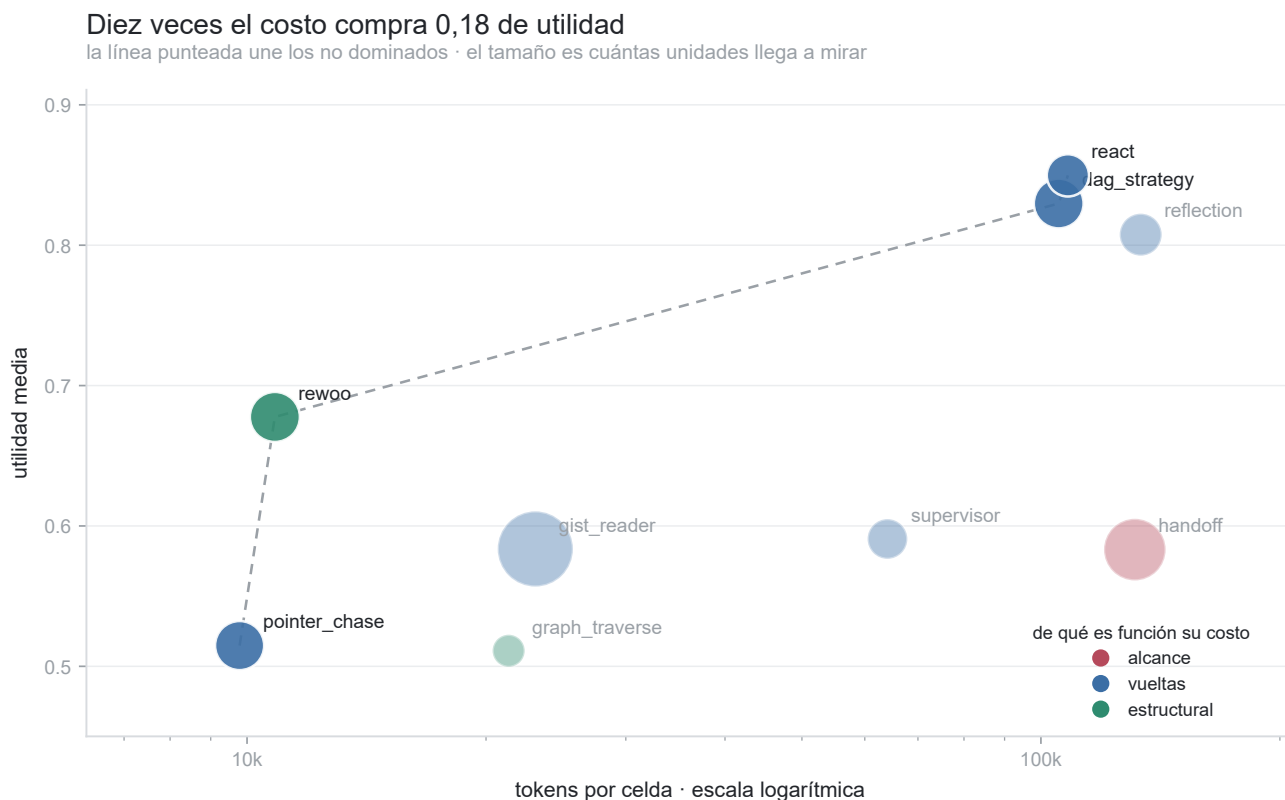


Figura 5. El negocio de cada brazo. Cada punto es un brazo: el eje horizontal son sus tokens por celda en escala logarítmica, el vertical su utilidad media, y el tamaño del punto cuántas unidades del material llega a mirar. El color dice de qué es función su costo: del alcance, de las vueltas, o de la forma del patrón. La línea punteada une a los brazos no dominados, los que nadie supera a la vez en utilidad y en costo; un brazo debajo de esa línea tiene un vecino que hace lo mismo por menos. Diez veces el costo compra 0,18 de utilidad.

En el ancho mayor, `react` saca 0,81 a 108.137 tokens por celda y `rewoo` 0,66 a 10.840: +0,15 de utilidad por un factor 10 de costo. Y el costo es casi enteramente de entrada (de 98,6% a 100,0% según el brazo, con la salida entre 0,0% y 1,4%), lo cual dice que los paradigmas se diferencian en lo que arrastran al prompt, no en lo que generan. Es la misma afirmación que la ley de costo, medida por otro lado.

Un brazo queda afuera del rango. En `graph_traverse` la suma de entrada y salida da 101,9% del costo registrado. Entrada y salida son las dos particiones del mismo total, así que por encima de 100% no hay una tercera categoría: es un defecto de contabilidad de ese brazo, no una propiedad medida. Se excluye del rango y queda anotado como deuda, no como hallazgo.

5.2.5 La ley de costo, dibujada

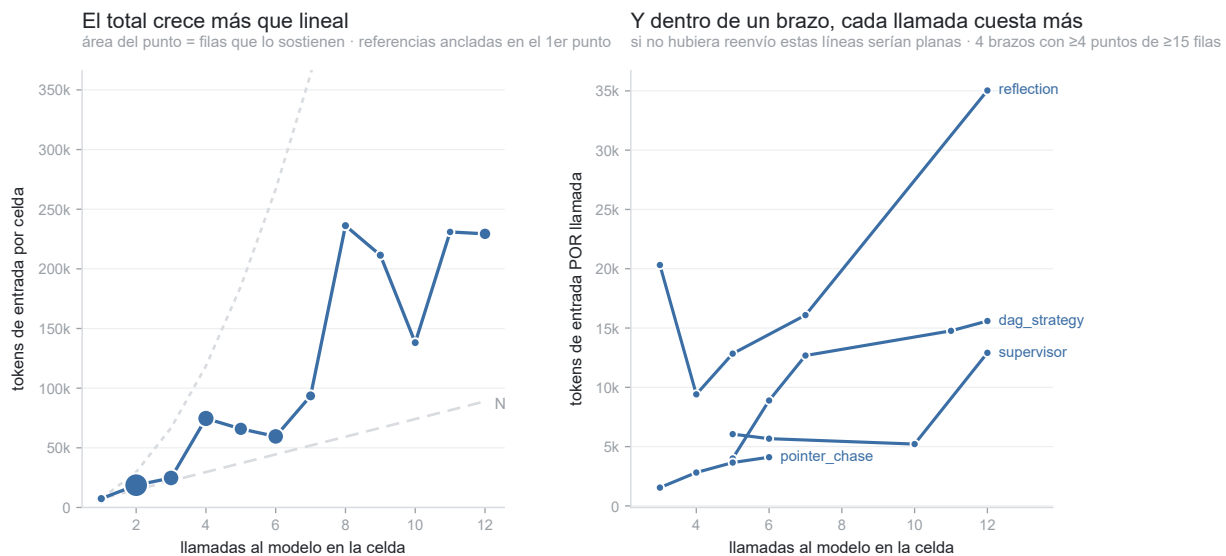


Figura 6. La ley de costo, en dos paneles. El izquierdo pone las llamadas al modelo por celda contra los tokens de entrada por celda; el área de cada punto es cuántas filas lo sostienen, y las curvas N y N^2 están ancladas en el primer punto para que el ojo compare. El total crece más que lineal. El derecho pone las mismas llamadas contra los tokens de entrada por llamada, una línea por brazo, para los cuatro brazos con al menos cuatro puntos de quince filas. Si cada llamada costara lo mismo, esas líneas serían planas; suben en los cuatro, porque cada vuelta reenvía todo lo anterior.

El costo de un bucle de herramientas crece como N^2 y la cobertura como N , porque la conversación se reenvía entera en cada vuelta. Dos números sueltos (« ≤ 2 llamadas dan 9.779 tokens, ≥ 8 dan 136.432») no alcanzan para afirmarlo: son compatibles con crecimiento lineal si uno no mira el resto.

Hacen falta dos paneles y no uno, porque un total creciente no distingue «cada llamada cuesta lo mismo y hay más llamadas» de «cada llamada cuesta más». El panel derecho separa las dos: si no hubiera reenvío, esas líneas serían planas.

brazo	3-5 llamadas	11-12 llamadas	factor
dag_strategy	4.002	15.592	3,9×
supervisor	6.052	12.910	2,1×
pointer_chase	1.544	4.107	2,7×
reflection	9.410	35.030	3,7×

El panel derecho condiciona por brazo, y esa es la única forma de leerlo. Agregado sobre todos, el costo por llamada zigzaguea (7.411, 18.654, 29.528, 13.815, 19.113) porque distintos brazos dominan distintos conteos de llamadas y sus alcances difieren en un orden de magnitud: «más llamadas» y «qué brazo» quedan confundidos, y el zigzag es esa confusión y no el fenómeno. Condicionado por brazo el trazo sube monótono en los cuatro que tienen puntos suficientes.

No se estima ningún exponente ni se reporta un R^2 : las curvas N y N^2 del panel izquierdo están ancladas en el primer punto para que el ojo compare, y el hallazgo es cualitativo. Con n desparejo por punto (de 26 a 540 filas) un exponente ajustado tendría más precisión aparente que evidencia.

5.3 Los paneles, y por qué son dos

Toda comparación de este trabajo se hace sobre uno de dos paneles, y la utilidad de un brazo cambia según cuál. La caja lo fija de una vez; las secciones lo citan.

DOS PANELES Y NADA MÁS, y por eso u toma dos valores según qué se compare. `react` figura con $0,850$ en el panel de §5.2 y §5.2.1, y con $0,843$ en todo lo demás. | sección | panel | por qué ése | |---|---|---| | §5.2, §5.2.1, §5.2.2 | 78 tareas $\times$ 12 brazos, cada brazo sobre las celdas donde CORRIÓ | mide cobertura y aporte, que exigen incluir a los brazos podados | | §6.1 a §6.4.2 | 64×8 , el rectángulo mecánico | toda comparación entre brazos exige que todos hayan corrido las mismas tareas con tres réplicas; las 64 tareas del rectángulo las tienen | La regla del rectángulo se aplica desde un solo lugar, `bench.panel`, sin exclusiones a mano, y devuelve también lo descartado: un panel que se achica sin declarar cuánto miente por omisión. Toda u reportada usa $\lambda = 0$ (calidad pura, con el costo en su propia columna) salvo donde se indique lo contrario.

5.4 Preguntas de investigación

Las mediciones de §6 contestan ocho preguntas, y cada una tiene un criterio de decisión fijado antes de mirar el dato. Las predicciones falsables quedaron registradas con fecha en el cuaderno del laboratorio; en esta tabla se enuncia lo que cada una decide. Las preguntas van ordenadas por la contribución a la que responden, no por el orden en que se midieron, y la última columna dice sobre qué modelo y qué registro corre cada una, porque no es el mismo en todas.

	contribución	pregunta	criterio	sección	modelo y registro
PI1	la condición estructural	¿Cuánta varianza de trayectoria hay en el régimen no confinado, y confinarla cambia el resultado?	fracción de celdas con $\text{pass}^k < \text{pass}@1$, y el efecto de absorber ramificaciones sobre una celda con $d > 0$	\$6.1	luna , campaña 78 × 12 × 3
PI5	la interfaz	¿Las capacidades declaradas transfieren a un paradigma no visto?	error de predicción dejando un paradigma afuera, contra el nulo de capacidades barajadas	\$6.3	luna , rectángulo 64 × 8
PI6	la interfaz	¿La ontología de la pregunta separa a los brazos mejor que la partición estructural?	señal/ruido entre brazos dentro de cada segmento, contra el nulo por permutación de cada segmentación	\$6.3.5	luna , 64 × 8; tres de siete ejes usan la etiqueta de diseño
PI2	el ciclo	¿Hay brecha de oráculo neta de ruido, y alguna política la captura?	brecha observada menos el piso por pseudo-brazos emparejados, con IC pareado, en muestra y sobre held-out	\$6.2, \$6.2.6	luna , 64 × 8 y held-out 24 × 8
PI3	el ciclo	¿Existe una señal disponible al decidir que explique la interacción tarea×paradigma?	fracción de y explicada, contra el máximo de los nulos por permutación (corrección por selección)	\$6.2.2	luna , 64 × 8
PI7	el ciclo	¿Cada refutación del ruteo produjo un medidor nuevo que la siguiente corrida pudo usar, sin romper la reproducibilidad?	por episodio: el eje que faltaba, si es COMPUTED , y decisión reproducida desde la base registrada	\$6.5	nano , tres mundos de 26 tareas, anteriores a la campaña
PI8	el premio	¿Qué aprende la política que pague fuera de muestra, y qué predice el comportamiento de un brazo antes de correrlo?	desempate por costo leave-one-task-out; participación del paradigma en la varianza del recall; efecto de una	\$6.4.1, \$6.4.3	luna en \$6.4.1 y \$6.4.3; nano en \$6.4.3

	contribución	pregunta	criterio	sección	modelo y registro	
			herramienta ofrecida sobre el releído			
PI4	el premio	¿El acuerdo entre paradigmas predice corrección sin oráculo ni juez?	$P(\text{correcta} \setminus$	k coinciden)', con tres controles: dificultad de tarea, largo de respuesta y costo	§6.4.2	luna , con réplica en terra

Dos secciones de §6, §6.5 y §6.4.3, salen del registro anterior a la campaña, sobre `gpt-5.4-nano`, y no de ella. Se incluyen porque miden algo que la campaña no puede medir, el ciclo de reparación del vocabulario y su mecanismo, y §6.5.1 dice qué transfiere.

Un término más, además de los del glosario de §3.1. La etiqueta de diseño es el modo del que salió una tarea: se conoce al construirla y no al decidir, así que en §6.2.2 funciona como cota superior y nunca como señal disponible.

Régimen de medición: el de la caja de §5.1.1, doce paradigmas sobre condiciones idénticas (mismo corpus, mismo modelo, misma superficie de herramientas, misma decodificación) con `repeat = 3`, sin juez LLM, con el corrector auditado y cada panel declarando su rectángulo. Los fallos de infraestructura se registran como tales y se excluyen de toda estadística.

6. Resultados

En el orden del argumento: qué varianza hay y de dónde viene, por qué elegir el paradigma por su nombre no tiene premio, sobre qué sí se aprende, qué paga, cómo se reparó el vocabulario, y si θ maduró.

6.1 La varianza de trayectoria, y confinarla

Primero qué varianza hay, de dónde sale, y qué pasa cuando una decisión de control se le saca al LLM y se le da a un medidor.

6.1.1 El embudo: `react` no gana buscando

``react` no gana buscando: gana no destruyendo lo que encontró`
 área del punto = utilidad sobre el corpus · 64 tareas x 8 brazos (82% de las 78 medidas)

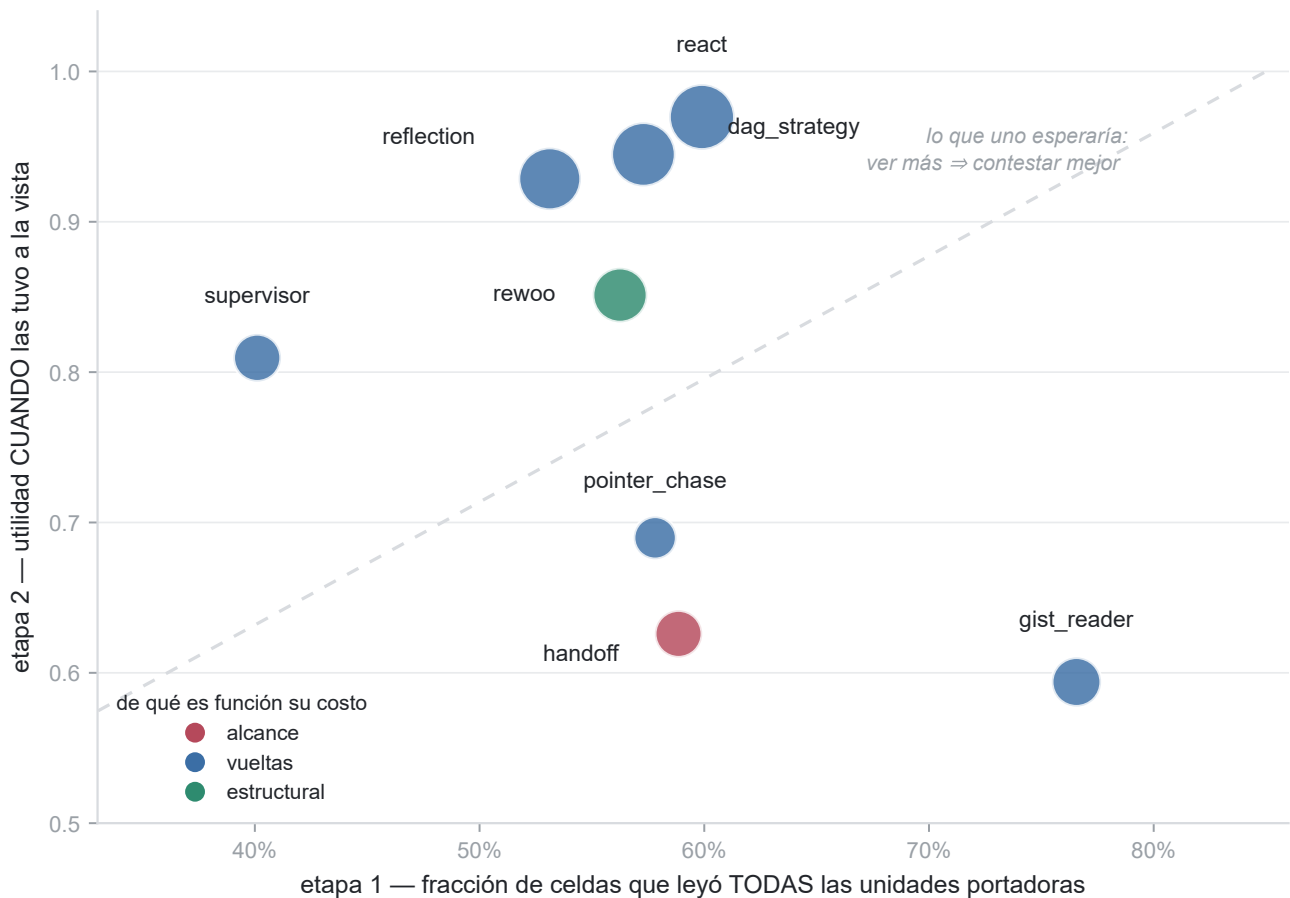


Figura 7. Ver contra usar. Cada punto es un brazo del rectángulo de 64 tareas × 8 brazos. El eje horizontal es la primera etapa, qué fracción de sus celdas leyó todas las unidades que portan la respuesta. El vertical es la segunda, la utilidad que sacó cuando las tuvo a la vista. El área del punto es su utilidad sobre el corpus, y el color, de qué es función su costo. Lo que uno esperaría es una diagonal: ver más, contestar mejor. `react` no está más a la derecha que nadie, está más arriba: no gana buscando, gana no destruyendo lo que encontró.

La utilidad de una celda es el producto de dos cosas que fallan por razones distintas:

$$u \approx P(\text{vio TODAS las unidades portadoras}) \times P(\text{contestó bien} \mid \text{las vio})$$

brazo	u	vio	u \	vio	u \	no vio	leídas
react	0,843	60%	0,970	0,708	7,8		
dag_strategy	0,822	57%	0,945	0,720	11,6		
gist_reader	0,611	77%	0,594	0,667	28,8		
supervisor	0,587	40%	0,778	0,479	6,8		

`react` no ve más que nadie, 60%, y `gist_reader` ve el 77%. Toda su ventaja está en la segunda etapa: con el material a la vista da 0,970, y `gist_reader` da 0,594, *peor que cuando no lo vio todo*. Pareado por

tarea, Δv_{io} es chico o negativo y $\Delta(u|v_{io})$ se lleva la brecha entera.

El mecanismo: cada representación intermedia más chica que el material es una pérdida que no se recupera aguas abajo. Un gist de 180 caracteres, una ventana recortada para el sub-agente, un índice de entidades, los tres tiran información *antes* de saber cuál hacía falta. `react` no tiene ninguna.

La simplicidad es acá la ausencia de un canal con pérdida, y eso se mide.

6.1.2 `pass^k` : el promedio esconde la mitad que importa

`pass^k` NO es `pass@k`, y significa casi lo opuesto. En la literatura de código `pass@k` mide «al menos un acierto en k intentos» y crece con k . `pass^k` mide «los k intentos acertaron todos» y decrece con k . El nombre viene de τ -bench [arXiv:2406.12045], que lo introdujo para agentes multi- turno por esta misma razón, y τ^2 -bench lo continúa; lo conservamos por consistencia con esa literatura y se señala porque el parecido tipográfico invita a leer la tabla al revés.

Todo lo anterior es `pass@1` (el promedio sobre réplicas) y responde «cuánto acierta». `pass^k` responde «se puede contar con que acierte», que para un sistema que promete «misma base de creencias $\Rightarrow$ misma decisión» es la mitad que importa.

brazo	pass@1	pass^3	caída	celdas inestables
react	0,843	0,734	-0,109	23%
dag_strategy	0,822	0,703	-0,119	20%
rewoo	0,688	0,531	-0,157	28%
supervisor	0,587	0,406	-0,181	28%

Una celda es inestable si sus tres réplicas no son idénticas; una celda con tres réplicas iguales y parciales es estable. El rango sobre los ocho brazos del rectángulo va del 12% (`gist_reader`) al 28% (`rewoo`, `supervisor`). Al menos entre el 12% y el 28% de las celdas cambian de resultado entre réplicas, con $t=0$, semilla fija y la misma huella; «al menos» porque con tres réplicas la potencia para detectar una celda que se da vuelta una de cada diez veces es 0,27 (Observación 1).

La varianza de una llamada y la de una trayectoria son cantidades distintas. Sobre una llamada, la varianza del LLM se manifiesta como un token distinto: el resultado se mueve poco y de forma acotada. Sobre una trayectoria, una elección distinta en el paso uno cambia qué documento se lee en el paso dos, y de ahí en adelante las dos réplicas ya no comparan la misma evidencia. La varianza no se promedia: se ramifica. Si la trayectoria delega d puntos de ramificación a la salida del modelo, la trayectoria es una variable aleatoria sobre un árbol de profundidad d . Con $d = 0$ la trayectoria es fija, y la varianza del LLM entra únicamente como el contenido que extrae de cada nodo: un error acotado y localizable en vez de una historia distinta.

Esta varianza es invisible para el instrumento habitual: vive dentro de una celda, así que ningún piso de ruido calculado *entre* brazos la muestra. Por eso hace falta `pass^k`, y por eso no basta con más réplicas

del mismo promedio.

6.1.3 En una cadena acoplada, el modo dominante es cortarla un escalón antes

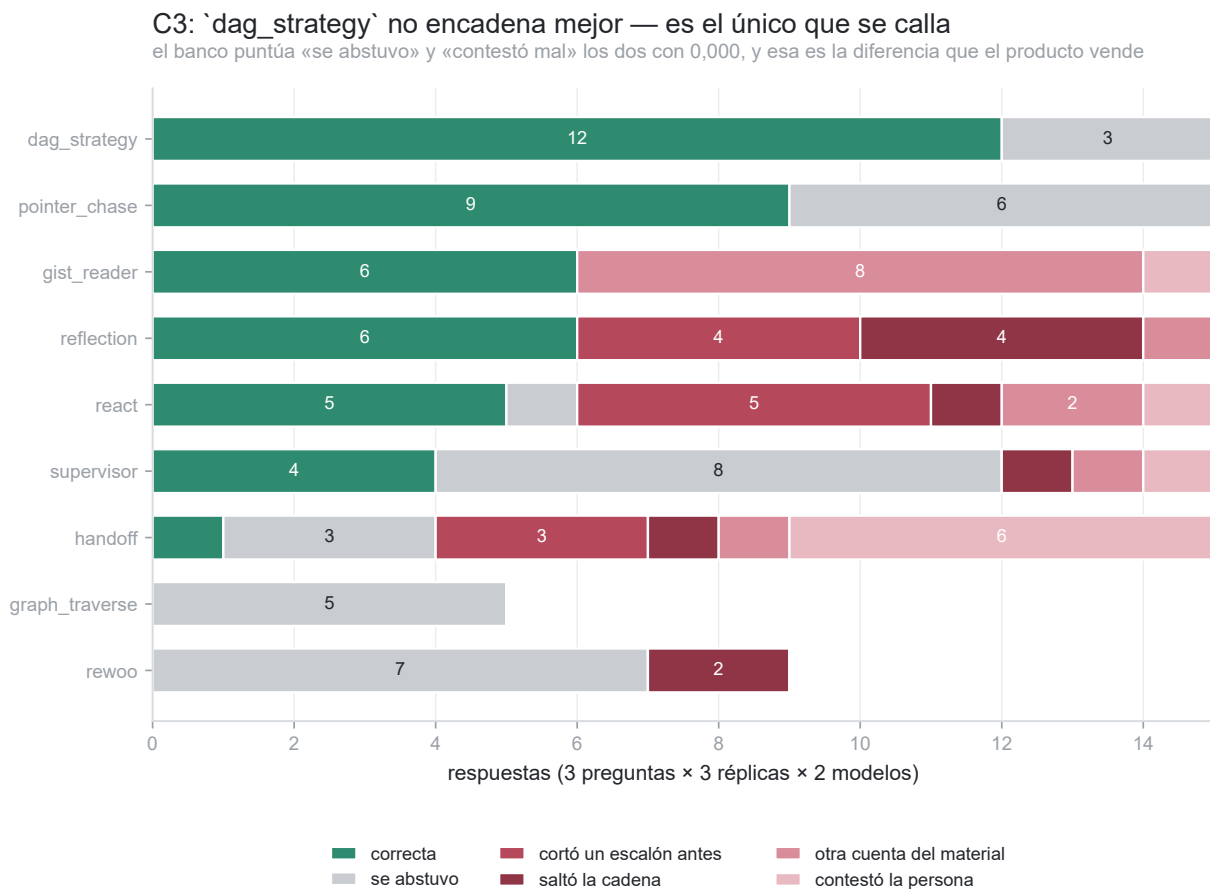


Figura 8. Modos de falla de C3, el modo de cadenas acopladas. Una barra por brazo, de `dag_strategy` a `rewoo`, y cada barra son sus respuestas sobre las tres preguntas del modo, tres réplicas y los dos modelos, `luna` y `terra`. Cada respuesta está clasificada por lo que hizo, no por su puntaje: correcta, se abstuvo, cortó la cadena un escalón antes, se saltó la cadena, devolió otra cuenta del material, o contestó con la persona en vez del dato. El banco puntúa «se abstuvo» y «contestó mal» los dos con 0,000; la figura los separa, y ahí se ve que `dag_strategy` no encadena mejor que los demás. Es el único que se calla cuando no llega.

`C3_coupled_chain`, el modo de cadenas acopladas del corpus, pide subir N escalones de una línea de reporte sobre 60 unidades y reportar un dato del último. La cadena está minada a propósito: cada unidad del camino lleva un dato del mismo tipo pegado al nombre que la ancla, el enlace va por anáfora («The above-named», «That person»), y el destino del salto viene abreviado (A. Vallejos apunta a Agustina Vallejos).

Este modo se midió sobre `gpt-5.6-terra`: sobre `nano` daba cero en toda la grilla y sobre `luna` el registro tiene dos tareas con seis celdas por brazo, así que la corrida de este modo se hizo con el modelo mayor, tres tareas, nueve celdas por brazo. Los números de `luna` van al lado.

Clasificar las respuestas por clase de falla, y no por puntaje, invierte el diagnóstico. La clase dominante es cortar la cadena un escalón antes (7 casos), y no saltársela (2). Los brazos devuelven el dato de un intermedio, que está a la vista y es indistinguible del correcto. Y `dag_strategy`, que gana el modo sobre

terra , no encadena mejor: hace 8 correctas y 1 abstención sobre 9 celdas, sin una respuesta equivocada. Sobre luna , en seis celdas, hace 4 correctas y 2 ceros, y uno de esos ceros es una respuesta y no una abstención. Los demás brazos se equivocan menos de lo que la frase «contestan igual» sugería: sobre luna , los cinco ceros de pointer_chase y seis de los nueve de rewoo son abstenciones explícitas. Lo que distingue al ganador es que además llega.

El banco puntúa «se abstuvo» y «contestó mal» los dos con 0,000. Es correcto para medir utilidad y ciego justo en el eje que el motor de decisión existe para gobernar.

6.1.4 Sustituir una ramificación delegada al modelo por un medidor de entorno

Sobre ese diagnóstico se corrigió pointer_chase (el brazo cuyo mecanismo es seguir cadenas y que sacaba 0,33 en este modo sobre terra). Cuatro correcciones, todas de flujo de control o de tipado, ninguna de fraseo. Las cuatro se desarrollaron y se probaron sobre las mismas nueve celdas de terra en que se las mide, en cuatro estados sucesivos del registro (0,333, 0,000, 0,000 y 0,889). Es un ajuste en muestra sobre tres tareas, no una transferencia; lo que transfiere es el mecanismo, y medirlo sobre celdas nuevas está en §9.1.

1. Regla de creencias: una entidad nombrada se busca con el índice léxico, no con el híbrido. Un vector denso codifica *de qué habla* un texto, y sesenta documentos con la misma plantilla hablan de lo mismo; el nombre propio es justo la parte que no es semántica, y fusionarle la rama densa le mete ruido a la única señal que discrimina. Medido: el híbrido devuelve la unidad equivocada para Ramiro Herrera y deja a M. Arrieta fuera del top-5; el léxico las pone primera y tercera.
2. La salida del LLM se tipa antes de usarse. El modelo emitía 'M. Arrieta settlement account', y esa cola arrastraba la consulta a clasificar como prosa: la regla era correcta y la entrada estaba sucia.
3. Un salto a una unidad que no nombra a quien se persigue no es un salto, y entre candidatos empatados gana el que la nombra antes, un documento que trata sobre una entidad la nombra antes que uno que la referencia de paso. Se resuelve con predicados que devuelven un booleano o una posición, nunca texto: no cuestan un token.
4. El ancla es el salto cero y lo resuelve el código. Era una llamada al modelo, y ahí quedaba la última decisión de flujo en manos del LLM.

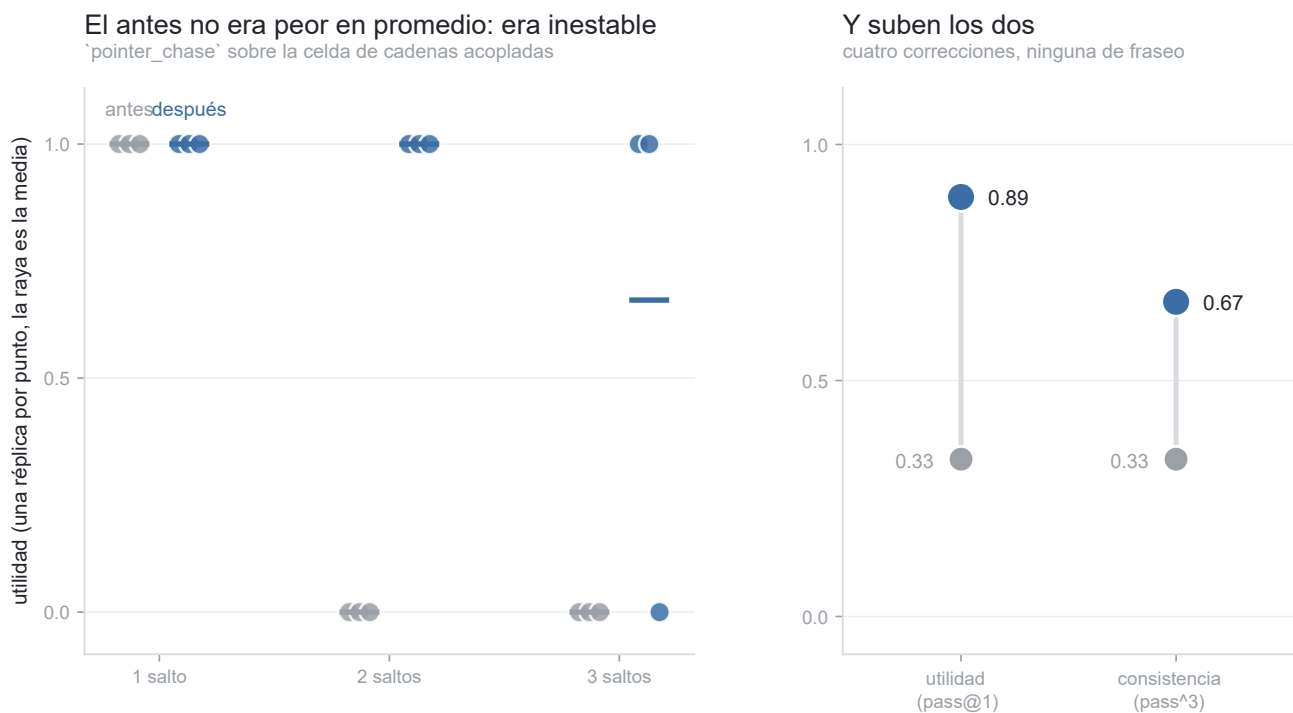


Figura 9. Una ramificación delegada, resuelta por código. El panel izquierdo muestra la utilidad de `pointer_chase` sobre las cadenas acopladas, una réplica por punto y la media como raya, para cadenas de uno, dos y tres saltos, antes y después de las cuatro correcciones. Ahí se ve lo que un promedio esconde. El «antes» era inestable, no peor en promedio: la misma pregunta, la misma huella y los mismos resultados de búsqueda daban 1,000 o 0,000 según la réplica. El panel derecho pone lado a lado utilidad (`pass@1`, de 0,33 a 0,89) y consistencia (`pass^3`, de 0,33 a 0,67). Suben juntas, y ésa es la afirmación entera: lo que se sacó del modelo era una decisión de flujo, no un fraseo.

el carril inferior es el sensor: emite proposiciones y nunca decide el flujo



Algoritmo 2. La caminata, con la frontera modelo/código explícita. `SENSOR` es la única llamada al modelo y devuelve una proposición; todo lo demás lo decide el código.

ALGORITMO 2 Caminata determinista sobre una cadena de referencias

Entrada : pregunta q , vista del corpus V , plantel de índices I

Salida : valor pedido, o ABSTENER

```
1   $n \leftarrow \text{saltos\_declarados}(q)$            ▷ forma cerrada sobre  $q$ ;  $\emptyset$  si no declara
2   $e \leftarrow \text{sujeto\_declarado}(q)$          ▷ forma cerrada sobre  $q$ 
3   $\iota \leftarrow \text{índice\_para}(e)$            ▷ REGLA DE CREENCIAS: entidad  $\Rightarrow$  léxico
4   $u \leftarrow \text{primer\_hit\_que\_nombra}(\text{buscar}(\iota, e, \kappa), e, \emptyset)$    ▷ ancla = salto cero
5  if  $u = \perp$  then return ABSTENER
6   $\text{camino} \leftarrow \langle u \rangle$ 
7  for  $i \leftarrow 1$  to  $n$  do                 ▷ el LARGO lo fija  $q$ , no el modelo
8     $\tau \leftarrow \text{leer}(u)$ 
9     $p \leftarrow \text{SENSOR}(\tau, \text{«¿hacia dónde sigue el rastro?»})$    ▷  $\leftarrow$  única llamada
10    $p \leftarrow \text{entidad\_en}(p)$            ▷ se TIPa la salida del sensor
11    $\iota \leftarrow \text{índice\_para}(p)$ 
12    $u \leftarrow \text{primer\_hit\_que\_nombra}(\text{buscar}(\iota, p, \kappa), p, \text{camino})$ 
13   if  $u = \perp$  then return ABSTENER       ▷ cadena incompleta  $\Rightarrow$  no se responde
14    $\text{camino} \leftarrow \text{camino} \frown \langle u \rangle$ 
15 return  $\text{SENSOR}(\text{leer}(\text{camino}[n]), \text{«el valor pedido»})$    ▷  $\leftarrow$  el CÓDIGO fija cuál
```

$\text{primer_hit_que_nombra}(C, p, \text{visitadas})$:

```
16  $A \leftarrow \{c \in C \setminus \text{visitadas} : c \text{ menciona } p \text{ en forma COMPLETA}\}$    ▷ ancla
17  $M \leftarrow \{c \in C \setminus \text{visitadas} : c \text{ menciona } p \text{ en forma ABREVIADA}\}$    ▷ referencia
18 return  $\text{argmin}_{\{c \in (A \text{ si } A \neq \emptyset \text{ si no } M)\}} \text{posición\_primera\_mención}(c, p)$ 
```

Las cuatro líneas que son el aporte. La 3 y la 11 aplican la regla de creencias: una entidad nombrada se busca con el índice léxico, porque un vector denso codifica *de qué habla* un texto y sesenta documentos con la misma plantilla hablan de lo mismo. La 10 tipa la salida del LLM antes de usarla: el modelo emite prosa, y esa prosa arrastraría la consulta a clasificarse mal. La 7 pone el largo bajo control del código. Y la 4 resuelve el ancla como cualquier otro salto, en vez de preguntárselo al modelo.

Las líneas 16–18 son el desempate, y son la parte que no es obvia: entre candidatos que mencionan la entidad, el que la nombra en forma completa es el que trata sobre ella y el que la abrevia sólo la referencia; con empate, gana el que la menciona antes. Ambas pruebas son predicados sobre el índice (devuelven un booleano o una posición, nunca texto) así que no cuestan un token.

Resultado: `pointer_chase` pasa de 0,33 a 0,89 en el modo de cadenas acopladas sobre `terra`, 8 de 9 celdas, empatando al mejor brazo. Y la corrección (4) prueba el punto por sí sola: con la misma huella y los mismos resultados de búsqueda, la réplica 0 elegía el ancla correcta y recorría la cadena entera ($u=1,000$) mientras las réplicas 1 y 2 elegían otra y sacaban 0,000. Una decisión de flujo en el LLM se lleva puesto el determinismo. Lo que el algoritmo tiene después de las correcciones son `n` ramificaciones de dominio tipado, y no `d = \emptyset` (Definición 3b); la que se sacó del LLM es una, el ancla, y ésta es la que hacía discrepar a las réplicas.

La hipótesis, y es falsable: sustituir una ramificación de control delegada al modelo por un medidor determinista sobre una señal del entorno mejora la utilidad y el determinismo a la vez. `pass^k` es la métrica que faltaba para medir el segundo efecto, y sin ella la mitad de la mejora era invisible.

Lo que queda declarado y no resuelto: la réplica que todavía falla se abstiene en vez de contestar mal, que es el comportamiento buscado y que el banco puntúa igual que un error. Y las réplicas no coinciden del todo: 8 de 9, no 9 de 9.

6.1.5 La latencia serial varía 3,6× entre paradigmas y no entra en ninguna decisión

El eje nuevo no destraba nada: `react`` es el mejor Y el más rápido

el frente une a los no dominados · el primer token es igual en todos (250–410 ms); lo que cambia es cuántas llamadas van en serie

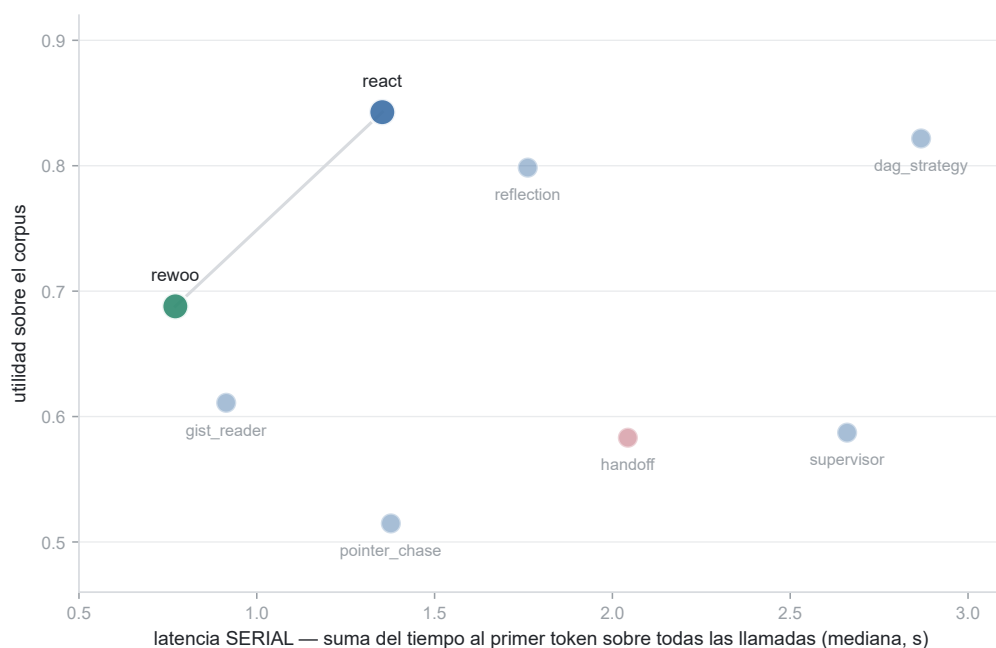


Figura 11. Latencia serial. Cada punto es un brazo: el eje horizontal es su latencia serial, la suma del tiempo al primer token sobre todas las llamadas de la celda, en mediana; el vertical, su utilidad sobre el corpus. El frente une a los no dominados. El tiempo al primer token es igual en todos, de 250 a 410 milisegundos, porque es una llamada al mismo modelo; lo que separa a los brazos es cuántas llamadas van en serie. El eje nuevo no destraba nada: `react`` es a la vez el de mayor utilidad y el más rápido entre los contendientes.

Hay dos relojes y confundirlos invalida el número. El tiempo de pared no sirve: el 63-67% de las filas de `react`` y `dag_strategy`` están por debajo de medio segundo porque son replays del caché en disco, eso mide cuánto tarda el banco en releer, no cuánto tarda el sistema en contestar. Lo que sí sirve viene del proveedor, en el objeto `usage`` de cada respuesta, y por eso el caché lo conserva: es la latencia de la llamada que efectivamente se hizo.

brazo	u	primer token	latencia serial	u por segundo
react	0,843	265 ms	1,35 s	0,62
dag_strategy	0,822	410 ms	2,87 s	0,29
rewoo	0,688	304 ms	0,77 s	0,89
supervisor	0,587	360 ms	2,66 s	0,22

El primer token es prácticamente igual en todos (250 a 410 ms, es una llamada al mismo modelo). Lo que cambia 3,6× es la latencia serial: la suma sobre todas las llamadas, o sea la parte que no se acelera con más tokens por segundo porque cada llamada espera a la anterior. Es la ley de costo `turn-driven` cobrada en tiempo del usuario en vez de en tokens, y es un eje que ninguna decisión del ruteador mira hoy.

Y no destraba nada: `react` es a la vez el de mayor utilidad y el de menor latencia serial entre los contendientes. El único que compra tiempo es `rewoo` (1,8× más rápido) y cuesta 0,165 de utilidad. Es un intercambio explícito: sólo lo compra quien tenga un techo de latencia declarado.

6.1.6 La varianza de trayectoria no crece con el número de ramificaciones

La sección anterior invita a una conjetura general, y el registro la refuta en su forma ingenua:

Si un brazo delega d decisiones de control al modelo, y cada una vuelve a salir igual entre réplicas con probabilidad q , entonces $\text{pass}^k \approx \text{pass}@1 \cdot q^d$. Más decisiones en el LLM $\Rightarrow$ menos determinismo, multiplicativamente.

Es contable: d se mide como iteraciones por celda, y `pass^3` ya está. Sobre siete de los ocho brazos del panel (`handoff` no tiene conteo de iteraciones comparable; con los ocho la correlación es `-0,237`, con los siete de la tabla `-0,244`):

brazo	decisiones	pass@1	pass^3	q implícita
dag_strategy	8,9	0,822	0,703	0,983
supervisor	8,6	0,587	0,406	0,958
reflection	5,9	0,798	0,688	0,975
react	4,3	0,843	0,734	0,968
pointer_chase	3,8	0,515	0,359	0,909
rewoo	2,0	0,688	0,531	0,879
gist_reader	1,9	0,611	0,516	0,913

La correlación entre número de decisiones y caída de `pass^3` es $r = -0,24$ con $n = 7$, débil, y con el signo contrario al que la conjetura predice: los brazos con más decisiones pierden *menos*. La explicación que el propio dato sugiere es que una decisión no sólo agrega varianza sino también una oportunidad de corregir: un brazo adaptativo que dobla mal puede volver, y uno de dos llamadas no. Los dos efectos casi se cancelan en este corpus.

Lo que sí se sostiene, y es más útil que la conjetura original, son dos cosas:

1. q está acotada lejos de 1 para todos. El máximo es 0,983 (`dag_strategy`) y el mínimo 0,879 (`rewoo`). Ninguna arquitectura de las medidas recupera la reproducibilidad por decisión, así que toda trayectoria con decisiones delegadas pierde determinismo, y la pregunta es cuánto se pierde, no si se pierde.
2. Importa más CUÁL decisión se saca que CUÁNTAS. La evidencia que lo sostiene es una intervención, no una correlación: sacar una sola decisión (el ancla) llevó a `pointer_chase` de réplicas que discrepaban (1,000 / 0,000 / 0,000) a réplicas que coinciden, sin tocar las otras. Ocho puntos de correlación no compiten con eso.

El no-determinismo no se reparte por igual entre las decisiones de una trayectoria. Contar decisiones no predice; identificar cuál decide el resultado, sí.

Ésta es una limitación declarada del análisis, no un resultado: con siete brazos y un corpus, lo correlacional no puede decidir casi nada. Lo que la sostiene es la intervención, y la intervención tiene la salvedad de §6.1.4: se desarrolló sobre las celdas en que se la mide.

6.2 Por qué rutear por nombre no tiene premio, aunque la interacción sea grande

Ésta es la sección que reordenó el programa, y lo hizo con un mecanismo preciso. La forma en que suele argumentarse que rutear conviene («hay mucha interacción entre tarea y método, así que elegir por tarea tiene que pagar») no se sostiene, y este registro muestra dónde se rompe. Lo que se sostiene en su lugar lo dice §6.3: los brazos que competirían tienen las mismas capacidades, y el premio que existe está sobre el costo (§6.4.1), no sobre la calidad.

6.2.1 Hay interacción, y es grande

Descomponiendo $u(\text{tarea}, \text{brazo}) = \mu + \alpha(\text{tarea}) + \beta(\text{brazo}) + \gamma(\text{interacción}) + \epsilon$ sobre el rectángulo de 64 tareas $\times$ 8 brazos, que es el 82% de las 78 medidas. El criterio de recorte es mecánico y está en un solo lugar del código, `bench.panel`, no se elige por sección: primero las tareas que tienen al menos 7 brazos medidos (lo que impide que una tarea corrida por un experimento parcial redefina el universo), después los brazos que cubren $\geq 95\%$ de ésas, y al final las tareas donde están todos. Los cuatro brazos que quedan afuera son los que la factibilidad poda en casi todas las celdas (`direct`, `streaming_scan`, `extract_compute`) más `graph_traverse`, que se declara infactible en el 56-59% de las celdas anchas.

Varianzas insesgadas (`ddof = 1`), con el ruido de réplica descontado de cada componente ($\epsilon/3$ de γ , $\epsilon/24$ de α , $\epsilon/192$ de β):

componente	varianza cruda	% crudo	% descontado el ruido
α , dificultad de la tarea	0,0759	45%	50%
β , calidad del brazo	0,0158	9%	11%
γ , interacción	0,0754	45%	39%
ϵ , ruido entre réplicas	0,0563		

El denominador, porque cambia el titular. Los porcentajes son sobre la varianza explicada ($\alpha + \beta + \gamma = 0,1671$ cruda). Contando ϵ , el total es $0,2235$ y γ pesa 34%. Un 45% sin denominador declarado es un número que se lee más grande de lo que es, y un 45% sin descontar el ruido de réplica que γ absorbe también.

y descontado el ruido da 0,0566, con señal/ruido 3,02. Bajo el argumento habitual, correspondería rutear.

La interacción tarea × brazo es el 45% de la varianza y vive fuera del marco:

el mejor brazo cae en un contendiente en 56 de 64 tareas

Rectángulo 64 × 8, filas agrupadas por región y columnas por media. El punto es el mejor brazo de la fila;

el marco punteado, los tres contendientes (a menos de 0,05 del mejor fijo). Oráculo por tarea 0.953, mejor fijo 0.843.

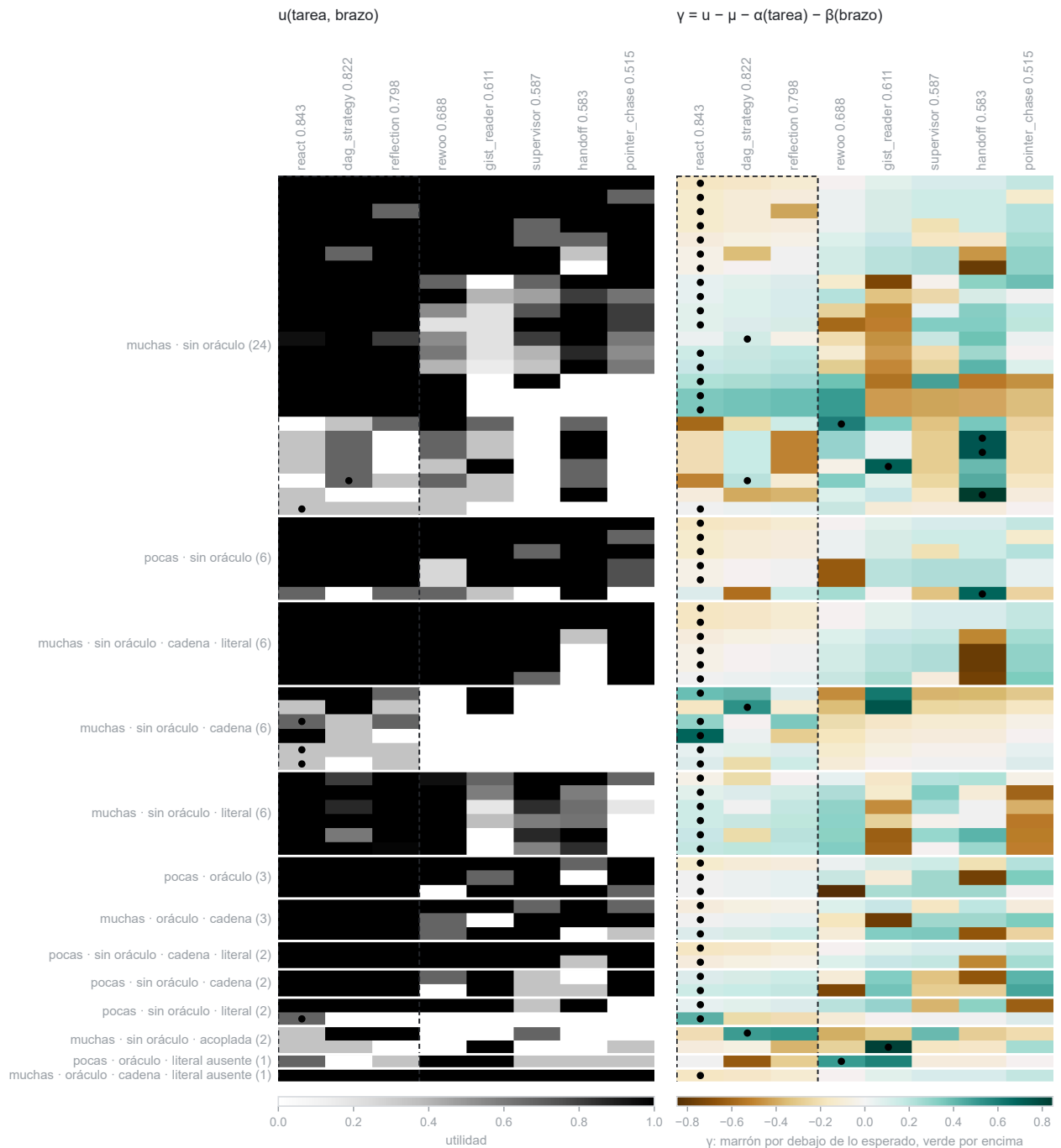


Figura 12. El rectángulo 64 × 8 celda por celda. El panel izquierdo es $u(\text{tarea}, \text{brazo})$, la utilidad de cada tarea con cada brazo, con las filas agrupadas por región y las columnas ordenadas por la media del brazo, de `react` a `pointer_chase`. El panel derecho es el residuo γ , lo que queda de esa utilidad una vez descontadas la dificultad de la tarea y la calidad del brazo: marrón donde el brazo hizo peor de lo que su media y la tarea predicen, verde donde hizo mejor. En ambos, el punto negro marca el mejor brazo de cada fila y el marco punteado encierra a los tres contendientes, los brazos a menos de 0,05 del mejor fijo. El punto cae dentro del marco en 56 de 64 tareas, y el color de γ está casi todo fuera de él: la interacción es grande y vive entre los brazos que nadie elegiría.

6.2.2 Una señal la explica y sobrevive la corrección por selección

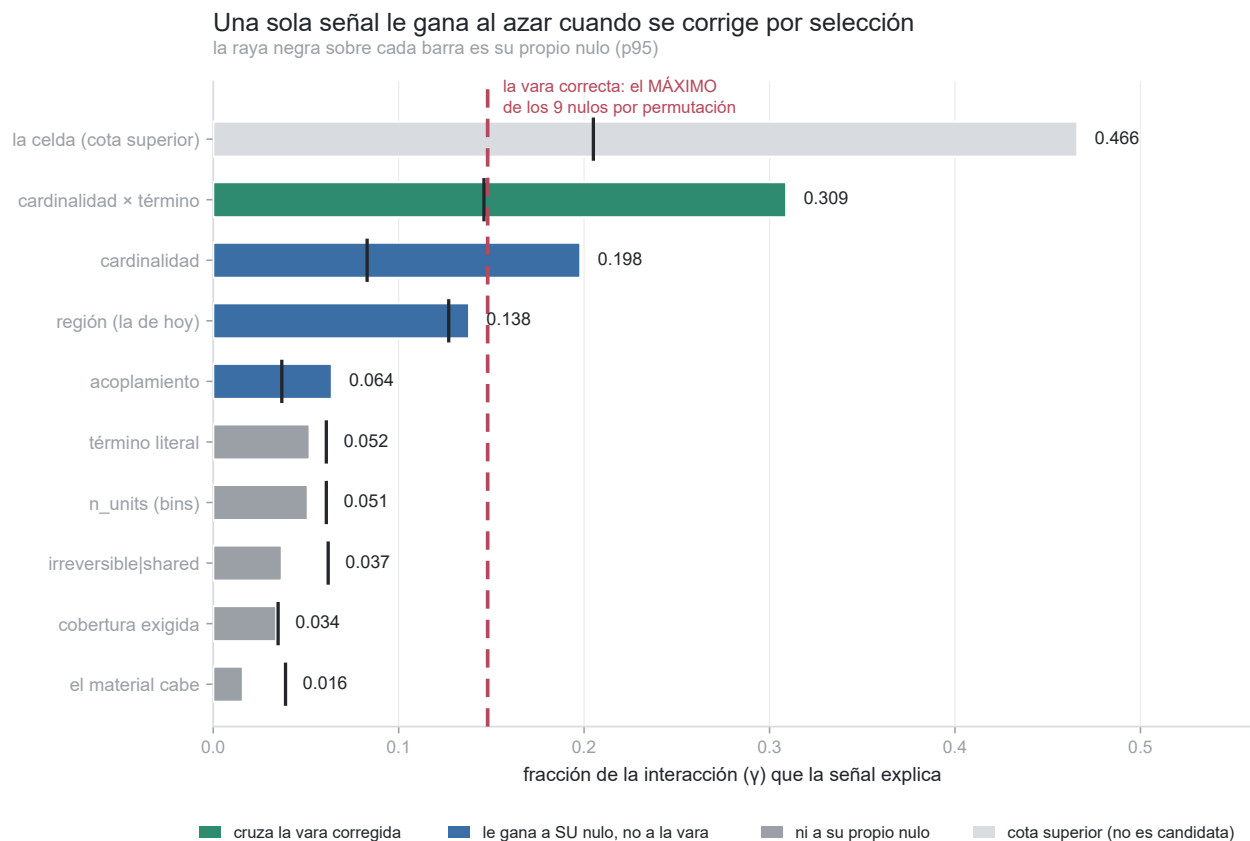


Figura 13. Qué señal explica la interacción. Una barra por señal candidata, ordenadas por la fracción de γ que explican: si el material cabe, la cobertura exigida, las banderas de riesgo, el número de unidades, el término literal, el acoplamiento, la región vigente, la cardinalidad, y `cardinalidad × término`. La raya negra sobre cada barra es su propio nulo por permutación al p95. La línea punteada vertical es la vara correcta: el máximo de los nueve nulos en cada permutación, que es lo que hay que superar cuando se probaron nueve y se eligió la mejor. La última barra es la celda misma, la etiqueta de diseño, y va como cota superior y no como candidata, porque no se conoce al decidir. Una sola señal cruza la vara corregida.

La figura ordena las señales por cuánto de γ explican, con su propio nulo por permutación dibujado como una raya negra sobre cada barra. Varias la superan, y esa comparación es exactamente la falacia que el nulo existía para evitar, porque se probaron nueve candidatas y se eligió la mejor. La vara correcta es la línea punteada: el máximo de los nueve nulos en cada permutación.

Sólo `cardinalidad × término literal` la cruza, con 0,309 y p corregido $< 0,001$ (2.000 permutaciones, así que ése es el mínimo reportable), y captura dos tercios del techo que marca la etiqueta de diseño (el modo del que salió la tarea), que está en el gráfico como cota superior, no como candidata: es la etiqueta de diseño del corpus, no se conoce al decidir, y ninguna señal real puede superarla. El procedimiento del máximo de los nulos es el maxT de Westfall y Young [1993].

Una salvedad sobre el p . La candidata ganadora se construyó después de que el eje literal se descubriera midiendo (§6.5.4), así que la familia de hipótesis efectivamente explorada es mayor que las nueve que la figura muestra, y el p corregido es optimista en esa medida.

6.2.3 Entre los contendientes el premio apenas se separa del piso; sobre los ocho brazos es neto, y nadie lo captura

$\text{var}(\gamma)$ grande $\neq$ premio de ruteo grande. El premio es $E[\max_p u] - \max_p E[u]$, y γ puede ser enorme porque los brazos malos son malos en lugares distintos. Esa estructura es real, es predecible, y no vale nada: nadie va a elegir el brazo que pierde por poco en vez del que pierde por mucho.

Lo único cobrable es la interacción entre los brazos que competirían, los tres a menos de 0,05 del mejor fijo: `react`, `dag_strategy` y `reflection`. Sobre el rectángulo:

	tres contendientes	ocho brazos
oráculo por tarea	0,901	0,953
mejor fijo (<code>react</code>)	0,843	0,843
brecha observada	+0,058	+0,110
IC95 pareado (tareas)	[+0,021, +0,090]	[+0,052, +0,161]
piso calibrado (pseudo-brazos, medias), media	0,030	0,042
piso calibrado, p95	0,047	0,065
piso conservador (<code>noise_floor</code> , réplicas sueltas)	0,069	0,052
brecha neta contra el piso calibrado, media	+0,028	+0,068
brecha neta contra el piso calibrado, p95	+0,011	+0,045
brecha neta contra el piso conservador	-0,010	+0,058

Entre los tres contendientes la brecha es chica y apenas se separa del piso: `+0,028` contra la media del piso calibrado, `+0,011` contra su p95, y negativa contra el estimador conservador. Sobre los ocho brazos la brecha es neta y positiva con cualquier estimador. Las dos lecturas conviven y ninguna contradice a la otra: el oráculo que elige entre ocho gana porque algún brazo malo resuelve una tarea que los buenos no, y el oráculo que elige entre tres casi no tiene qué elegir.

El piso de ruido, que es la cantidad de la que depende todo veredicto de esta sección. Un oráculo toma un máximo sobre estimaciones ruidosas de utilidad por tarea, y un máximo sobre estimaciones ruidosas está sesgado hacia arriba: $E[\max_p \hat{u}_p] > \max_p E[u_p]$ incluso cuando todos los paradigmas son idénticos. Así que toda brecha de oráculo medida contiene una componente de ruido, y reportarla entera exagera el premio. La estimación es directa y no modelada: para cada brazo real se toman sus tres réplicas por tarea y se construyen con ellas tantos pseudo-brazos como brazos compara el panel, y cada pseudo-brazo vale, por tarea, la media de tres réplicas remuestreadas, porque la brecha observada se computa sobre medias de celda y el piso tiene que tener la misma varianza que lo que descuenta. La brecha de oráculo entre pseudo-brazos es ruido por construcción, porque todos son el mismo brazo. Se reporta la media y el p95 sobre 400 corridas por brazo. Al lado va el estimador de `metrics.noise_floor`, que usa las tres réplicas sueltas como tres pseudo-brazos: al no promediar, cada pseudo-brazo tiene raíz de tres veces más desvío que una celda, así que es una cota conservadora y no el estimador calibrado; el test §60 de la suite lo verifica sobre un sintético, con brazos idénticos el calibrado queda a menos de 0,03 de la brecha observada y el conservador arriba. El piso crece con cuántos brazos compara el oráculo, y por eso se empareja. Y al lado va el intervalo bootstrap pareado de la brecha misma, remuestreando tareas, que es la incertidumbre de muestreo y no la de medición. Un estimador que parece natural y no sirve. Remuestrear las réplicas de cada celda real y recalcular la brecha es la distribución bootstrap del propio estadístico, cuya media es igual o mayor que la brecha observada por construcción, así que «brecha menos piso» da cero o negativo con cualquier dato; sobre un sintético de premio real +0,50 devuelve neto +0,005. El test §60 de la suite lo verifica. El fenómeno tiene nombre en la literatura de decisión: es la maldición del optimizador, el sesgo positivo del valor esperado de la alternativa elegida cuando se elige por el máximo de estimaciones ruidosas [Smith y Winkler, 2006], y la misma desigualdad de Jensen que la sostiene. Lo propio de este paper es el estimador directo por pseudo-brazos, no el fenómeno.

Y ninguna señal separa a los tres contendientes entre sí: todas con $p > 0,29$. Sólo la etiqueta de diseño lo logra ($p = 0,007$), y ésa no se conoce al decidir.

La interacción es real y vive sobre todo entre los brazos que nadie elegiría. Reportar $\text{var}(\gamma)$ como evidencia de que rutear conviene mide la estructura equivocada. Y reportar la brecha de oráculo sobre ocho brazos como premio alcanzable mide un oráculo, no una política: §6.2.4 y §6.5 miden qué pasa cuando alguien intenta cobrarla.

6.2.4 El premio no reaparece agrupando paradigmas en familias

Una respuesta natural es que las señales quizá no separen individuos pero sí grupos: un ruteador que elige familia y después toma el más barato de la familia es otro ruteador, con otro premio. Se probó, declarando las familias desde el código (por cuándo el brazo decide su próxima llamada) y no desde el resultado:

familia	brazos	u media
adaptativo	react , reflection	0,818
plan_fijo	dag_strategy , rewoo	0,750
canal_con_pérdida	gist_reader , handoff , pointer_chase , supervisor	0,571

Valuar una familia por su máximo sería hacer trampa dos veces (le regala una elección por tarea que el ruteador de familias no puede hacer, y premia a la familia más numerosa por puro sesgo del máximo), así que cada familia se representa por su brazo de mejor media, elegido una vez sobre todo el panel.

familia	gana en
adaptativo	54 de 64 tareas (84%)
plan_fijo	6 de 64 (9%)
canal_con_pérdida	4 de 64 (6%)

oráculo	premio	IC95 pareado	piso calibrado, media	piso, p95	neto, media	neto, p95
rutear familias (piso de 3)	+0,079	[+0,032, +0,121]	0,027	0,047	+0,053	+0,032
rutear brazos (piso de 8)	+0,110	[+0,052, +0,161]	0,042	0,065	+0,068	+0,045

Y hay que hacer la resta que el paper acaba de enseñar, con el piso emparejado al número de brazos que cada oráculo compara, porque un piso de tres no sirve para un máximo sobre ocho. Los dos netos son positivos, contra la media del piso y contra su p95. Y aun así ninguno es cobrable, por una razón distinta de la de §6.2.3:

allá el premio no existía; en este panel existe y no hay con qué agarrarlo. El premio es lo que capturaría un oráculo, y un oráculo no es una política: ninguna señal predice la familia ganadora. La mejor, cardinalidad , da información mutua 0,092 y no sobrevive la corrección por selección ($p = 0,365$).

Un premio sin señal que lo prediga es una cota superior, no un resultado.

No falla porque las señales sean débiles: falla porque no hay frontera que cruzar. Una familia que gana el 84% de las veces no es un cluster que rutear, es un default.

6.2.5 La región de una tarea depende del modelo que la sensó

El vocabulario de región vigente tiene cinco ejes. Cuatro son `COMPUTED` (se calculan del texto de la pregunta y del material) y uno, el acoplamiento, es `ELICITED` : lo emite el modelo. Sobre las 26 tareas del corpus `gold_transfer` medidas con dos familias de modelo, `nano` y `luna` (dato del registro anterior a la campaña, sin script que lo reproduzca):

eje de la región	procedencia	cambia entre modelos
cardinalidad	COMPUTED	0%
oráculo disponible	COMPUTED	0%
acoplamiento	ELICITED	27%
continuidad	COMPUTED	0%
término literal	COMPUTED	0%

Los cuatro ejes computados son idénticos en las 26 tareas; el elicitado cambia en 7. La misma pregunta, con el mismo material, cae en regiones distintas según qué modelo la sensó:

`few/oracle/loose/flat/no_lit` con una familia y `few/oracle/mixed/flat/no_lit` con la otra.

Es la clave de la política corriendo aguas abajo del LLM, no ruido de medición. Una política keyed en región hereda la estocasticidad del modelo en su propia clave, dos corridas del mismo request pueden buscar en dos filas distintas de la tabla. Es el mismo mecanismo de §6.1 aplicado un nivel más arriba: una ramificación delegada al LLM, sólo que la ramificación es qué política se consulta.

Una clave de política tiene que ser `COMPUTED`. Un eje elicitado describe lo que el modelo cree sobre la tarea, no la tarea, y usarlo para indexar convierte la tabla de decisión en una variable aleatoria.

Y da un mecanismo verificable para el fracaso del ruteo por región que §6.2.3 mide: parte de la señal que una política de este tipo puede aprender está en un eje que no se reproduce entre modelos, así que lo aprendido sobre un LLM no transfiere a otro aunque el corpus sea el mismo. Es la Proposición 5 en su caso (2), medida.

Cierre de la sección, y puente. §6.2 mostró tres cosas sobre el ruteo por identidad: que la interacción es grande y vive entre brazos dominados, que entre los contendientes no hay premio de calidad ni señal que los separe, y que la clave elicitada le agrega varianza a la tabla. Las tres tienen la misma causa, que §6.3.1 ya dibujó y §6.3 formaliza: los contendientes caen en el mismo punto del espacio de capacidades. Son el mismo brazo para decidir. Lo que sigue es qué pasa cuando la clave deja de ser un nombre y pasa a ser lo que el brazo puede hacer.

6.2.6 Fuera de muestra: el corpus held-out completo

Todo lo anterior es en muestra. El criterio de éxito declarado del producto es otro (brecha de oráculo neta positiva sobre datos que el sistema nunca vio).

El held-out corre por el mismo camino de código que la campaña, con el corpus como argumento del runner. Es una condición y no un detalle de implementación: un held-out medido con otro arnés no mide generalización, mide dos arneses.

6.2.7 La brecha de oráculo es positiva y neta en los tres estratos, y ninguna política la captura

26 tareas × 12 paradigmas × 3 réplicas, 936 filas de las cuales 282 infactibles, 48,5M tokens, 135 minutos, cero errores de infraestructura. El rectángulo queda en 24 tareas × 8 paradigmas (el 92% de las medidas) bajo el mismo criterio mecánico que el resto del paper.

paradigma	u
dag_strategy	0,802
react	0,777
reflection	0,722
supervisor	0,666
gist_reader	0,623
rewoo	0,613
pointer_chase	0,498
handoff	0,466

	base + w4	+ w16	+ w48
tareas del panel	12	18	24
oráculo por tarea	0,972	0,981	0,931
mejor fijo	0,833	0,813	0,802
brecha observada	+0,139	+0,168	+0,128
IC95 pareado (tareas)	[+0,000, +0,194]	[+0,034, +0,222]	[+0,029, +0,221]
piso calibrado (8), media	0,018	0,023	0,026
piso calibrado (8), p95	0,083	0,074	0,071
brecha neta contra la media	+0,121	+0,145	+0,102
brecha neta contra el p95	+0,056	+0,094	+0,057
piso conservador (noise_floor)	0,031	0,042	0,037

La brecha de oráculo supera su piso de ruido en los tres estratos, sobre 5, 20 y 60 unidades de alcance, contra la media del piso calibrado, contra su p95 y contra el estimador conservador, y su intervalo de muestreo sólo toca el cero en el estrato más chico. Fuera de muestra el oráculo entre ocho brazos tiene premio, como lo tiene en muestra (§6.2.3, +0,110 bruto). Y lo que no tiene es quien lo cobre: los tres intentos de capturarlo con una política por identidad (§6.5) perdieron contra el mejor fijo o empataron con el fallback, y ninguna señal disponible al decidir predice la familia ganadora (§6.2.4). El estimador de piso que hace posible esta lectura, y el que no, están en §6.2.3.

Sobre los tres contendientes del held-out, en cambio, la lectura de §6.2.3 se repite: los dos primeros quedan en 0,802 y 0,777, con una brecha de 0,025 que el piso p95 (0,071) cubre entera.

El piso emparejado por número de brazos crece con cuántos brazos compara el oráculo y con la dispersión de las réplicas, y no decrece con el número de tareas como decrecería el error de una media. Por eso se recomputa por panel y no se proyecta.

El piso de un premio de máximo se recomputa, no se proyecta. Cualquier cálculo de potencia que use la intuición de una media subestima el piso justo donde la comparación se decide.

6.2.8 El plantel no tiene un ganador estable, y eso es el resultado

Sobre el corpus completo los dos primeros quedan en 0,802 y 0,777 , con una brecha de 0,025 contra un piso p95 de 0,071 . Sobre sub-paneles del mismo corpus el orden entre los punteros se permuta (tres paneles anidados dan tres punteros distintos) sin que ninguno se despegue.

No es que el mejor paradigma fijo cambie entre corpus: es que no hay uno. La afirmación «el paradigma X es el mejor» exige una separación que este registro no tiene, y un panel más chico no la produce, la fabrica, porque el máximo de pocas muestras es sesgado hacia arriba.

Es el mismo mecanismo de §6.2.3 mirado desde otro lado: donde los candidatos empatan, el máximo por tarea es el sesgo del máximo y no una elección mejor.

6.2.9 Costo de la corrida

estrato	tareas	material medio	celdas	tokens	tiempo
base + w4	14	91k	504	16,0M	75 min
w16	6	150k	216	13,7M	27 min
w48	6	451k	216	18,8M	33 min
total	26		936	48,5M	135 min

Las seis tareas de w48 cruzan el umbral de contexto largo, así que pagan el doble de tarifa sobre el request entero: es el acantilado de §6.2.6 aplicado a la propia corrida, y la razón de que el estrato más grande cueste 1,4× lo que su material sugiere.

Puente. Este held-out mide el oráculo, no una política: dice que hay premio para quien pueda elegir el brazo correcto por tarea, y no dice cómo elegirlo. Los tres episodios de §6.5, sobre otro modelo y con vocabularios anteriores, son los intentos de cobrarlo con una política por identidad de paradigma, y los tres perdieron. La diferencia entre §6.2.6 y §6.5 es la diferencia entre un oráculo y una clave: el primero ve la respuesta, la segunda ve la región.

6.3 Qué es aprendible: la capacidad, no la identidad del paradigma

P15 , la predicción registrada de que rutear por región hacia un nombre de paradigma le ganaría al mejor fijo, se refutó: mapear ontología de la pregunta → nombre de paradigma perdió -0,087 contra el mejor fijo. La sección anterior explica por qué ese premio no existía; ésta propone el eslabón que faltaba y lo somete a la prueba que lo puede matar.

ontología de la pregunta → capacidades que EXIGE → brazos que las tienen

Doce capacidades declaradas desde el código, cada una con la medición que la justifica y con el sitio donde se ve: PAYLOAD_COMPLETO , ADAPTA , COSTO_NO_ESCALA_CON_ALCANCE , LECTURA_SIN_PERDIDA , CONTEXT_VISION , AUTOCOMPACTA , COBERTURA_GARANTIZADA , VERIFICA_Y_REPLANIFICA , RESOLVER_REFERENCIA , LARGO_GOBERNADO_POR_CODIGO , ELIGE_INDICE_POR_CONSULTA y ABSTIENE_SIN_PRUEBA . Las cuatro últimas las destapó resolver la celda de cadenas acopladas, y ninguna es visible desde la taxonomía de control de flujo.

CONTEXT_VISION es que cada llamada ve todo lo que el request leyó hasta ahí, en crudo o compactado: ninguna vuelta pierde lo que trajo la anterior. No es **PAYLOAD_COMPLETO** (una llamada puede tener todo el alcance) ni **ADAPTA** (puede cambiar lo que pide): **supervisor** adapta y no la tiene, porque cada sub-agente arranca con una ventana de ocho unidades; **rewoo** tiene el payload en el solver y no la tiene, porque sus pasos no se ven entre sí. Es la propiedad que la ley de costo mide desde la factura, el costo por llamada de **react** sube 3,7 a 3,9× entre 3 y 12 llamadas porque reenvía el hilo entero (§6.4.3), y la que explica que el paradigma determine cuánta evidencia llega a la llamada que responde.

AUTOCOMPACTA es que el arnés reduce ese hilo por su cuenta, de forma determinista, reemplazando lo ya leído y anotado por una referencia: es la única forma medida de tener visión del hilo sin pagarlo entero en cada vuelta. Ofrecida como herramienta opcional, el modelo escribió una nota, compactó una vez y nunca planificó en 28 filas; la versión incondicional desde el código está implementada y no corrió en la campaña, así que ningún brazo del rectángulo la tiene y hoy es un hueco del catálogo del mismo tipo que el de ausencia.

Y el catálogo encuentra un hueco sin correr nada: ningún brazo del plantel junta **COBERTURA_GARANTIZADA** con **ABSTIENE_SIN_PRUEBA**, que es lo que una pregunta de ausencia exige. Eso es para lo que sirve declarar capacidades en vez de medir paradigmas, predice sobre un brazo que todavía no existe. Es una predicción registrada, no un resultado: el brazo no está construido y §9.1 lo pone primero en el orden de lo que sigue.

6.3.1 El espacio de capacidades

El espacio de capacidades: cada brazo, un punto
el tamaño es su utilidad; el color, de qué es función su costo

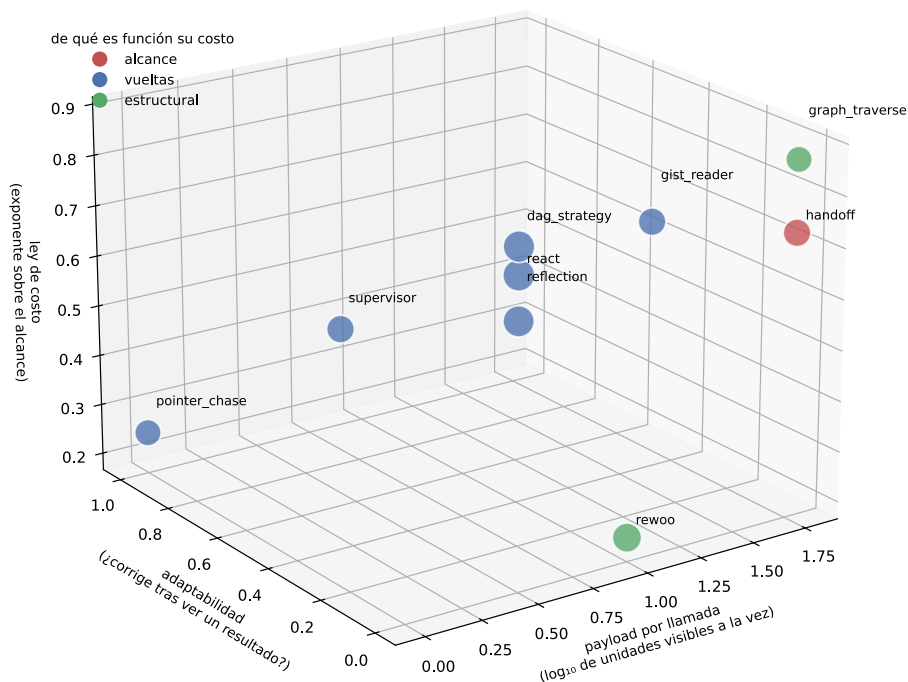


Figura 14. El espacio de capacidades. Cada brazo es un punto en tres ejes: cuántas unidades ve el modelo en una llamada (el payload), si puede corregir el plan tras ver un resultado (la adaptabilidad), y de qué es función su costo. Los dos primeros se leen del código, y por eso ubican a un brazo que todavía no corrió; el tercero se mide. `react`, `dag_strategy` y `reflection` caen prácticamente en el mismo punto, y ésa es la lectura de la figura: los tres contendientes son el mismo brazo para decidir.

La taxonomía de control de flujo («plan-ejecuta», «supervisor», «cadena») no predice rendimiento. Lo que sí predice es qué capacidades le da cada topología al modelo, y son tres:

eje	qué es	de dónde sale
payload por llamada	cuántas unidades ve el modelo de una vez	del código
adaptabilidad	¿puede corregir el plan tras ver un resultado?	del código
ley de costo	de qué es función su costo	medida

Las dos primeras se leen del código, y por eso ubican a un brazo que todavía no se corrió, cosa que una tabla de resultados no puede hacer. El dibujo tiene tres ejes porque son los que la contradicción y el ancho exigen; la visión de contexto, que cada llamada vea el hilo entero, es una cuarta propiedad del código y entra al catálogo de §6.3.2, no a esta figura.

El caso que las valida es la pregunta de contradicción, donde la respuesta es una relación entre dos unidades y ninguna unidad la contiene:

capacidades	utilidad
payload ≥ 2 unidades y adaptabilidad	0,71 – 0,91
sólo payload	0,13
ninguna	0,00 – 0,40

`handoff` lee las dos unidades relevantes y saca 0,067, porque cada sub-agente ve su mitad y ninguna llamada tiene el par. La capacidad que cuenta es tenerlas juntas, no leerlas. Y `rewoo` saca 0,133 aunque *puede* tenerlas juntas, porque le falta la otra, poder elegir cuáles dos exige ver un resultado antes de pedir el siguiente.

En la figura, `react`, `dag_strategy` y `reflection` caen prácticamente en el mismo punto. Eso no es un defecto del dibujo: son el mismo brazo para decidir, y por eso sus utilidades quedan dentro de 0,05 entre sí.

6.3.2 Definiciones: capacidad, exigencia, brazo capaz

La interfaz aprendible: de la pregunta a los brazos capaces, sin pasar por un nombre

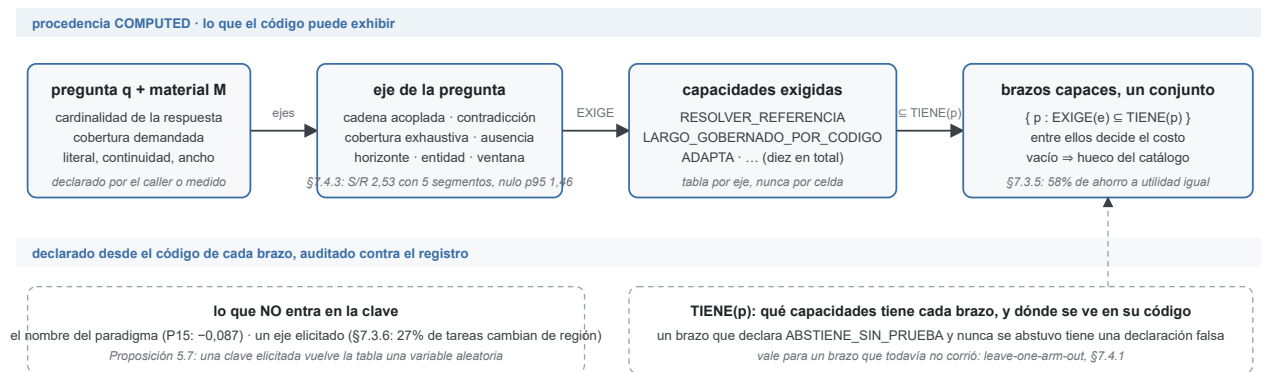


Figura 15. La interfaz aprendible en una imagen. El carril de arriba es **COMPUTED** de punta a punta: los ejes de la pregunta salen de lo que el caller declara o de lo que se mide sobre el material, la tabla de exigencias va del eje a la capacidad y nunca al nombre, y la salida es un conjunto de brazos capaces entre los que decide el costo. El carril de abajo es lo que cada brazo declara desde su código y una auditoría contrasta con lo corrido. Lo que no entra en la clave está a la izquierda: el nombre del paradigma y cualquier eje elicitado.

Definición 6 (Capacidad). Una *capacidad* es un predicado booleano sobre el código de un brazo, $c(p) \in \{0, 1\}$, que se declara con tres cosas: qué es, el sitio del código donde se ve, y el número medido que se explica con ella y no sin ella. El *catálogo* C es el conjunto de capacidades declaradas; hoy tiene diez.

Definición 7 (Lo que tiene un brazo). $TIENE(p) = \{ c \in C : c(p) = 1 \}$. Es una propiedad del código y no del registro, así que está definida para un brazo que todavía no corrió. La auditoría contrasta cada $c \in TIENE(p)$ con la conducta registrada: un brazo que declara **ABSTIENE_SIN_PRUEBA** y nunca se abstuvo tiene una declaración falsa.

Definición 8 (Lo que exige un eje). Para cada eje e de la ontología de la pregunta, $EXIGE(e) \subseteq C$ es el conjunto de capacidades sin las cuales el eje no se satisface. Se declara por eje y no por celda del corpus, porque una celda es un artefacto de este banco y un eje es una propiedad de la pregunta. Hoy **EXIGE** es una conjunción; §6.3.4 anota el contraejemplo que pide una disyunción de conjunciones.

Definición 9 (Brazos capaces). Para una pregunta con ejes activos $E(q, M)$, $capaces(q, M) = \{ p : \bigcup_{e \in E(q, M)} EXIGE(e) \subseteq TIENE(p) \}$. Es un conjunto y no un ranking: entre los capaces decide el costo, la utilidad esperada o lo que la política diga. Si $capaces(q, M) = \emptyset$, el catálogo tiene un hueco, y eso es información antes de gastar.

Las cuatro definiciones son **COMPUTED** en el sentido de la Definición 4 siempre que los ejes de $E(q, M)$ lo sean, así que una política keyed en capacidades hereda la Proposición 5 (1).

brazo	capacidades declaradas
direct	PAYLOAD_COMPLETO , LECTURA_SIN_PERDIDA , COBERTURA_GARANTIZADA , CONTEXT_VISION
react , reflection	PAYLOAD_COMPLETO , ADAPTA , LECTURA_SIN_PERDIDA , CONTEXT_VISION
dag_strategy	las cuatro anteriores más VERIFICA_Y_REPLANIFICA , ABSTIENE_SIN_PRUEBA
rewoo	COSTO_NO_ESCALA_CON_ALCANCE , LECTURA_SIN_PERDIDA
gist_reader	COBERTURA_GARANTIZADA , y el gist es la pérdida
handoff	COBERTURA_GARANTIZADA , y cada sub-agente ve su mitad
supervisor	ADAPTA , con ventana recortada de ocho unidades
pointer_chase	ADAPTA , RESOLVER_REFERENCIA , LARGO_GOBERNADO_POR_CODIGO , ELIGE_INDICE_POR_CONSULTA , ABSTIENE_SIN_PRUEBA
graph_traverse	RESOLVER_REFERENCIA , por su índice de entidades, y nada más
extract_compute	COBERTURA_GARANTIZADA , COSTO_NO_ESCALA_CON_ALCANCE
streaming_scan	COBERTURA_GARANTIZADA

Que react , reflection y dag_strategy compartan las tres primeras es lo que §6.2 midió como empate de calidad entre contendientes, ahora leído desde la tabla.

Por qué se declara y no se mide. Una capacidad es una propiedad del código, no del registro: se lee del paradigma y por eso vale para un brazo que todavía no corrió. Cada declaración nombra dónde en el código se ve (RESOLVER_REFERENCIA de pointer_chase es la función que exige que un candidato nombre a la entidad perseguida, VERIFICA_Y_REPLANIFICA de dag_strategy es su tope de replanificaciones) y una auditoría contrasta lo declarado contra lo corrido: un brazo que declara ABSTIENE_SIN_PRUEBA y nunca se abstuvo tiene una declaración falsa, y eso se ve. Cada una de las diez entra al catálogo con el número que se explica con ella y no sin ella; una capacidad sin ese número es vocabulario.

El eslabón del medio, declarado por eje de la pregunta y no por celda del corpus. Una celda es un artefacto de este banco; un eje es una propiedad de la pregunta, y por eso la tabla va de eje a capacidad y nunca a nombre de paradigma:

eje de la pregunta	capacidades que exige
cadena acoplada	RESOLVER_REFERENCIA , LARGO_GOBERNADO_POR_CODIGO , ADAPTA
contradicción	PAYLOAD_COMPLETO , LECTURA_SIN_PERDIDA
cobertura exhaustiva	COBERTURA_GARANTIZADA
ausencia	COBERTURA_GARANTIZADA , ABSTIENE_SIN_PRUEBA , ningún brazo las junta
horizonte desconocido	ADAPTA
entidad nombrada	ELIGE_INDICE_POR_CONSULTA
material mayor que la ventana	COSTO_NO_ESCALA_CON_ALCANCE

La función que la consume devuelve el conjunto de brazos capaces, nunca un ganador: entre los capaces decide el costo, la utilidad esperada o lo que la política diga. Mezclar «puede» con «conviene» es lo que P15 hizo mal al mapear ontología a nombre.

6.3.3 La prueba es dejar un BRAZO afuera, no una tarea

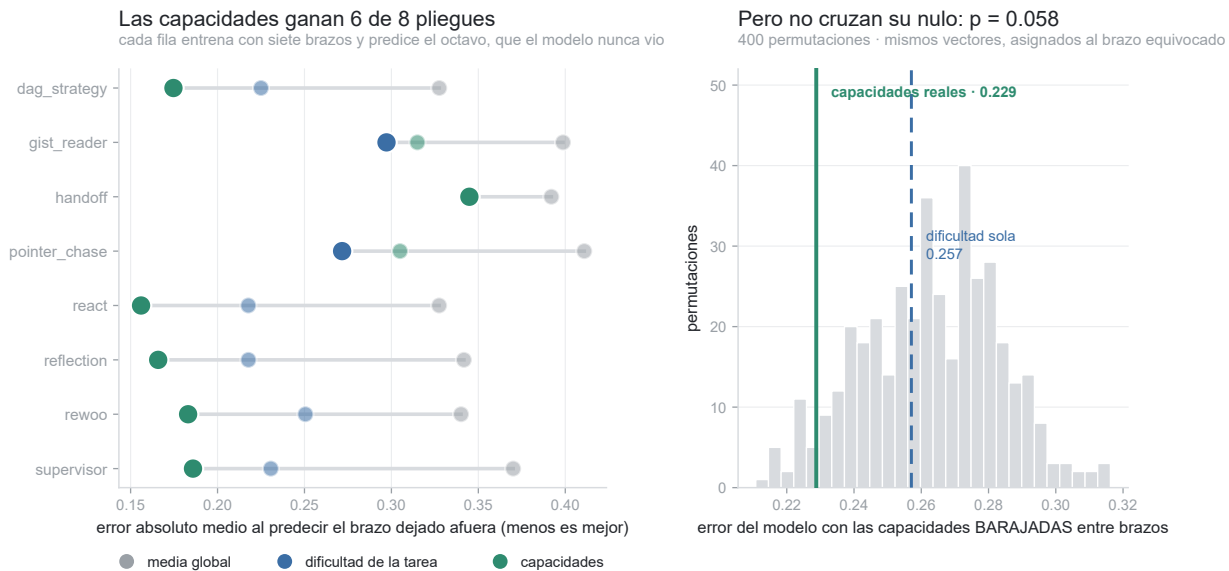


Figura 16. El EDA de capacidades, en dos paneles. El izquierdo tiene una fila por brazo: cada fila entrena con los otros siete y predice al octavo, que el modelo nunca vio, y muestra el error absoluto medio de tres predictores, la media global, la dificultad de la tarea sola y las capacidades. Las capacidades ganan 6 de 8 filas. El derecho es el nulo: el error del mismo modelo con las capacidades barajadas entre brazos, mismos vectores asignados al brazo equivocado, sobre 400 barajadas, con el error real (0,229) y el de la dificultad sola (0,257) marcados. El error real queda en la cola del nulo sin cruzarla: $p = 0,058$ con las 400 barajadas, 0,066 con las 40.320 permutaciones exactas.

La asimetría es el punto: un modelo con la identidad del paradigma no puede decir nada de un brazo que no vio (no tiene parámetro para él, y es un límite estructural, no de ajuste). Uno con capacidades sí, porque el brazo nuevo trae su vector declarado del código.

El modelo de capacidades, para que se pueda reproducir: la predicción para una celda es la dificultad de la tarea (media de la celda sobre los siete brazos de entrenamiento) más la suma de un efecto por cada capacidad que el brazo declara, con los efectos ajustados por mínimos cuadrados sobre los siete brazos con una contracción mínima hacia cero (`_eda_capacidades.py`); once efectos sobre siete brazos por pliegue (la doceava capacidad no la tiene ningún brazo), y por eso el nulo de §6.3.4 es obligatorio.

modelo	MAE al predecir el brazo dejado afuera
media global	0,364
dificultad de la tarea sola	0,257
capacidades	0,229
identidad del brazo sin dificultad de la tarea (<i>viendo al brazo</i>)	0,339
identidad del brazo aditiva: dificultad de la tarea más efecto del brazo (<i>viendo al brazo</i>)	0,225

La quinta fila es la que decide: la cuarta compara un modelo sin término de tarea contra uno con él, y eso no es una comparación. Con el mismo término de tarea, la identidad que ve al brazo dejado afuera predice mejor que las capacidades (0,225 contra 0,229). Lo que las capacidades tienen que la identidad no tiene es que existen para un brazo que no corrió; lo que no tienen, hoy, es mejor precisión que saber cuál es el brazo.

Y por pliegue, porque con ocho puntos la media sola esconde dónde gana y dónde no. El asterisco marca el mejor de cada fila entre los tres primeros modelos:

brazo dejado afuera	media global	dificultad de la tarea	capacidades
<code>dag_strategy</code>	0,328	0,225	0,175 *
<code>gist_reader</code>	0,399	0,297 *	0,315
<code>handoff</code>	0,392	0,346	0,345 *
<code>pointer_chase</code>	0,411	0,272 *	0,305
<code>react</code>	0,327	0,218	0,156 *
<code>reflection</code>	0,342	0,218	0,166 *
<code>rewoo</code>	0,340	0,251	0,183 *
<code>supervisor</code>	0,370	0,231	0,186 *

Las capacidades ganan 6 de 8 pliegues y bajan el error contra la dificultad de la tarea sola. Los dos que pierden son los dos brazos con canal con pérdida más marcado, `gist_reader` y `pointer_chase`, donde la dificultad de la tarea predice mejor que las capacidades declaradas. Dice que el catálogo describe peor a los brazos que tiran información antes de saber cuál hacía falta, y eso es una capacidad que falta declarar o una que está mal declarada.

La cuarta fila no es un techo: sale peor que capacidades pese a ver al brazo, porque ignora α (la dificultad de la tarea, el 41% de la varianza). Saber qué brazo es, sin saber qué pregunta es, predice mal. Ese fracaso es parte del argumento: la identidad del paradigma no es una buena representación ni cuando se la deja mirar la respuesta.

6.3.4 El efecto no cruza su nulo de capacidades barajadas

El nulo correcto no es la media global: es barajar las capacidades entre brazos. Mismos vectores, mismo número de rasgos, misma estructura, asignados al brazo equivocado. Si el modelo con capacidades reales

no le gana a ése, lo que mide es la capacidad de ajustar, no la de transferir.

	MAE
capacidades reales	0,2288
nulo de capacidades barajadas, media	0,2631
nulo, p5	0,2274
p	0,058

No cruza. El script que lo computa imprime «no transfieren en este corpus», y el paper lo llama sugestivo; las dos lecturas son del mismo número, `p = 0,058` con 400 barajadas. Con las 40.320 permutaciones exactas de los conjuntos de capacidades entre los ocho brazos (`_ontologia_nulo.py`), el `p` exacto es `0,066` , la media del nulo `0,265` y su p5 `0,225` , por debajo del `0,229` real. Queda como resultado sugestivo y no establecido: ocho brazos son ocho puntos, y con esa `n` la prueba no puede decidir. Lo que la sentenciaría son más brazos, no más tareas.

Y la tabla ya tiene su propio contraejemplo anotado. `EXIGE` declara que una cadena acoplada pide `RESOLVER_REFERENCIA` y `LARGO_GOBERNADO_POR_CODIGO` , y bajo esa regla el único candidato es `pointer_chase` . Pero `dag_strategy` saca `0,89` en ese modo sobre `terra` sin ninguna de las dos: llega por otra ruta, con `VERIFICA_Y_REPLANIFICA` para insistir y `ABSTIENE_SIN_PRUEBA` para no contestar cuando no llegó. A `EXIGE` le falta expresar rutas alternativas (hoy es una conjunción, y la realidad admite «A y B, o bien C y D»). Se deja como conjunción y con el contraejemplo escrito, porque una tabla que se arregla sola para tapar su propio contraejemplo deja de ser falsable.

6.3.5 La ontología de la pregunta separa más con menos segmentos

La otra mitad de la interfaz es qué exige la pregunta, y la pregunta que motivó esta medición fue si la segmentación de la política tenía que ser por ontología en vez de por estructura del material. Se contestó con el registro ya pagado, sin una llamada nueva. Y antes del número, la salvedad que gobierna toda la sección: la segmentación ontológica se mide acá usando la etiqueta de diseño, el modo del que salió cada tarea, que se conoce al construir el corpus y no al decidir. Lo que sigue establece que hay estructura ontológica; no establece que se la pueda ver desde un request.

Un eje de la ontología es una propiedad de la pregunta tipada con vocabulario cerrado, nunca con prosa libre: cardinalidad de la respuesta (singular, enumerativa, agregada), cobertura demandada (suficiente, exhaustiva), acoplamiento requerido, ausencia, vigencia, contradicción. Cada eje entra con su modo de falla propio; un eje sin modo de falla no es un eje. Dos ya son `COMPUTED` desde lo que el caller declara; los demás tienen a `ELICITED` como techo.

La cuenta: dentro de cada segmento, varianza entre brazos contra ruido entre réplicas. Si un segmento agrupa tareas donde gana el mismo brazo, la señal sube y el ruido no, que es exactamente lo que una política necesita de su clave.

Cómo se mide, y con qué control. La cantidad es, por segmento, la varianza entre brazos de sus medias por brazo, descontado el ruido esperado de una media sobre tantas celdas con tres réplicas, agregada ponderando por tareas y relativa a la partición trivial ($S/R = 1,00$). El control por número de segmentos es el nulo por permutación de §6.2.2 aplicado a cada segmentación: se barajan las etiquetas de segmento entre las 64 tareas conservando los tamaños de cada segmento, 2.000 veces, y se recalcula. Eso controla

por número y por tamaño de segmentos sin inventar una normalización. Sobre el rectángulo de la campaña (`bench/analysis/_ontologia_nulo.py` , 2026-09-01):

segmentación	segmentos	S/R	nulo, media	nulo, p95	p	exceso sobre el nulo
ninguna, todo junto	1	1,00				
región, sólo el primer eje	2	1,16	1,07	1,17	0,060	0,09
región, vocabulario vigente	13	2,59	1,83	2,12	< 0,001	0,76
cardinalidad × cobertura, declaradas por el caller	5	2,20	1,28	1,46	< 0,001	0,92
ontología, eje principal	5	2,53	1,28	1,46	< 0,001	1,25
ontología, contradicción binaria	2	1,31	1,07	1,18	0,002	0,24
ontología, horizonte binario	2	1,33	1,07	1,18	< 0,001	0,26
modo del corpus, etiqueta de diseño	10	3,33	1,62	1,89	< 0,001	1,71

Tres lecturas, y las tres con el nulo al lado. La ontología por eje principal, con cinco segmentos, extrae casi la misma separación que la región vigente con trece (2,53 contra 2,59) y le saca más al azar que la región (1,25 contra 0,76 de exceso sobre la media del nulo), porque un nulo con cinco segmentos espera 1,28 y uno con trece espera 1,83 . Una clave hecha sólo de lo que el caller declara, cardinalidad de la respuesta por cobertura demandada, ambas COMPUTED , también supera a la región en exceso sobre su nulo (0,92) con cinco segmentos. Y cada eje binario suelto supera su propio nulo. La etiqueta de diseño es el techo, como en §6.2.2.

El caso que lo vuelve concreto. Dos modos del corpus, horizonte desconocido y vigencia, son idénticos en los ejes computables del vocabulario de región, caen en la misma región, y tienen efecto opuesto sobre los brazos. Lo que los separa es un eje que la ontología ya nombraba y la región no puede ver: si la pregunta pide el valor vigente o el valor a una fecha. Una política keyed en región promedia sobre las dos poblaciones; una keyed en el eje, no.

Lo que esto establece y lo que no. Establece que hay estructura ontológica en el corpus, que la política debería segmentar por ella, y que una clave COMPUTED de dos ejes declarados por el caller ya la captura en parte. No establece que los demás ejes se puedan detectar en un request real: el eje principal de la tabla usa, para tres de los siete ejes, el modo del que salió la tarea, que se conoce al construir la tarea y no al decidir. Cardinalidad y cobertura son COMPUTED desde el request; los otros tienen a ELICITED como techo, y ése es el siguiente experimento (§9.1).

6.4 Lo que sí paga, y qué predice

Lo anterior dice sobre qué no hay premio. Esto dice dónde sí lo hay: en el costo entre brazos capaces, en el acuerdo entre brazos como verificador, y en los predictores de conducta que el registro entrega.

6.4.1 El desempate por costo entre brazos capaces, y con qué clave

Sobre calidad la decisión entre contendientes no tiene premio separable del piso. Sobre costo a utilidad igualada, el mismo registro da un ahorro grande con una pérdida chica, evaluado fuera de muestra (leave-one-task-out sobre el rectángulo, agrupando por la señal y eligiendo en las otras tareas del grupo el brazo más barato que empata; `bench/analysis/_predictores.py`, 2026-09-01):

señal	utilidad vs mejor fijo (react , 0,843, 112.851 tokens)	ahorro
cardinalidad × término	+0,000	58%
región (vocabulario vigente)	−0,079	76%
n_units	−0,101	13%

Sobre el rectángulo la señal `cardinalidad × término` iguala al mejor fijo en utilidad y ahorra 58% de los tokens. El número depende del recorte del panel, y el que va es el del rectángulo mecánico; el Apéndice C.1 registra los de paneles anteriores. La pregunta «qué paradigma da la mejor respuesta» está agotada en este corpus; la pregunta «cuál es el más barato que da una respuesta indistinguible» no.

Y lo que la política entera, no una señal suelta, consigue sobre ese objetivo, con la clave que la Proposición 5 permite. Lo que produce `_plasticidad.py` sobre el rectángulo con el objetivo de costo es esto:

política sobre el objetivo de costo	utilidad	Δ vs constante, IC95	ahorro
constante, no aprende	0,817		0%
θ con la región completa, incluido el eje elicitado	0,753	−0,063 [−0,139, +0,002]	68%
θ con la región <code>COMPUTED</code> solamente	0,713	−0,104 [−0,181, −0,030]	31%

La política que sólo usa la clave reproducible ahorra 31% y pierde utilidad de forma significativa; la que usa el eje elicitado ahorra 68% con una pérdida que roza el cero, sobre una clave que §6.2.5 mide que no se reproduce entre modelos. La señal suelta de la primera tabla, que es `COMPUTED`, hace mejor que las dos: 58% sin perder utilidad. Eso dice dos cosas: que hay una decisión de costo aprendible sobre una clave computada, y que la política que hoy la consume no la aprende. Lo segundo es una deuda del sistema, no del corpus, y está en §9.1.

6.4.2 El consenso entre paradigmas es un verificador sin oráculo ni juez

Este resultado no se buscó. El banco corre ocho brazos sobre la misma pregunta y siempre los comparó contra el oráculo, nunca entre sí, y en el registro había ocho respuestas por tarea que nadie había mirado juntas.

¿El acuerdo entre paradigmas predice la corrección, sin oráculo y sin juez?

La idea de votar respuestas es Self-Consistency [arXiv:2203.11171] con votantes heterogéneos. Lo que esta sección agrega está en tres lugares. Los votantes son estructuras de control de flujo distintas y no muestras de la misma cadena. El fenómeno se somete a una réplica sobre otra familia de modelo con el criterio escrito antes de correr (§6.4.2). Y el voto mueve credencia y nunca procedencia (§6.4.2), así que no habilita lo que un voto en la literatura habilita por defecto.

La curva

k brazos coinciden	celdas	P(la respuesta es correcta)
0	208	0,424
1	36	0,389
2	24	0,600
3	64	0,812
4	20	1,000
5	42	1,000
6	70	1,000
7	48	1,000

La tabla es sobre `gpt-5.6-1una`, el modelo de la campaña, y cuenta por celda cuántos otros brazos del rectángulo dieron la misma respuesta que ella. 180 de 180 celdas correctas con `k ≥ 4`, sobre igualdad exacta de la cadena normalizada. La curva tiene un umbral en 4 y no una pendiente suave. Dos salvedades que un `1,000` invita a saltar: sobre 180 casos la cota inferior de Wilson al 95% es `0,980`, así que lo afirmable es «≥ 0,98»; y el 4 es el punto donde la perfección aparece en este registro, no un umbral derivado de nada.

Tres controles: no marca dificultad, no depende del largo, y no abarata

¿El acuerdo sólo marca «tarea fácil»? No. En las mismas 27 tareas donde existe consenso:

en las 27 tareas con consenso	celdas	u
brazos dentro del consenso	180	1,000
brazos fuera, en esas mismas tareas	36	0,100

Sobre la misma pregunta, estar adentro o afuera del consenso es la diferencia entera: discrimina dentro de la tarea, no entre tareas. Si sólo marcara dificultad, los dos grupos rendirían igual.

¿Es un artefacto de comparar cadenas cortas? Tampoco. Una cuenta se compara fácil y una enumeración de cuatro ítems no, así que el efecto podría vivir sólo en `singular`. No lo hace:

cardinalidad	con consenso	sin consenso
<code>aggregate</code>	1,000 (n=34)	0,267
<code>boolean</code>	1,000 (n=41)	0,000
<code>enumerative</code>	1,000 (n=28)	0,555
<code>singular</code>	1,000 (n=77)	0,532

¿Sirve para abaratar? No, y se reporta igual. Es la lectura comercial obvia (comité barato, escalar sólo al discrepar) y se probaron las 56 cascadas de dos y tres brazos: ninguna ahorra. El comité se paga en todas las tareas y el brazo caro se paga igual en la mayoría, así que el total sube. El consenso no es un ruteador barato.

Algoritmo 3. El verificador, y qué le está permitido tocar de la base de creencias.

ALGORITMO 3 Verificación por consenso, sin oráculo ni juez

Entrada : pregunta q , plantel P , umbral k , base de creencias B

Salida : B con la credencia actualizada

```
1   $R \leftarrow \{ \text{ejecutar}(p, q) : p \in P \}$  ▷  $|P|$  trayectorias
2   $R \leftarrow \{ \text{normalizar}(r) : r \in R, r \neq \varepsilon \}$  ▷ vacía ≠ abstención
3  for cada respuesta distinta  $a \in R$  do
4       $m(a) \leftarrow |\{ r \in R : r = a \}| - 1$  ▷ cuántos coinciden
5   $a^* \leftarrow \text{argmax}_a m(a)$ 
6  if  $m(a^*) \geq k$  then
7       $B \leftarrow \text{asentar}(B, a^*, \text{procedencia} = \text{ELICITED},$  ▷ NO sube de nivel
8          credencia = calibración[ $m(a^*)$ ])
9  else
10      $B \leftarrow \text{asentar}(B, a^*, \text{procedencia} = \text{ELICITED},$ 
11         credencia = calibración[ $m(a^*)$ ])
12      $\text{marcar\_para\_verificación}(B, a^*)$  ▷ el dial decide si se paga
13 return  $B$ 
```

calibración : $m \mapsto P(\text{correcta} \mid m \text{ coinciden})$, medida en §6.4.2

La línea 7 es la que hay que leer con cuidado, y dice lo contrario de lo que uno espera: la procedencia no cambia. El consenso mueve la credencia y nunca el nivel.

Qué es, y qué NO autoriza

Es un detector de corrección de precisión total y cobertura parcial (27 de 64 tareas), que es exactamente la forma de una regla de abstención: no dice qué brazo usar, dice cuándo no hace falta verificar. Hasta este punto, la única forma de saber si una respuesta estaba bien era tener el gold.

La lectura natural es «si cuatro brazos coinciden, la creencia sube de nivel». No. La escalera de procedencia (ASSUMED < ELICITED < OBSERVED < COMPUTED) clasifica cómo se obtuvo algo, no cuánta confianza merece. Cuatro paradigmas de acuerdo siguen siendo el modelo hablando: votar no toca el documento, así que nada puede ascender a OBSERVED por consenso. Permitirlo sería exactamente la falla que la escalera existe para impedir, que una mayoría de votos del LLM se promueva sola al rango de un hecho computado.

Lo que sí autoriza es mover la credencia dentro de ELICITED, que en el motor de decisión es un campo distinto del de procedencia. Y ahí la tabla de arriba es la curva de calibración: 0,42 · 0,39 · 0,60 · 0,81 · 1,00. El módulo de creencias declara como riesgo abierto que «las credencias elicitedas pueden estar mal calibradas... hasta que existan datos de calibración». Éstos son datos de calibración.

procedencia = de dónde vino · credencia = cuánto se le cree. El consenso mueve la segunda y no puede tocar la primera, y confundirlas convierte un detector útil en un permiso para que el modelo se autoacredite.

El efecto se reproduce sobre una segunda familia de modelo

Los ocho brazos no son independientes: comparten modelo, corpus y recuperador. Hay dos mecanismos compatibles con la curva de §6.4.2, y sólo uno la hace utilizable:

- convergencia de trayectorias: estructuras de control distintas llegan al mismo lugar cuando ese lugar es el correcto, y se dispersan cuando no. Sería una propiedad de los paradigmas, y debería reproducirse con otro modelo debajo;
- el mismo modelo repitiéndose: ocho envoltorios alrededor del mismo LLM producen la misma salida por la misma razón, y el acuerdo no es evidencia de nada.

La predicción se registró antes de correr, como P29, con los tres desenlaces escritos: $P(\text{correcta} \mid k \geq 4) \geq 0,90$ sería convergencia; $\leq 0,65$ (cerca de la tasa base) sería el mismo modelo; entre medio no distinguiría. Se corrió sobre otra familia de modelo, gpt-5.6-terra. El registro vigente de esa corrida tiene 26 tareas y 350 filas; restringido a los ocho brazos del panel y a la réplica 0, 200 celdas factibles, cero errores de infraestructura. El tamaño relativo: 350 filas contra las 2.511 de la campaña, porque ese modelo cuesta diez veces la entrada, así que la réplica cubre un subconjunto de celdas y no la grilla. Y con una sola réplica no se puede estimar el piso de ruido de esta corrida.

La predicción registrada exigía publicar la curva entera, y va entera:

k brazos coinciden, terra	celdas	P(correcta)
0	52	0,397
1	16	0,500
2	12	0,250
3	16	1,000
4	20	1,000
5	6	1,000
6	14	1,000
7	64	1,000

segunda familia de modelo	celdas	P(correcta)	cobertura
$k \geq 4$	104	1,000	15 de 26 tareas
$k \geq 3$	120	1,000	19 de 26 tareas

1,000 contra un criterio de 0,90: convergencia. El consenso se reproduce sobre otra familia, así que la explicación barata (que sea el mismo modelo repitiéndose) queda descartada. En k bajo la curva no es monótona (0,500 en $k = 1$ sobre 16 celdas, 0,250 en $k = 2$ sobre 12), y con esos n no se lee nada de ahí. El control dentro de la tarea también se reproduce: donde hay consenso a $k \geq 3$, los brazos que quedan afuera sacan 0,263 ($n=30$) contra 1,000 de los que están adentro.

El umbral es del modelo; el fenómeno no

El umbral se movió, y hacia abajo: $k \geq 4$ en el primer modelo, $k \geq 3$ en el segundo. La guarda registrada en P29 anticipaba exactamente este caso («si el umbral se mueve, eso ya es información») y lo que dice

es que el número exacto de acuerdos es una propiedad del modelo y la señal no. Sobre el modelo mejor hacen falta menos acuerdos para la misma precisión, que es lo que uno esperaría si la señal fuera convergencia y no coincidencia.

La cobertura también sube: 19 de 26 tareas (73%) en el segundo modelo contra 27 de 64 (42%) en el primero. El detector es más útil sobre el modelo mejor, no menos, lo cual descarta la lectura de que el consenso sea una muleta para modelos flojos.

El umbral se compara sobre planteles del mismo tamaño, y esa condición no es opcional. `k` acuerdos no significan lo mismo sobre planteles distintos: `k=4` sobre ocho paradigmas son 4 de 7 otros (57%), sobre doce son 4 de 11 (36%). Comparar el conteo crudo entre corridas con planteles distintos mide el denominador y no la señal. Las dos lecturas de esta sección están restringidas a los mismos ocho.

Lo que sigue sin estar probado. Dos familias no son la población de los modelos, y las dos comparten corpus y recuperador. Lo que se descartó es la explicación más barata; no se probó que valga para cualquier modelo ni para cualquier corpus.

6.4.3 Predictores del comportamiento, extraídos del registro

Lo que la política aprende se escribe en §3.5.2 como superficies. Esta sección reporta los predictores que el registro entrega sobre cómo se va a comportar un brazo antes de correrlo, que es lo que una regla puede consumir. Ninguno se obtuvo con una llamada nueva.

El paradigma determina cuánta evidencia se lee, y eso vale más que el paradigma

Sobre el corpus fuera de ventana donde `P15` corrió (`gold_transfer` , semilla 47, modelo `gpt-5.4-nano` , cinco brazos estructurados, 90 celdas), la diferencia entre las celdas que leyeron toda la evidencia portadora y las que no:

	celdas	u	brecha	diferencia entre el mejor y el peor brazo	razón
recall completo	29 de 90	0,869			
recall parcial	61 de 90	0,336	+0,533	0,209	2,5×

Leer la evidencia vale más que elegir el paradigma: la brecha entre recall completo y parcial, `+0,533` , es 2,5 veces la distancia entre el mejor y el peor brazo del panel (`0,209` ; comparar contra la mayor desviación de un brazo respecto de la media daría 4,2 veces, y eso no es una diferencia entre brazos). La brecha sobrevive dentro de cada modo que tiene los dos grupos, así que no es la dificultad de la tarea con otro nombre; y es máxima en horizonte desconocido, `+0,721` , el modo donde `P15` más perdió.

La pregunta que decide si esto es accionable es quién determina el recall. Las participaciones de varianza sobre 90 celdas se reportan con ajuste por grados de libertad, porque el R^2 crudo de una agrupación con 21 grupos sobre 90 celdas espera cerca de 22% bajo el nulo:

qué lo determina	grupos	R^2 crudo	R^2 ajustado
la región, lo que la decisión ve	5	5,2%	0,8%
el paradigma, lo que la decisión elige	5	62,2%	60,4%
la tarea, lo que el mundo aporta	21	10,5%	0 (por debajo de su nulo)

El paradigma explica el 60% de la varianza del recall ajustado, y la región y la tarea no explican nada por encima de su nulo. Hay una evidencia mejor que la partición: entrenando sobre 119 celdas de los otros tres corpus y probando sobre estas 90, el error medio de predecir el recall es 0,412 con una constante, 0,392 por región, 0,205 por paradigma y 0,183 por región × paradigma. Rutear es la palanca principal sobre la variable dominante, no un arbitraje al margen de ella, y eso reencuadra qué es rutear: elegir la estructura que va a leer la evidencia, antes que la que razona mejor.

El 0,8% es el problema, y es el mismo problema de §6.5.2 llegando por otro lado: la clave de la política casi no ve la variable que decide.

Son participaciones sobre grupos desbalanceados; no suman nada y región y paradigma no son ortogonales. Y una salvedad de método: el recall es consecuencia del paradigma, no una covariable previa, así que condicionar por él no da un efecto directo insesgado. La magnitud se sostiene sola; la lectura de «tal brazo no resolvía mejor, encontraba mejor» es una pista y no un resultado. Los cinco brazos de este análisis son los del catálogo de esa etapa e incluyen a `map_reduce`, que no corrió la campaña.

Ofrecer una herramienta cambia la conducta, aunque no se use

P30 es una predicción registrada antes de correr, sobre el modelo de la campaña, `gpt-5.6-luna`: ofrecerle a `react` la herramienta de leer todo el material en una llamada, sobre 21 tareas del ancho medio con tres réplicas, 63 celdas pareadas, haría que el brazo la usara y leyera más unidades por llamada. Se refutó en la dirección contraria, y el ahorro apareció igual:

	tokens por celda	llamadas	u
sin la herramienta	137.211	4,3	0,822
con la herramienta ofrecida	87.495	4,0	0,825

1,57× más barato en tokens, 1,36× en dinero por el caché del proveedor, con la utilidad media igual: 7 de las 63 celdas cambian (4 suben, 3 bajan) y la media se mueve +0,003. Las utilidades son las del registro re-puntuado (§5.1.1). Y la herramienta se llamó en 3 de 63 celdas. El ahorro no vino de usarla:

	sin	con	
caracteres releídos	2.836.465	322.094	8,8× menos
fracción releída	12,6%	1,9%	

El efecto está en la oferta, no en el uso. Con una salida barata a la vista, el agente dejó de releer lo que ya había leído. Es la misma forma que dio un pizarrón ofrecido como herramienta (llamado 1 vez en 125) pero con signo positivo: la especificación de herramientas es parte de la política, y su efecto no se mide contando llamadas.

El costo de una vuelta es reenvío, medido por llamada

La misma corrida fue la primera con traza por llamada, 273 llamadas. La ventana crece así:

turno	prompt, tokens	contra el turno 0
0	607	1,0×
1	9.914	16,3×
2	33.548	55,3×
8	67.233	110,8×

El primer turno consume 38.238 tokens de 5.503.757: el 1%. El 99% restante es material que ya se pagó, viajando otra vez. Es la ley N^2 de §5.2.5 medida directo por primera vez, y no inferida de comparar poblaciones. La consecuencia para la política es que la clase de costo por vueltas es la única sobre la que una regla de parada puede actuar, y el registro da la señal: entre réplicas de la misma celda con la misma utilidad, el 33% de los tokens son evitables (49% en `dag_strategy`, 0% en el brazo cuyo abanico lo fija el código), y la racha máxima de búsquedas estériles es 1,17 en la réplica barata contra 2,28 en la cara, sobre 72 pares empatados. Es contable, determinista, y es el tipo de señal que este trabajo prefiere sobre el andamiaje por prompt. La regla que la consume está registrada como predicción (P_{20}) y no corrió; se reporta como predictor medido y regla pendiente, no como resultado.

Qué predice cada uno, en una línea

predictor	procedencia	qué anticipa antes de correr
capacidades declaradas (§6.3)	COMPUTED del código	si un brazo puede resolver lo que la pregunta exige
ley de costo del brazo (§5.2.3)	medida, estable entre corpus	si su costo va a escalar con el material, con las vueltas, o con nada
paradigma → recall (§6.4.3)	medida, sobre nano	cuánta evidencia va a leer; el paradigma explica el 60% ajustado de esa varianza
oferta de herramientas (§6.4.3)	medida	que el releído cae con una salida barata a la vista
racha estéril (§6.4.3)	contable en runtime	que la trayectoria se está desbocando, antes de que termine

Ninguno es un nombre de paradigma. Todos son cosas que una regla puede leer al decidir, y eso es lo que la interfaz aprendible de §6.3 pide de sus entradas.

6.5 El ciclo que reparó el vocabulario

Las secciones anteriores miden a los brazos. Ésta mide el ciclo que reparó el vocabulario sobre el que la política decide, y el registro que la sostiene es la cronología de predicciones falsables del laboratorio, con cada predicción anotada y fechada antes de correr y su script de veredicto congelado en el mismo commit. Tres refutaciones sobre tres mundos que la política nunca había visto, y cada una produjo un medidor nuevo. Lo que sigue son los episodios, en orden.

En qué capa ocurre esto. Lo que cada episodio agregó fue un medidor, un eje que el sistema no sabía medir, y los medidores están en la capa de código (§3.5, tabla de las dos capas), así que agregarlos es trabajo de diseño y lo hicieron personas: cada refutación la leyeron el autor y su asistente, diagnosticaron el eje que faltaba y lo agregaron. Las creencias que esos medidores emiten desde entonces, y las tablas

que se llenan con ellas, se acumulan solas. El sistema consume los medidores `COMPUTED` que ese trabajo produjo, y la consolidación reajusta las tablas sobre ellos con su guarda; eso es lo que §6.6 mide. Que la etapa de abstracción proponga ejes nuevos sola está en el diseño y no corrió, y hasta que corra la frontera queda donde está. Esta sección establece el método con su condición formal, y §8.3 anota qué se leería mal si se lo tomara por plasticidad de la clave.

6.5.0 La afirmación de transferencia, y su refutación medida

La consolidación de §3.5 se apoya en un condicional: *si el corpus es representativo del dominio, la capa ajustada debería transferir*. Eso es falsable, y este registro ya lo falsó una vez.

P15. Sobre un mundo que la política nunca había visto (seed 47, 390 celdas, cero errores de infraestructura, modelo `gpt-5.4-nano`) el ruteo por request perdió contra el mejor paradigma fijo por $-0,087$, más allá del piso de ruido, mientras reproducía cada decisión 26/26 desde su base de creencias registrada. §6.5.2 lo desarrolla como primer episodio del ciclo.

El mecanismo es el hallazgo, no el número. El vocabulario de región no tiene eje de horizonte, así que las tareas que castigan una elección fija eran indistinguibles de las que la premian. La política ruteó contra su propio veredicto registrado porque ninguna etiqueta le dijo nunca en qué caso estaba. Chequeo de sensibilidad: reparar la validez del aprendizaje (agregación por episodio, holdout limpio) deja el número idéntico, así que la refutación no es un artefacto del procedimiento.

Eso afila la condición hasta volverla chequeable:

La representatividad hay que enunciarla sobre los ejes que el vocabulario de región distingue. «Representativo del dominio» no alcanza. Un corpus que varía en una dimensión que el mapa de features no mira produce episodios que la política no puede separar, y entonces aprende un promedio sobre dos poblaciones. La condición se puede chequear sobre un corpus *antes* de correrlo, porque la región es una función determinista de los features.

6.5.1 Régimen de los episodios

Los tres episodios son anteriores a la campaña de §6 y corrieron sobre otro modelo y otros corpus. Se declara todo lo que difiere:

episodio	fecha de registro	mundo	modelo	filas	tokens	veredicto congelado antes
P15	2026-08-27	gold_transfer , semilla 47	gpt-5.4-nano , t = 0 , semilla fija	390	14,07M	sí
P16	2026-08-27	gold_p16 , semilla 61	gpt-5.4-nano	390	13,95M	sí, en el mismo commit
P17	2026-08-27	gold_p17 , semilla 73, detectores declarados por tarea	gpt-5.4-nano	390	12,69M	sí
eje literal	2026-08-30	rectángulo de la campaña, leave-one-task-out	gpt-5.6-luna	64 × 8	ninguno nuevo	n/c

Qué transfiere a la campaña y qué no. Transfiere el mecanismo: que a la clave le faltaba un eje es una propiedad del vocabulario y del corpus, no del modelo, y el eje de continuidad separa el horizonte 6 de 6 en tres corpus con cualquiera de los dos modelos porque es función pura del material. Transfiere la reproducibilidad 26 de 26, que es una propiedad de que la decisión sea función de la base registrada. No transfieren las magnitudes: $-0,087$, $+0,121$ y el cruce de λ son números de nano sobre esos tres mundos, y sobre luna la campaña dio su propia versión del mismo veredicto en §6.2 y §6.2.6. Los dos modelos no se mezclan en ninguna estadística; el cargador del registro levanta si un archivo los mezcla.

6.5.2 Primer episodio: el vocabulario no tenía eje de horizonte

P15 , mundo con semilla 47, 390 celdas, cero errores de infraestructura. La política ruteó por región de features hacia un nombre de paradigma y perdió $-0,087$ contra el mejor fijo, más allá del piso de ruido de $0,057$, mientras reproducía cada decisión 26 de 26 desde su base de creencias registrada. Ese piso se estimó con `metrics.noise_floor` sobre las 112 tareas de los corpus de entrenamiento, no sobre las 26 del mundo held-out.

El mecanismo, verificado contra el registro y no conjeturado: las tareas de horizonte desconocido caían en las mismas regiones que las de cobertura independiente, porque el vocabulario tenía cardinalidad, oráculo y acoplamiento y no tenía ningún eje para «no se sabe cuántos saltos hay». La política mandó a esas tareas el brazo que gana en cobertura, contra su propio veredicto registrado de que ese brazo falla en horizonte, porque ninguna etiqueta le dijo nunca en qué caso estaba. Donde el registro sí tenía la señal, la selección funcionó: $+0,121$ sobre cobertura.

Dos incertidumbres. El criterio preregistrado era el piso de ruido por celda, y contra él $-0,087$ está fuera de $\pm 0,057$, así que P15 falla su propio test. Pero el intervalo de muestreo sobre 26 tareas, bootstrap pareado con 10.000 remuestreos, incluye al cero: $[-0,228, +0,037]$, $p = 0,19$. Son dos cosas distintas, ruido de medición y ruido de muestreo, y se reportan las dos: P15 está refutada contra su criterio registrado y no está establecida en ninguna dirección por un intervalo de 26 tareas. Puntuando la acción real en vez del primer escalón, el neto contra el mejor fijo queda en $-0,011$, $[-0,121, +0,091]$, y contra siempre-react en $+0,112$, $[+0,001, +0,238]$, $p = 0,045$; bajo Benjamini-Hochberg con $q = 0,05$ sobre los cuatro contrastes, ninguno sobrevive, ése incluido.

Lo que produjo el ciclo. Un eje nuevo, la continuidad: recurrencia de una clave literal entre unidades distintas, función pura del material, `COMPUTED`. Separa el horizonte desconocido 6 de 6 en tres corpus, con cero falsos positivos sobre las demás celdas. Y una corrección al método, medida: agregar el eje sin más fragmentó el espacio de regiones por debajo del piso de confianza (de 12 tareas con margen sobre 26 a 0), así que la política dejó de especializarse en absoluto. Con acumulación jerárquica y retroceso a la región padre, 16 de 26. Un vocabulario más expresivo cuesta potencia estadística, y a este tamaño de registro el costo se ve.

6.5.3 Segundo episodio: la selección por identidad compra calidad sólo a costo cero

P16, semilla 61, con el eje nuevo, la acción real puntuada y el costo de escalar cobrado. El veredicto lo decidió un barrido sobre λ , que estaba preregistrado como la medición:

λ	neto contra el mejor fijo	neto contra siempre- react
0,00	+0,121	+0,177
0,02	-0,404	+0,000
0,05	-1,289	-0,265

Con el costo a cero, rutear por identidad captura `+0,121`. A λ igual a 0,02 el neto es negativo: la selección por nombre compra calidad sólo cuando los tokens son gratis. Reproducibilidad, de nuevo 26 de 26. Y una salvedad: el piso de ruido con que se compararon esos netos se computó a $\lambda = 0$, y el costo es entre dos y cinco veces más disperso entre réplicas que la utilidad, así que «dentro del ruido a $\lambda = 0,02$ » no está medido contra el ruido de $\lambda = 0,02$. Lo que sí está medido es el signo.

Lo que produjo el ciclo. Que la valuación tiene que puntuar la acción real y cobrar la escalera, y que en un corpus de coincidencia exacta la cascada precede a la selección en 20 de las 22 tareas de la cohorte de ruteo de ese mundo (§B.5): la propiedad que hace gradeable a una tarea es la que hace correcta a la escalación.

El tercer mundo, P17, semilla 73, se generó con detectores declarados por tarea y la cascada bajó de 22 a 2 de 26 (sobre `gold_p17` bajo el régimen anterior habría disparado en 22); la sonda disparó en 14 y no resolvió ninguna, así que la decisión fue diferir. Y su número: neto contra el mejor fijo `-0,146` a $\lambda = 0$ y `-1,04` a $\lambda = 0,05$; contra siempre- react, `0,000` a $\lambda = 0$, porque 20 de las 22 tareas de la cohorte terminaron en el fallback y ahí la política es `react`. Reproducibilidad 26 de 26. La restricción que quedó es una sola y está nombrada: la evidencia que la sonda devuelve no alcanza el piso de procedencia que tiene que alcanzar.

6.5.4 Tercer episodio: el literal, medido contra el eje anterior

El vocabulario vigente agrega un eje: en cuántas unidades aparece el literal que la pregunta cita. Es hermano de la continuidad (forma de token cerrada, verificada por contención, sin modelo en el medio) y no es un disparador léxico: la misma pregunta cambia de valor si cambia el material. No se eligió; se midió. Sobre el rectángulo de la campaña, `leave-one-task-out` con el objetivo de costo (§6.4.1, primera tabla), la señal `cardinalidad × término literal` ahorra 58% a utilidad igual al mejor fijo, contra 76% a `-0,079` de la región vigente y 13% a `-0,101` de `n_units`.

Y midiendo se encontró un defecto del medidor que sólo se ve midiendo: una pregunta booleana cita sus opciones de respuesta, no un término de búsqueda, y esos literales no están en el material por

construcción. La guarda usa la cardinalidad de la respuesta, que el caller declara. Recomputado con el medidor corregido, el vocabulario sobrevive.

6.5.5 Lo que el ciclo establece

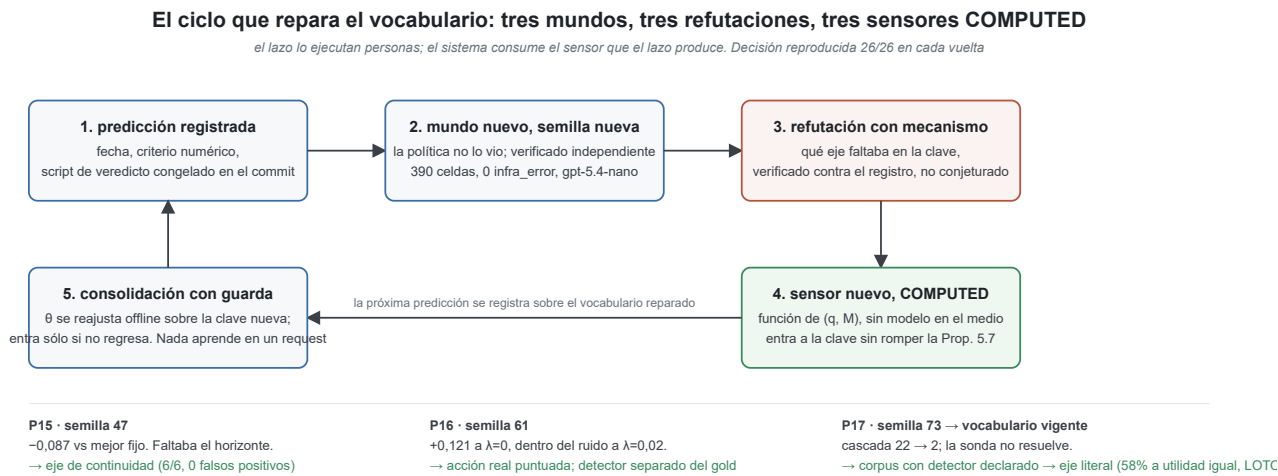


Figura 17. El lazo, con los tres episodios debajo. Predicción registrada con veredicto congelado, mundo nuevo, refutación con mecanismo, medidor `COMPUTED`, consolidación con guarda, y la próxima predicción sobre el vocabulario reparado. Los pasos 3 y 4 los ejecutaron personas; los pasos 1, 2 y 5 son código. Ésa es la frontera que §8.3 declara.

episodio	qué faltaba	qué se agregó	procedencia	decisión reproducida
P15	eje de horizonte	continuidad entre unidades	COMPUTED	26/26
P16	valuación de la acción real	costo de la escalera cobrado; detector separado del gold	—	26/26
P17	vía para que la selección dispare	corpus con detector declarado por tarea	COMPUTED	26/26
vocabulario vigente	separación entre brazos capaces	eje literal	COMPUTED	—
§6.3.5	segmentación que la región no ve	ontología de la pregunta	dos ejes COMPUTED, el resto ELICITED	—

Tres cosas se leen de la tabla. Primero, qué produjo cada episodio: uno un medidor `COMPUTED` nuevo, la continuidad; el segundo una corrección de la valuación; el tercero un corpus con detector declarado por tarea. El eje literal salió de una medición comparativa y no de una refutación, y la ontología es mayormente `ELICITED`.

Segundo, ninguno rompió la garantía: lo que entró a la clave o al método fue `COMPUTED` o fue código, nunca una salida del modelo, y cambiar qué se sensa no cambió que la decisión sea función de la base registrada.

Tercero, lo que el ciclo produjo es el vocabulario y el método sobre los que una política puede aprender, no una política sobre paradigmas. Eso está un nivel arriba de lo que §3.5 describe, y la frontera es ésta: la

consolidación aprende la política sobre el vocabulario que tiene, con guarda y sin tocar un peso; el vocabulario lo repararon personas, con la disciplina de abajo, y esa reparación es lo que este registro tiene para mostrar como método. La plasticidad autónoma del vocabulario es la superficie de gobierno de §4.4, diseñada y sin medir.

La disciplina que lo hace posible. Aprender del registro y confabular sobre el registro son la misma operación con distinto control. Lo que separa una de otra es lo que §6.2 construyó: el piso de ruido por sesgo del máximo, la corrección por selección con el máximo de los nulos, la partición proponer/puntuar/promover por tarea, y las predicciones escritas antes del número. Sin eso, tres refutaciones serían tres ajustes post hoc. Con eso, son tres episodios.

6.6 La trayectoria de maduración de θ

Con la definición de §1.1, aprender se mide en la trayectoria de la propia política mientras absorbe episodios, no en una curva de utilidad. Sobre el registro de la campaña, los 616 episodios (una celda por par tarea y brazo, media sobre réplicas) se le dieron a θ en seis lotes por orden de tarea, que es un proxy de llegada porque el registro llegó de una vez; en cada ciclo se construyó el candidato con la consolidación de §3.5, se lo evaluó con la guarda de promoción sobre el lote que todavía no había visto, y se lo construyó dos veces para ver si las tablas coinciden (`bench/analysis/_maduracion.py` , cero llamadas al modelo):

ciclo	episodios	regiones con evidencia	pares con $n \geq 8$	tareas que θ gobierna	guarda sobre el lote siguiente	reproducible
1	89	4	0	0 de 78	pasa, sin cambio de decisión	sí
2	191	6	0	0 de 78	pasa, sin cambio	sí
3	299	8	8	24 de 78	pasa, sin cambio	sí
4	409	12	8	24 de 78	pasa, sin cambio	sí
5	517	12	11	36 de 78	no pasa: $+0,018$ con IC95 $[-0,042, +0,067]$	sí
6	616	14	11	36 de 78	sin lote posterior	sí

Cuatro cosas se leen. θ madura: de cuatro regiones sin ningún par confiado a catorce regiones con once pares que cruzaron el piso de evidencia, y de gobernar cero tareas a gobernar 36 de 78 con su regla de confianza y margen, todo sin que nadie tocara una regla. Cada ciclo es reproducible: dos construcciones sobre los mismos episodios dan las mismas tablas, y el bundle verifica su firma.

La guarda hizo su trabajo una vez, en el ciclo 5, donde el candidato ganaba $+0,018$ sobre el lote siguiente con un intervalo que cruza cero, y no lo dejó entrar: el θ vigente al final del registro es el del ciclo 4, con 24 tareas gobernadas, y las 36 del candidato esperan más evidencia. Y lo que θ no hizo también está en la tabla: en ningún ciclo ganó utilidad separable del ruido sobre lo que no había visto, que es lo que §6.2 predice para un catálogo cuyos contendientes empatan. Maduró en evidencia, cobertura y seguridad; no maduró en premio, porque en este corpus no hay premio de calidad que madurar. El eje donde sí lo hay, el costo, es el que la política todavía no aprende (§6.4.1), y está primero en §9.1.

El artefacto, tal como lo lee un auditor. Lo que sigue es un extracto literal de la política del ciclo 6, versión 6, firma 4a982dcee85d7093 , para las dos regiones con más tareas del corpus:

region many/no_oracle/loose/flat/no_lit		(24 tareas)	
dag_strategy	u= 0.833 w=0.652 n=24	cost=168469.0	
react	u= 0.775 w=0.640 n=24	cost=163152.2	
rewoo	u= 0.747 w=0.450 n=24	cost= 10923.2	
reflection	u= 0.742 w=0.575 n=24	cost=140034.1	
handoff	u= 0.694 w=0.326 n=24	cost=167357.3	
supervisor	u= 0.489 w=0.129 n=24	cost= 57869.5	
pointer_chase	u= 0.484 w=0.272 n=24	cost= 14043.2	
gist_reader	u= 0.472 w=0.272 n=24	cost= 29765.7	
graph_traverse	u= 0.458 w=0.268 n=15	cost= 27068.2	
region few/no_oracle/loose/flat/no_lit		(12 tareas)	
extract_compute	u= 1.000 w=0.549 n=2	cost= 28882.0	(below episode floor -> abstain)
streaming_scan	u= 1.000 w=0.549 n=2	cost= 29315.0	(below episode floor -> abstain)
handoff	u= 0.972 w=0.730 n=12	cost= 33337.6	
react	u= 0.944 w=0.571 n=6	cost= 64796.2	(below episode floor -> abstain)
...			
rewoo	u= 0.638 w=0.390 n=12	cost= 7265.4	

Se lee sin explicación: en la región grande el mejor par confiado supera al fallback por 0,058 , apenas sobre τ , y ahí θ gobierna; en la región chica dos brazos tienen utilidad perfecta y dos episodios, y θ se abstiene de ellos hasta que crucen el piso. Cada decisión que sale de esta tabla lleva su propio artefacto, `Plan.explain()` : la acción, el brazo, el modelo, la escalera si hubo cascada, si hubo compuerta, si pidió sonda, qué patrones excluyó el nivel de garantía y por qué, el veredicto del dial, la versión y la firma de θ , y un digest del conjunto; con eso y la base de creencias registrada la decisión se re-deriva sin correr el modelo, que es lo que 26 de 26 significa en §6.5.

7. Discusión

7.1 Qué cambia para cómo se evalúan agentes

Tres correcciones de este registro valen fuera de él y no cuestan un token.

Reportar `pass^k` al lado de `pass@1` . Un banco que da sólo `pass@1` no distingue un sistema que acierta de uno con el que se puede contar, y acá la diferencia llega a 0,196 .

No leer la varianza de interacción tarea×paradigma como evidencia de que rutear conviene. Esa interacción vive entre los brazos que nadie elegiría. El premio de un oráculo es un máximo sobre estimaciones ruidosas, y hay que descontarlo con un piso que tenga la misma varianza que lo que descuenta.

No leer el plantel como un torneo. Un banco que ordena sus doce brazos y se queda con el primero tira lo único que el registro tenía para enseñar, que es qué exige cada pregunta y qué puede hacer cada estructura.

7.2 Contra una ventana frontera: qué compra el arnés, y qué no

La objeción obvia a todo lo anterior es que un modelo grande con una ventana enorme lee todo el material de una vez y se ahorra la orquestación. Sobre calidad, la objeción es correcta y los datos de este registro la respaldan: el embudo de §6.1.1 muestra que toda representación intermedia más chica que el material es una pérdida, y una ventana grande es el caso extremo de *no tener canal con pérdida*. Medido: `direct` (leer todo en una sola llamada) saca 0,917, el mejor del plantel donde corre.

El arnés no le compite ahí. Le compite en otros cuatro ejes.

7.2.1 El precio

Una tarea de la banda ancha lleva 455.476 tokens de material. Con los aranceles vigentes, y contando que por encima de 272.000 tokens de entrada se cobra tarifa larga por el request entero:

leer todo de una	USD por pregunta
ventana de 922k, gama media	0,18
ventana de 922k, gama alta	1,82
ventana de 922k, tope de gama	4,55
<code>react</code> sobre el modelo de la campaña (medido)	0,0216

211×. Y el escalón es un acantilado, no una pendiente: cruzar el umbral por un token duplica la tarifa del request completo.

7.2.2 Lo que no entra, y lo que no es un archivo

El corpus de este trabajo cabe en una ventana de 922k. Un corpus de producción no. Y ése es el caso fácil: el difícil es que el material no siempre es un texto que se pueda pegar. Un arnés extiende el alcance a lugares donde una ventana no llega por definición:

- búsqueda externa: un índice que cambia entre una pregunta y la siguiente, o que vive detrás de permisos por usuario. No hay ventana que contenga un índice vivo;
- corpus sin cota: cuando el material crece con el negocio, «entra en la ventana» es una propiedad que vence. La ley de costo de §5.2.5 dice qué pasa cuando no entra;
- herramientas con efecto: una escritura, una transacción, un ticket. Un modelo que lee no ejecuta, y lo que gobierna una acción irreversible es una precondition verificada, no un contexto grande.

Nada de esto está medido en este trabajo. El corpus de este paper es estático, cabe, y no tiene herramientas con efecto. Son razones por las que un arnés existe, no resultados de este trabajo.

7.2.3 Los tres ejes donde una ventana no ayuda

Éstos sí están medidos, y son la parte que no depende del tamaño del modelo:

Abstención. El paradigma que gana el modo de cadenas acopladas sobre `terra` lo hace con 8 correctas y 1 abstención sobre 9 celdas, sin una respuesta equivocada; los que fallan en ese modo se abstienen más de lo que inventan (§6.1.3). No gana por razonar mejor: gana porque llega, y cuando no llegó, lo dice. Saber *cuándo no contestar* es una decisión de flujo de control, y §6.1.4 muestra que sacarla del LLM sube la utilidad y el determinismo a la vez.

Procedencia. Un contexto grande no dice de qué unidad salió un número. El Teorema 2 (soundness del ensamblador) garantiza que si se emite, todo valor emitido está implicado por la base de creencias al piso pedido; eso exige una base de creencias tipada, no una ventana.

Determinismo. `pass^3` mide al menos 12-28% de celdas inestables con temperatura cero y semilla fija, y esa varianza no es del tamaño del modelo: es la trayectoria componiéndola. Un modelo más grande con las mismas decisiones delegadas al LLM tiene el mismo problema.

El arnés no existe para ganarle a un modelo grande en calidad. Existe para que la respuesta cueste dos órdenes de magnitud menos, para alcanzar material que ninguna ventana contiene, y para poder decir de dónde salió y cuándo no la hay, que es lo único que un dial de garantía puede prometer.

La comparación directa no está corrida. No se midió un modelo de ventana frontera sobre las celdas anchas: el precio de §7.2.1 sale de los aranceles y del tamaño del material, y la calidad de `direct` sale de las celdas donde la factibilidad lo dejó correr. Es una inferencia bien apoyada, no una medición, y sería una corrida acotada (18 tareas de banda ancha, unos USD 82) que decidiría el punto de una vez.

7.3 Impacto más amplio

§1.1.1 invoca el Reglamento de IA y el artículo 22 del RGPD para motivar que una norma pide una propiedad y no una probabilidad. Un sistema así deja cuatro cosas del otro lado.

Abstenerse ante un auditor. Un sistema que puede callarse traslada la decisión a una persona en el momento en que la evidencia no alcanza, y eso es lo que la supervisión humana pide. La abstención tiene su propio modo de falla. Un sistema que se abstiene en las preguntas difíciles y contesta las fáciles produce un registro que se ve impecable y no dice nada sobre lo que importaba. La curva riesgo-cobertura tiene que reportarse junto con la utilidad, y este paper la reporta; un despliegue que exhiba sólo la precisión de lo contestado está ocultando el denominador.

Responsabilidad sobre lo aprendido. Un piso de garantía que sube solo, aunque suba con guarda de replicación y quede en un artefacto firmado y diffeable, es una decisión de gobierno que nadie tomó explícitamente. La Proposición 2 acota cuántas veces puede pasar y §3.4 exige que nunca baje solo, pero quién responde por un piso aprendido es una pregunta organizacional que el diseño hace posible contestar y no contesta. El artefacto está hecho para que un auditor lea dos versiones de una tabla; falta que alguien tenga la obligación de leerlas.

Clasificar mal una pregunta regulada. Si la ontología de §6.3.5 se recupera desde el request con un clasificador elicitado, un error de clasificación puede mandar una pregunta de acción irreversible por el camino de una pregunta de consulta. Por eso las banderas de riesgo las declara el caller y nunca se infieren del texto (§3.4.1), y por eso la Proposición 5 exige que lo que gobierna la decisión sea `COMPUTED`. La ontología elicitada puede informar el costo, nunca el piso.

El valor superado con procedencia impecable. El punto de vigencia de §9.1 existe por una falla que este sistema hace más peligrosa y no menos: un valor viejo, encontrado en un documento real, con procedencia `OBSERVED` correcta, emitido como vigente. El Teorema 2 lo deja pasar, porque es sound y falso. Un sistema con procedencia le da a una respuesta equivocada una credencial que una respuesta sin procedencia no tiene, y eso obliga a que la vigencia sea un eje de la clave y no un detalle del material.

8. Limitaciones y amenazas a la validez

Las amenazas se agrupan por el tipo de inferencia que ponen en riesgo. Validez interna: si el efecto medido es del mecanismo o del arreglo experimental. Validez externa: hasta dónde llega lo medido fuera de este corpus. Validez de constructo: si lo que se mide es lo que se dice medir. Ninguna toca la Proposición 4 ni los dos teoremas (esos son verificados por máquina y no dependen del corpus); todas tocan las afirmaciones empíricas de §6.

8.1 Validez interna

La campaña mide un solo régimen, el de material fuera de ventana. Las tareas de ancho medio y ancho grande llevan entre 150k y 455k tokens, por encima del umbral de contexto largo del modelo; en las de ancho chico leer todo cabe y `direct` es correcto y barato ahí.

Falta una corrida completa a 1,27M tokens de material. Para esa escala el paper tiene sólo la aritmética de factibilidad de §3.3, y que la compuerta se comporte igual ahí es una predicción de la desigualdad, no una medición.

El ruido es por celda, y por eso todo delta de calidad se reporta con su conteo de celdas que cambian, nunca como media pelada. Sobre el rectángulo, entre el 12% y el 28% de las celdas tienen réplicas distintas (§6.1.2), y el costo es menos reproducible que la utilidad: la dispersión de tokens entre réplicas de la misma celda llega a $2\times$ y $5\times$ a calidad idéntica, lo que hace que cualquier comparación a $\lambda > 0$ cargue un ruido que el piso a $\lambda = 0$ no ve (§6.5.3).

La amenaza del sampleo está en combinar `tools` con razonamiento. Sobre las tres familias disponibles la temperatura no llega a ser el problema. Los deployments de razonamiento la rechazan de plano, y `terra` en su propio default es el más reproducible de los tres.

La restricción viva es estructural. En la familia `5.6`, `tools` y un `reasoning_effort` distinto de `none` no se pueden combinar en Chat Completions, y la diferencia que eso hace es total, 0,000 contra 1,000 sobre la misma tarea. Por eso toda fila de campaña corre con `reasoning_effort = none`, que es un régimen declarado y no un default.

El paper corre sobre dos modelos y ninguna estadística los mezcla. La campaña de §6 y el held-out de §6.2.6 corren sobre `gpt-5.6-luna`; los tres episodios de §6.5 y el análisis de recall de §6.4.3 corren sobre `gpt-5.4-nano`, sobre corpus anteriores y con vocabularios de región anteriores. El cargador del registro levanta si un archivo los mezcla, y §6.5.1 lo tabula.

Lo que cruza entre los dos es la inferencia cualitativa: que a la clave le faltaba un eje, que los medidores agregados son `COMPUTED`, que la decisión se reproduce. Las magnitudes de §6.5 y §6.4.3 no cruzan, y valen

para `nano` y para esos mundos. Ver el ciclo entero sobre un solo modelo exige repetir los tres episodios sobre `luna`, y eso no está hecho.

8.2 Validez externa

Una rama de la propia partición no se pudo ejercitar, y el motivo es estructural. §B.5 parte el problema según `v`, la disponibilidad de un detector barato en runtime, y manda `v = 1` a una cascada y `v = 0` a un ruteador. Todos los corpus de este registro caen del lado `v = 1` por construcción. Corregir sin juez significa corregir por coincidencia exacta, la coincidencia exacta necesita una respuesta de referencia, y el mismo campo se leía como detector de runtime.

La regla de cascada dispara con prioridad más alta que la de selección. Sobre dos corpus held-out la regla de selección no disparó nunca, así que los dos veredictos de ruteo son mediciones de la cascada. La forma general es una advertencia sobre toda una clase de experimento:

Un benchmark que establece corrección por coincidencia exacta contra una referencia tiene un detector barato en cada tarea por construcción, y por lo tanto no puede ejercitar la rama `v = 0` de su propia partición. Ser corregible implica ser verificable.

Si la selección paga donde verificar es genuinamente imposible queda abierto, y necesita un corpus cuya disponibilidad de detector se declare por tarea en vez de heredarse de la clave.

La transferencia de veredictos por modo entre mundos está medida, y es parcial. Sobre `gpt-5-chat`, veredictos por modo derivados de mundos de una sola semilla se pusieron a prueba en un mundo con semilla nueva, bajo `P8`, una de las predicciones falsables fechadas del laboratorio (cuatro tareas, cuatro brazos, dos réplicas, un diseño de screening). Dos de sus cinco predicciones no transfirieron: que `react` fuera el fallback general con `u ≥ 0,9` en toda celda factible falló en las cuatro celdas, y que `gist_reader` alcanzara `0,75` en tres modos también. Las de `rewoo` y `dag_strategy` sí transfirieron.

La refutación no se apoya en celdas que nadie resolvió: en las dos donde otro brazo llegó a puntaje perfecto, `react` devolvió 0,667 y 0,000. Por la regla de decisión registrada, todo veredicto por modo anterior a la campaña quedó rotulado como local a su mundo. Los veredictos por modo que este paper reporta salen de la campaña de §6, sobre otro modelo y un corpus con entidades reales, y el held-out de §6.2.6 es su test de transferencia.

La varianza de trayectoria tiene un confundente que este montaje no puede separar. Las celdas inestables se miden con `t = 0`, semilla fija y el mismo prompt, así que la varianza observada tiene dos fuentes posibles: la ramificación delegada al modelo, que es la que §6.1 estudia, y el no-determinismo del stack de servicio, que produce logits distintos para entradas idénticas sin que el cliente lo pueda ver (agrupamiento dinámico de requests, orden de reducción en punto flotante, versión del kernel).

La Proposición 4 no depende de cuál domine, porque con `d(T) = 0` la trayectoria es fija cualquiera sea la fuente del jitter. Lo que sí depende es atribuir a la ramificación el 12 a 28% de celdas inestables, y separarlo exige un servidor de un solo request por lote, que este montaje no tuvo.

Corpus sintético. El ground truth es exacto y se re-deriva de forma independiente, y los parámetros estructurales son diales y no esperanzas, pero la distribución de tareas reales sobre esos diales es desconocida. Hacen falta benchmarks públicos para validez externa y no se usan todavía.

La ontología se midió en parte sobre la etiqueta de diseño. En §6.3.5, tres de los siete ejes que componen el eje principal se derivan del modo del que salió cada tarea, que se conoce al construir el corpus. Eso establece que la estructura existe y que la política debería segmentar por ella, y la variante hecha sólo de cardinalidad y cobertura declaradas, `COMPUTED`, muestra que parte de esa estructura ya es visible desde el request. No establece que los demás ejes se recuperen desde un request real con la precisión que una regla necesita. Esos ejes tienen a `ELICITED` como techo, y una clave eliciteda es el caso (2) de la Proposición 5. Recuperar la ontología desde el request es la medición que decide si la interfaz de §6.3 sirve afuera de este banco, y está en §9.1 como segunda prioridad.

El leave-one-arm-out tiene ocho puntos. Las capacidades ganan 6 de 8 pliegues y no cruzan su nulo de capacidades barajadas (`p` exacto `0,066`). Con esa `n` la prueba no puede decidir, y lo que la decide son más brazos, no más tareas. El brazo de ausencia que el catálogo predice es el primero, y está en §9.1 como primera prioridad.

La latencia no es comparable. El DAG de este banco corre secuencial donde correría en oleadas concurrentes; y una vez que los workers compiten, el wall-clock por fila deja de medir latencia. La calidad y los conteos de tokens siguen siendo exactos.

8.3 Validez de constructo

La superficie de acciones es arquitectura declarada y no está ejercitada. §3.4.1 filtra las acciones irreversibles con un piso de procedencia y §4.4 afirma la maquinaria sobre cuatro superficies. Una de esas cuatro nunca corrió. El inventario completo de herramientas, sobre todos los corpus y los quince paradigmas registrados, son doce:

qué hacen	cuáles
leen el mundo	<code>search</code> <code>keyword_search</code> <code>semantic_search</code> <code>read</code> <code>read_all</code>
escriben el estado del propio agente	<code>note</code> <code>notes</code> <code>plan</code> <code>advance</code> <code>post</code> <code>board</code>
leen su propia contabilidad	<code>coverage</code>

Ninguna de las doce cambia nada fuera del proceso. No se escribe un archivo, no se manda un mensaje, no se actualiza una fila.

Las banderas de riesgo están puestas y sí levantan el dial de garantía. Seis tareas de setenta y ocho llevan `irreversible = True` y tres llevan `shared_writes = True` (estas tres corrieron sobre nueve paradigmas y no doce). La tarea que etiquetan es «*decidí si esta relación debe escalar para congelamiento; contestá 'escalate' o 'no escalation'*», calificada por exact-match contra una clave, o sea una clasificación sobre documentos con la etiqueta de una acción encima. No existe una herramienta que congele una cuenta, así que el piso que filtra lo irreversible nunca tuvo un acto irreversible que filtrar.

Resolverlo es otro experimento. El balance queda así. §4 se afirma para agentes en general y está verificado como tal; §6–§8 se afirman para extracción de respuesta exacta sobre documentos y se midieron ahí; la superficie de acciones está diseñada, tipada, probada en unidad y sin medir. Lo más parecido a evidencia sobre una capacidad que no es recuperación es el pizarrón ofrecido como herramienta en §6.4.3, llamado una vez en 125, y es un nulo.

La plasticidad medida es la de lo que se acumula, creencias y tablas, no la de los sentidos. §6.6 muestra que θ madura sobre el registro sin intervención: regiones con evidencia, pares confiados, tareas gobernadas, reproducibilidad y guarda. Los medidores nuevos, en cambio, los agregaron personas en los tres episodios de §6.5, porque un medidor es código.

Si un lector entiende «plasticidad» como que el sistema descubrió qué sensar, entiende algo que el registro no sostiene. Lo que el registro sostiene es que la frontera entre lo que aprende el sistema y lo que diseña una persona está declarada y respetada, y que el paso que la movería, la etapa de abstracción proponiendo ejes nuevos, está en el diseño y no corrió (§9.1).

Las afirmaciones de novedad están verificadas como conjunciones, no como partes. Los dos trabajos ancla se leyeron completos el 2026-08-26: todos los números citados de Select-then-Solve verifican contra su cuerpo, y la cesión a SCL se sostiene. Las afirmaciones restantes, la consolidación de la política de control (§3.5), el descubrimiento de proposiciones con compuerta de procedencia, la abstención en el ruteo de paradigmas, sobreviven búsquedas fechadas sólo como conjunciones cuyos conjuntos individuales tienen cada uno un vecino publicado (§2.2, §2.5, §D.4). Un campo que se mueve así de rápido puede cerrar cualquiera de ellas en meses; las búsquedas están fechadas para que un lector pueda re-correrlas.

8.4 Reproducibilidad

Una semilla sola no fija un corpus. Cuando el algoritmo de generación cambió, la misma semilla produjo un mundo distinto. Los manifiestos estampan versión de generador, y los resultados de versiones distintas no deben mezclarse.

9. Conclusión

Una sola máquina (LLM estocástico, creencia tipada, regla determinista, registro) que aprende con los pesos del LLM congelados, y cuatro cosas que se pueden decir de ella con medida.

	qué dice	evidencia
La condición estructural	dónde puede aparecer la varianza, y qué cuesta gobernarla	sacarle al LLM la elección del ancla lleva un paradigma de 0,33 a 0,89 sobre terra , 8 de 9 réplicas coinciden, con la salvedad de que se desarrolló sobre esas celdas; el dial es gratis hasta A2 y en A3 se lleva el 60% del catálogo y el 31% de la utilidad, sobre nano
La interfaz aprendible	sobre qué se aprende: capacidades del brazo y ejes de la pregunta, nunca nombres	capacidades declaradas predicen un brazo no visto mejor que la dificultad de la tarea sola y peor que la identidad aditiva, sin cruzar su nulo exacto ($p = 0,066$) con ocho brazos; el catálogo predice un brazo que no existe; la ontología por eje separa con cinco segmentos casi lo que la región con trece, y más sobre su nulo por permutación
Lo que madura y lo que se diseña	creencias, proposiciones y tablas se acumulan solas, con guarda; un medidor nuevo lo escribe una persona, y la frontera está declarada	θ sobre la campaña: $4 \rightarrow 14$ regiones, $0 \rightarrow 11$ pares confiados, $0 \rightarrow 36$ de 78 tareas gobernadas, reproducible, la guarda frenó el paso con IC que cruza cero (§6.6); tres refutaciones sobre nano con la decisión reproducida 26/26 (§6.5)
El premio de elegir	qué paga fuera de muestra, y qué predice	58% de costo a utilidad igual con una señal computada, leave-one-task-out; el paradigma explica el 60% ajustado de la varianza del recall y la región menos del 1%; ofrecer una herramienta baja el releído 8,8× sin usarla

Lo que las une es la condición que la Proposición 5 enuncia, §6.2.5 midió y §6.5 respetó en cada paso: una clave de política tiene que ser COMPUTED . Determinismo y aprendizaje no compiten; el primero define sobre qué puede aprender el segundo. El estimador de piso de ruido, la corrección por selección y la partición por tarea separan aprender del registro de confabular sobre él. Sin esa disciplina los tres episodios de §6.5 serían tres ajustes post hoc.

Dos resultados más quedan medidos y en su lugar. El consenso entre paradigmas es una curva de calibración de credencia que se reproduce sobre una segunda familia de modelo y no abarata, así que su lugar es el dial, como regla de abstención.

Sobre el ruteo, lo que el registro dice tiene dos mitades. Entre los tres brazos que compiten por la calidad la brecha de oráculo apenas se separa del piso, porque tienen las mismas capacidades. Sobre los ocho brazos y en el held-out la brecha de oráculo es neta y positiva (+0,07 en muestra, +0,10 a +0,15 held-out), y ninguna política medida la captura, ni por identidad de paradigma (§6.5, tres mundos) ni por familia (§6.2.4, sin señal que la prediga). Ese resultado cierra la pregunta mal formulada y deja abierta la que el sistema puede aprender, que es sobre qué clave se cobra un premio que existe.

9.1 Lo que sigue, en orden

Construir el brazo que el catálogo predice y correrlo en las celdas de ausencia, contra el resto del plantel. Ningún paradigma junta cobertura garantizada con abstención por prueba faltante, y ésa es la capacidad que la pregunta de ausencia exige. Un brazo diseñado desde la tabla y medido después es la prueba más fuerte de que las capacidades predicen. Sube además el n del leave-one-arm-out, hoy en ocho brazos con p exacto 0,066 , que lo cruzan más brazos y no más tareas.

Medir si un clasificador calibrado recupera desde el request los ejes de la ontología que el caller no declara, con precisión suficiente para que la regla los consuma. §6.3.5 ya establece que la ontología separa contra su nulo por permutación, y que dos ejes declarados por el caller capturan parte de eso; los demás tienen a `ELICITED` como techo. Exige una llamada al modelo por tarea, y quedó pendiente por eso.

Medir el modo de vigencia por clase de falla y promover el eje a la clave si separa. Las dos clases son no encontrar la enmienda y encontrarla pero devolver igual el valor superado, y el F1 las confunde. El modo existe en el corpus de la campaña (297 filas, sobre enmiendas con precedencia declarada en forma fija) y su utilidad está en las tablas de §5.2; el eje no existe en el vocabulario de la política, y §6.3.5 muestra que sin él vigencia y horizonte desconocido caen en la misma región con efecto opuesto.

Generar tareas de cadena acoplada con otra semilla y correr sobre ellas la corrección de `pointer_chase`, en `luna` y en `terra`. Las cuatro correcciones de §6.1.4 se probaron sobre las nueve celdas de `terra` en que se las mide, en cuatro estados sucesivos, y la semilla nueva es lo que separa un mecanismo que transfiere de un ajuste en muestra sobre tres tareas. Cuesta una fracción de la campaña.

Cerrar la brecha entre la señal suelta y la política aprendida sobre la clave computada. La señal ahorra 58% sin perder utilidad; la política entera ahorra 31% a -0,104 (§6.4.1). La brecha es del sistema y no del corpus, y la consolidación tiene que llegar a aprender lo que una agrupación por una señal ya muestra.

Ver si la consolidación de §3.5 propone del registro exigencias con rutas alternativas, de la forma «A y B, o bien C y D», y puntuarlas con la guarda. La tabla de §6.3 es una conjunción con su contraejemplo escrito: `dag_strategy` resuelve la cadena acoplada sin dos de las tres capacidades que la tabla exige, por otra ruta.

Darle a la etapa de abstracción de §3.5 el registro de `P15`, el primero de los tres episodios de §6.5, con sus episodios y sus features crudos, y ver si propone sola una partición equivalente a la continuidad: una función del material que separe el horizonte desconocido 6 de 6 sin falsos positivos. Es lo que convertiría el ciclo de §6.5 en plasticidad del sistema y no en método de desarrollo. Si la propone, el lazo de la Figura 17 se cierra sin personas en los pasos 3 y 4.

Correr `P15`, `P16` y `P17`, los tres episodios de §6.5, sobre el modelo de la campaña. §6.5 vale para `nano`, y §6.2 y §6.2.6 dan el mismo veredicto sobre `luna` por otro camino. Repetirlos sobre `luna` cerraría la única costura de modelo que el paper tiene, y cuesta lo que costaron, unos 40M tokens.

Queda además lo pendiente de las secciones anteriores. Las superficies que cambian lo que el sistema *promete* (piso por estadísticas de rechazo, calibración por proposición) piden un corpus con evidencia insuficiente en algunas tareas. La comparación directa contra una ventana frontera sobre las 18 tareas anchas decidiría §7.2 por unos 82 dólares. La superficie de acciones exige una herramienta que cambie algo fuera del proceso. Y las tres correcciones de §7.1 para cómo se evalúan agentes no cuestan una corrida.

Apéndice A, Artefactos

artefacto	contenido
el control nulo	el andamiaje por prompt se conserva en el registro y no se ejecuta nunca: dominado por <code>direct</code> en toda celda medida, a igual utilidad y jamás más barato. Es la evidencia de que andamiar por fraseo no compra nada, no un brazo
<code>PATTERNS.md</code>	catálogo de patrones: 10 estructurales, 4 de control, 15 anti-patrones, con aplicabilidad enunciada sobre el vector de features
<code>ANALYSIS.md</code>	el análisis de fallas de §8 completo, por paradigma y por celda
<code>GATE.md</code>	ocho criterios binarios de publicación y su veredicto actual
<code>PLAN.md</code>	historia de revisiones de la tesis, incluidos dos encuadres superados y por qué
<code>D:\Apps\MAPO\lab</code>	el harness: 15 paradigmas registrados, de los cuales 12 corren la campaña, 9 brazos de recuperación implementados de los cuales 2 corrieron alguna vez, 4 superficies de herramientas, 4 niveles de garantía, 12 herramientas, generador de corpus con verificador independiente, 573 aserciones chequeadas por máquina (521 + 52 en dos suites)

Siete de los quince anti-patrones del catálogo son errores cometidos y medidos en este trabajo, dos de ellos contra predicciones registradas de antemano.

Apéndice B, Resultados formales auxiliares

Lo que §4 cita y no necesita en el cuerpo. Los enunciados están verificados por máquina en `tests/test_science.py`.

B.1 El umbral de imposibilidad, y contra qué pérdida se mide

Definir, por brazo.

$\bar{L}_j = E[-\Delta_j \mid S_j]$	la pérdida media de la TAREA	(no depende del ruteador)
$\rho_j = L_j / \bar{L}_j$	la SELECTIVIDAD DE PÉRDIDA	($\rho_j := 1$ cuando $\beta_j = 0$)

Corolario 2. El ruteador le pierde a siempre-fallback si y sólo si $\sum_j \pi_j \alpha_j G_j < \sum_j v_j \beta_j \rho_j \bar{L}_j$, y con un solo brazo

$$\beta_{\max}(\rho) = \pi \cdot \alpha \cdot G / (v \cdot \rho \cdot \bar{L})$$

p	qué ruteador es	qué hace el umbral
$p = 1$	ciego a la magnitud de la pérdida, se equivoca de forma representativa	el enunciado clásico, y ahí es exacto
$p < 1$	evita las equivocaciones caras	se afloja
$p > 1$	anti-calibrado: falla justo donde más duele	se endurece

Por qué el parámetro es necesario y no un refinamiento. Enunciado sólo con la pérdida distribucional, el umbral no es una imposibilidad. Alcanzan tres tareas: una ganancia de $+0,10$ con probabilidad $0,10$, ruteada; una pérdida de $-0,001$ con $0,45$, ruteada; y una de $-1,00$ con $0,45$, no ruteada. Entonces $\beta = 0,500$ supera $\beta_{\max} = 0,0222$ por $23\times$ y el ruteador igual captura $+0,00955$. Un ruteador que se equivoca *seguido pero barato* es lo que produce un ruteo selectivo bien construido.

Tampoco alcanza con sustituir por la pérdida realizada: un ruteador perfecto no realiza pérdida, así que el umbral reportaría infinito y parecería no imponer restricción alguna, la objeción que motivó la forma distribucional en primer lugar. Las dos objeciones son correctas. Se disuelven juntas en cuanto la pérdida realizada entra como *factor de la pérdida* y no como *denominador de un umbral*: con $\beta = 0$ el término entero se anula antes de que p se consulte.

Corolario 2b. Un ruteador obligado a elegir no controla ni β ni p . Uno selectivo controla los dos, y p es la palanca más barata: bajar β exige acertar más seguido; bajar p sólo exige abstenerse donde la apuesta es cara. Eso le da al Corolario 1 un mecanismo y no sólo una desigualdad, y p se recupera de cualquier registro que reporte pérdida potencial y realizada.

B.2 Cobertura óptima

Sea $v(c)$ el valor capturado a cobertura c , admitida bajando un umbral de confianza. Entonces $dv/dc = E[\Delta \mid \text{tarea marginal en } c]$, y por lo tanto:

(a) v es unimodal si y sólo si $c \mapsto E[\Delta \mid \text{marginal en } c]$ es no creciente, es decir, si y sólo si la señal de confianza ordena las tareas por *ganancia esperada*. La calibración sola no da eso. La calibración restringe la probabilidad de ganar; el valor depende de su magnitud. Construido, con $\Pr[\text{acertar} \mid \kappa] = \kappa$ exactamente en cada grupo: tres grupos con $E[\Delta]$ de $+0,008$, $-0,170$ y $+0,593$ a confianzas $0,90$, $0,60$ y $0,30$ producen una curva de valor capturado que sube, baja y vuelve a subir, con su óptimo en cobertura total.

(b) Sin ningún supuesto: la cobertura óptima es < 1 siempre que alguna tarea con $\Delta_{\{r(t)\}}(t) < 0$ fuera ruteada a cobertura total. Esto es lo que el argumento necesita, y se sigue en una línea, sacar un término negativo aumenta la suma.

Corolario 3. Afirmamos (b). Afirmar unimodalidad afirma más de lo que el diseño necesita, y regala un contraejemplo.

B.3 La identidad de la brecha del oráculo, y su alcance

Con un solo especialista, la brecha del oráculo iguala $\pi \cdot G$, lo que permite recuperar el β implícito de un ruteador publicado a partir de sus números de portada. Con k brazos no factoriza: la brecha es $E[\max_j \Delta_j^+]$, que no es $\pi_j \cdot G_j$ de ningún par fijo. Los 17,1pp reportados para una suite publicada pueden leerse como $\pi \cdot G$ sólo donde ese trabajo reporte un *par*; sobre una grilla, la lectura del β implícito no está disponible.

B.4 Qué es el Teorema 1, y qué no es

Es una *identidad*: una descomposición algebraica exacta, verdadera por construcción. Ninguna medición puede falsarla, y nada en este paper debe leerse como que la confirmó. Lo empírico es sólo si sus términos satisfacen la desigualdad sobre una distribución dada, una pregunta sobre un ruteador, no sobre el teorema.

El Teorema 1 y los tres corolarios están verificados en `tests/test_science.py` contra distribuciones cuyos términos se conocen por construcción. La identidad se chequeó además sobre 4.000 distribuciones al azar con k de 1 a 4 (3.269 de ellas con empates) con discrepancia máxima $1,67 \times 10^{-16}$. Los Corolarios 2 y 3 están enunciados arriba en su forma corregida; los contraejemplos que forzaron la corrección se reproducen ahí.

Y la distinción importa en este punto porque los términos nunca se separaron. En los dos corpus held-out el margen de decisión del ruteador fue 0 en todas las tareas, así que la curva riesgo–cobertura colapsa a un solo punto en el origen: AURC 0,000 contra un techo de +0,400. Un ruteador que nunca se abstiene no tiene α ni β distintos de siempre-fallback, así que la identidad se cumple vacuamente, con los dos términos medidos sobre una cobertura que el ruteador no eligió.

El Teorema 1 aporta la contabilidad; §B.5 aporta las afirmaciones falsables que este registro resolvió, dominancia de cascada y sensibilidad del detector. La rama de selección de la partición sigue sin ejercitarse (§8.2).

Una observación une las tres correcciones. π , α y β responden *si* el ruteador acierta; G , L y p responden *cuánto cuesta cuando no*. Cada lugar donde el enunciado anterior falló (el umbral, los empates, la unimodalidad) es un lugar donde esos dos ejes se trataron como uno.

B.5 Dominancia de la cascada, y su corrección medida

Un ruteador que se equivoca paga L , una pérdida de calidad: se entrega una respuesta peor y nada la recupera. Una cascada que se equivoca paga $\text{cost}(p_1)$, una pérdida de costo: el intento barato se desperdicia y la buena respuesta llega de todas formas tras escalar.

Con sensibilidad de detector s :

```
regret(ruteador) =  $v \cdot \beta \cdot p \cdot L$   
regret(cascada) =  $E[\text{costo de la escalera}] + (1-s) \cdot L$ 
```

Una corrección que le debemos a la medición. Un enunciado anterior de este resultado daba el término de costo como $\text{cost}(p_1)$. Es falso. Cuando ningún escalón satisface al detector, la cascada corre la escalera *completa*: en el estudio sintético capturó el 100% de la brecha a $4,4\times$ el costo del mejor paradigma fijo. El límite es el costo esperado de la escalera, y el patrón requiere un tope de presupuesto.

La sensibilidad del detector es la variable crítica, y un detector débil es catastrófico y no simplemente subóptimo. Con $s = 0,6$ la fracción capturada midió $-5,98$: una falla no detectada deja a la cascada detenida en un escalón barato con una respuesta mala. Es el análogo estructural del Corolario 2, como un ruteador con β alto pierde, una cascada con s bajo pierde, y pierde más fuerte.

Corolario (la partición por verificabilidad).

```
v = 1  (existe un oráculo barato)  ->  cascada; no hace falta ruteador
v = 0  (no hay oráculo)            ->  rutear, bajo la disciplina del Teorema 1
```

Predecir sólo es necesario donde verificar es imposible. Si esto se sostiene, en el régimen verificable la cascada resuelve lo que un selector intenta resolver, y más barato.

Y la partición tiene una consecuencia metodológica filosa que pagamos por aprender. La rama $v = 0$ es la que necesita un ruteador, y es precisamente la rama que un benchmark de coincidencia exacta no puede contener: el gold es lo que hace que corregir no necesite juez, y el gold es un detector. En este registro la confluencia fue literal (un mismo campo servía para las dos cosas) y el resultado es que dos predicciones de ruteo preregistradas, sobre dos corpus held-out independientes, las contestó la regla de cascada mientras la de selección no se evaluó nunca. Los números son reales; lo que miden es la rama $v = 1$.

Separar las dos no es un cambio de calibración. Requiere un corpus que declare la disponibilidad de detector por tarea sobre bases independientes de la clave de respuestas (para nosotros, si verificar es más barato que resolver) y mueve la cascada de disparar en 22 de 26 tareas a disparar en 2.

B.6 Soundness del ensamblador

Los tres resultados que siguen son estructurales: ninguno depende de un corpus, de un modelo ni de una corrida. Están en esta sección porque la maquinaria que describe §3.5 existe para hacer posibles enunciados de esta forma, y sin ellos la procedencia es contabilidad, se registra, se muestra, y no compra nada que alguien pueda nombrar.

Sea T una plantilla con ranuras $S = \{s_1 \dots s_n\}$, B una asignación de ranuras a proposiciones, Γ una base de creencias y ϕ un piso de procedencia.

Teorema 2 (soundness del ensamblador). Si $\text{fill}(T, B, \Gamma, \phi)$ emite una cadena R , entonces para toda ranura $s_i \in S$ existe $\beta_i \in \Gamma$ tal que

1. β_i es la creencia vigente sobre la proposición que B asigna a s_i ;
2. $\text{rank}(\text{procedencia}(\beta_i)) \geq \text{rank}(\phi)$;
3. la subcadena de R en la posición de s_i es exactamente $\text{str}(\text{valor}(\beta_i))$.

Y R no contiene ninguna subcadena que provenga de otro lado que no sea T o esos valores.

En una línea: si el ensamblador emite, todo número que emitió está implicado por la base de creencias al piso pedido. No «probablemente». No «salvo alucinación».

Demostración. Por construcción, y depende de una línea. `fill` recorre `slots_of(T)` (las ranuras que la plantilla realmente usa, extraídas de ella y no declaradas aparte) y para cada una hace exactamente tres cosas: resuelve la proposición asignada, pide la creencia vigente y compara el rango de procedencia contra el piso. Cualquier fallo anota el motivo y sigue, sin emitir. La sustitución ocurre después de comprobar que la lista de rechazos está vacía, así que `values` tiene una entrada por cada ranura de `T`, todas provenientes de una creencia vigente que pasó el piso; y la sustitución reemplaza ocurrencias del patrón de ranura dejando intacto el resto de la plantilla, que es texto fijo escrito por el código. Las tres condiciones se corresponden una a una con las tres guardas, y no hay un cuarto camino por el que un valor llegue a la salida. ■

Falla cerrada y falla ENTERA, que es una decisión y no una consecuencia. Si una sola ranura no llega al piso, no se emite una versión parcial. Emitir «*el saldo de la cuenta es ___*» no es más honesto que emitir un número inventado: es el mismo acto con mejor caligrafía, e invita al lector a completar lo que el contrato rechazó.

Cuatro límites, dichos adentro del teorema y no en una nota al pie.

límite	qué significa
el alcance es la ranura, no la oración	« <i>el saldo NO supera {x}</i> » con <code>x</code> correcto es sound y falso. No es un defecto de la implementación: es la frontera de la familia entera, y un red-team la mide en vez de suponerla
la procedencia es del registro, no del mundo	<code>COMPUTED</code> significa que alguien la computó y la asentó. El teorema traslada confianza desde el piso hacia la salida; no la crea. Un LLM que miente se emite con procedencia impecable
vigente, no histórica	la garantía es sobre el estado de creencias al ensamblar, no sobre todo lo que alguna vez se creyó
<code>str()</code> es parte del teorema	la condición 3 dice <code>str(valor)</code> , no «el valor». El ensamblador no formatea, porque formatear sería empezar a decidir algo sobre el número

Verificado exhaustivamente y no por casos elegidos: sobre el producto cartesiano de las cuatro procedencias por las condiciones de rechazo, un espacio chico, y por eso uno que se puede recorrer entero.

Apéndice C, Diferencias con los borradores anteriores

Qué cambió en v2. El título y la máquina son los mismos. Cambia qué se pone como columna vertebral: el banco deja de leerse como torneo entre paradigmas y pasa a leerse como fuente de episodios para un sistema que aprende. Lo que se aprende es qué exige un request, qué puede hacer cada brazo, y qué predice su comportamiento. Los resultados negativos del borrador anterior siguen todos, con sus

números, pero reubicados como lo que fueron: los episodios que repararon el vocabulario del sistema (§6.5).

Qué cambió en la revisión doctoral del 2026-09-01 (plan en `paper-es_PLAN.md`). El modelo de cada medición está declarado (§5.4, §6.5.1): la campaña corre con `gpt-5.6-luna`, los tres episodios de §6.5 con `gpt-5.4-nano`, y la celda de cadenas acopladas de §6.1.3 y §6.1.4 con `gpt-5.6-terra`. Las dos tablas del plantel se reconciliaron contra el registro. La agencia de la contribución 3 se corrigió: el ciclo lo ejecutaron personas dirigidas por refutaciones preregistradas, y el sistema consume lo que el ciclo produjo. Entraron la Proposición 5 (clave de la política), las definiciones de capacidad (§6.3.2), el régimen de los episodios, el impacto amplio (§7.3) y la literatura que faltaba.

Y lo que cambió tras la segunda ronda de revisión, el mismo día, con dos revisores externos de contexto limpio (`paper-es_REVIEW-A-2026-09-01.md` y `-B-`). El piso de ruido de §6.2.3 y §6.2.6 se computaba con el bootstrap del propio estadístico, que por construcción da piso igual o mayor que la brecha; se reemplazó por el estimador que el texto siempre describió, pseudo-brazos del mismo paradigma emparejados por número de brazos y por varianza, más el intervalo pareado de la brecha, con un test en la suite que lo verifica sobre un sintético de premio conocido. Con eso el veredicto del held-out cambia de signo: la brecha de oráculo es positiva y neta, y lo que no existe es una política que la capture. Todos los paneles pasan al único rectángulo mecánico del registro, 64×8 ; la inestabilidad se redefine como réplicas realmente distintas y baja a 12 a 28%; la identidad aditiva entra como baseline y gana a las capacidades; la tabla de política 41×7 se retira por no tener script que la reproduzca; P30 se actualiza al registro repuntado. Script de todos los recómpitos: `lab/bench/analysis/_recomputo_revision.py`.

Y tras la revisión de narrativa del mismo día: «aprender» queda definido en §1.1 como maduración auditable y no como optimización; §3.5 declara qué es fijo y qué es plástico y por qué; §6.6 mide la trayectoria de θ sobre la campaña y muestra el artefacto que un auditor lee. El título se mantiene: policy-as-code nombra el patrón que se usa.

Y el borrador 3.1 del 2026-09-02 reordena sin cambiar un número: el motor pasa adelante de la teoría y abre con un request real (§3.0); la teoría queda en lo que el argumento usa (§4) y lo auxiliar va al Apéndice B; el método se dice una vez (§5); los resultados siguen el orden del argumento (§6); el trabajo relacionado que el cuerpo sólo cita va al Apéndice D. Las definiciones, proposiciones, algoritmos y figuras se reenumeran por orden de aparición. El mismo día el catálogo de capacidades pasa de diez a doce: `CONTEXT_VISION` (cada llamada ve el hilo entero, crudo o compactado; la tienen `direct`, `react`, `reflection` y `dag_strategy`) y `AUTOCOMPACTA` (el arnés reduce el hilo de forma determinista; implementada, sin brazo en la campaña). El leave-one-arm-out se recomputó: MAE de capacidades $0,233 \rightarrow 0,229$, `p` exacto $0,080 \rightarrow 0,066$; la identidad aditiva sigue ganando por $0,004$.

C.1 Erratas respecto del borrador 2.0

Lo que el borrador del 2026-08-31 decía y esta versión no, con el motivo. Va en un apéndice y no en el cuerpo porque un lector nuevo no necesita la historia para entender el resultado, y un lector del borrador anterior la necesita entera. Las cuatro correcciones con contenido científico (el estimador de piso, el empate en el Teorema 1, la definición de inestable y la cota del ratchet) quedan además dichas en el cuerpo, en positivo.

dónde	decía	dice ahora	por qué
§6.2.3, §6.2.6	brecha neta negativa en muestra (−0,008) y en los tres estratos del held-out (−0,028 a −0,034)	entre contendientes apenas se separa del piso; sobre ocho brazos y held-out es positiva y neta	el piso era el bootstrap del propio estadístico, con media igual o mayor que la brecha por construcción; test §60
§6.1.2, resumen	17 a 34% de celdas inestables	al menos 12 a 28%	«inestable» se definía como $0 < media < 1$, que cuenta réplicas idénticas y parciales
§6.1.3, §6.1.4, resumen, §1.1, §9	0,33 a 0,89 y «12 correctas, 3 abstenciones, cero equivocadas» como campaña	los mismos números declarados como terra sobre nueve celdas, con los de luna al lado, y la corrección declarada como desarrollada sobre esas celdas	eran de otro modelo y el texto afirmaba que ninguna estadística mezclaba modelos
§6.1.2, §6.2	panel de 59 tareas × 8 brazos «con criterio mecánico»	rectángulo de 64 × 8	el 59 excluía cinco tareas a mano sin declararlo
§6.3.3, resumen, §9	«las capacidades superan a la identidad incluso cuando ésta ve la respuesta»	la identidad aditiva (tarea más brazo) da 0,225 contra 0,233 de capacidades (0,229 con el catálogo de doce)	la comparación era contra una identidad sin término de tarea
§6.4.1, §6.5.4, resumen	política sobre 41 × 7 con 0,951 y 46% de ahorro; señal con 42% a −0,017 sobre un panel de 59 tareas con cinco exclusiones a mano, y 69% a +0,008 sobre uno más chico	_plasticidad.py sobre el rectángulo: 68% a −0,063 con la clave completa, 31% a −0,104 con la computada; la señal da 58% a +0,000	la tabla 41 × 7 no la produce ningún script del repositorio; los 42% eran sobre el panel de 59
§6.3.5, resumen	«la ontología separa 40% más por segmento» (1,74 por eje contra 1,84 la región, sobre 46 tareas)	ontología por eje: S/R 2,53 con cinco segmentos contra nulo p95 1,46; región 2,59 con trece contra 2,12	la tabla venía de 46 tareas, sin script y con un control lineal ad hoc; ahora nulo por permutación
§6.3.4	p = 0,065 con 400 barajadas	p exacto sobre 40.320 permutaciones (0,080 con diez capacidades; 0,066 con las doce del catálogo vigente)	se podía computar exacto
§6.4.3, resumen	«el paradigma determina el recall seis veces más que la tarea»; brecha 4,2×	R ² ajustado: paradigma 60%, región 0,8%, tarea 0; brecha 2,5× la distancia mejor-peor	R ² crudo con 21 grupos contra 5; el 4,2× comparaba contra la desviación de un brazo respecto de la media
§6.4.3	utilidad 0,540 con y sin herramienta	0,822 y 0,825, con 7 de 63 celdas que cambian	veredicto anterior al re-puntuado del 2026-08-30
§6.5.5, resumen	«cada refutación produjo un sensor COMPUTED»; «el ciclo	un LLM, una corrección de valuación y un corpus; el	dos de tres episodios no produjeron LLMs; la

dónde	decía	dice ahora	por qué
	que repara su propio vocabulario»	vocabulario lo repararon personas, y §3.5 declara la frontera	plasticidad medida es la de las tablas (§6.6)
§6.5.3	sin el neto de P17	$-0,146$ a $\lambda = 0$, $-1,04$ a $\lambda = 0,05$	estaba en el registro y no en el paper
§6.4.2	terra: 23 tareas, 374 filas, 209 celdas	26 tareas, 350 filas, 200 celdas, curva completa	el registro creció y la predicción exigía la curva entera
§1.3, §5.4	$78 \times 12 \times 3$, 121,4M tokens	2.511 filas, 123,3M tokens, nueve brazos sobre 67 tareas	conteo contra el registro
§5.2, §5.2.1	dos tablas del plantel con números distintos	una, recomputada contra el registro	snapshots distintos
§6.2.1	α 41%, β 10%, γ 48%, S/R 5,30	α 45/50%, β 9/11%, γ 45/39%, S/R 3,02	ddof = 1 y ruido descontado en las tres componentes, sobre 64×8
§2.5, referencias	HADD como «la base de la que §3.4 toma sus invariantes», con EVR como mecanismo	antecedente de vocabulario; EVR no disponible en Zenodo	verificado el 2026-09-01
§5.1.1	«59,4% de los fallos eran defectos del arnés», sin fuente	auditoría de OpenAI sobre SWE-bench Verified: 59,4% de 138 tareas con defectos en tests o enunciado	fuelle encontrada; el denominador y el objeto eran otros
§8.1, §8.2	párrafos sobre un corpus de 16k tokens, celdas «†» y P8 sin contexto	retirados o reescritos con contexto	texto heredado de un borrador anterior a la campaña
§3.4.2	la cota del ratchet importaba un teorema sobre varianza bajo oscilación	Proposición 2: el daño total acotado por conteo, $\leq 2R$ endurecimientos	una secuencia monótona y acotada tiene varianza que tiende a cero, así que la cota se cumplía vacuamente
§4.3.1	β definida sobre el complemento de S_+ , cobrando los empates como misruteo	β sobre S_- ; los empates aportan cero a los dos lados	un ruteador que sólo rutea donde gana o empata registraba una tasa de misruteo alta sin daño, y el Corolario 2 usa β y L por separado
§6.2.6	held-out medido sobre una corrida anterior al cambio de tokenizador, sin analyzer, huella ni vocabulario de región estampados	held-out re-corrido por el mismo camino de código de la campaña	la corrida anterior no era replayable

Apéndice D, Trabajo relacionado extendido

D.1 Búsqueda automática de workflows

AFlow reformula la optimización de workflows como búsqueda sobre workflows representados como código, con MCTS sobre operadores [arXiv:2410.10762]; ADAS y GPTSwarm buscan en espacios afines. Estos producen un workflow por benchmark, offline. Nuestro interés es la selección por solicitud dentro de un conjunto fijo y, antes de eso, qué miembros del conjunto pueden correr siquiera.

D.2 Modelos de costo para workflows

GLOW predice performance de workflows agénticos a partir de features de grafo y de lenguaje [arXiv:2512.15751]; Cost-Aware Optimization for Agentic Query Execution hace explícita la analogía con la optimización clásica de consultas [arXiv:2606.03152]. Tomamos la analogía como establecida y no la reclamamos. Nuestro aporte está aguas arriba: un filtro duro de factibilidad que un modelo de costo no reemplaza, porque un plan infactible no tiene costo.

D.3 Predicción selectiva y aprender a diferir

Nuestra teoría es una aplicación de un marco establecido. La regla de Chow da el rechazo óptimo; Mozannar y Sontag dan un surrogate consistente para diferir a un experto [PMLR v119]; Verma y Nalisnick agregan deferral one-vs-all calibrado [arXiv:2202.03673]; Mao, Mohri y Zhong dan formulaciones multi-experto principiaadas [arXiv:2310.14774], y Verma, Barrejón y Nalisnick el caso multi-experto calibrado [AISTATS 2023]; Madras, Pitassi y Zemel son el antecedente de aprender a diferir [NeurIPS 2018]. La curva riesgo-cobertura y su área, AURC, que §B.4 usa, son de El-Yaniv y Wiener [JMLR 2010] y de Geifman, Uziel y El-Yaniv [arXiv:1805.08206]. Wen y colegas relevan la abstención en modelos de lenguaje [arXiv:2407.18418]. Contribuimos la aplicación a la selección de paradigmas y los corolarios específicos de §4.3, no el marco.

El consenso de §6.4.2 también tiene antecedente directo. Self-Consistency muestrea varias cadenas de razonamiento del mismo modelo y vota la respuesta final [arXiv:2203.11171]; Universal Self-Consistency extiende el voto a respuestas de formato libre con el propio modelo como agregador [arXiv:2311.17311]. §6.4.2 es esa idea con tres diferencias: los votantes son estructuras de control de flujo distintas y no muestras de la misma, el fenómeno se replica sobre una segunda familia de modelo contra un criterio registrado antes de correr, y el resultado del voto mueve credencia dentro de ELICITED y nunca procedencia, así que no habilita una acción irreversible. La agregación por votación es prestada; la epistemología de qué autoriza el voto es lo que se agrega.

D.4 Agentes que aprenden sin actualizar pesos

Los enfoques de memoria experiencial, ExpeL, aprendizaje reflexivo experiencial, MemSkill, R²-Mem, acumulan insights, reglas o entradas de memoria. Sleep-time compute corre inferencia en tiempo ocioso y reporta ~1/5 de los tokens en inferencia [Letta]; SCM y trabajo afín agregan consolidación de inspiración biológica [arXiv:2604.20943, arXiv:2605.26099]. Todos consolidan contenido. §3.5 consolida la política de control, y la afirmación sobrevive una búsqueda fechada (2026-08-26) sólo como conjunción, así que la

enunciamos como tal: la política se aprende de los episodios propios del agente, se consolida offline, en un artefacto determinista versionado que se ejecuta fuera del LLM, sobre features estructurales de la tarea. Cada conjunto tiene un vecino fuerte. Trace2Policy destila control desde trazas de *expertos* a bases de reglas que se reinyectan *como texto de prompt* [arXiv:2606.10457]; la compilación declarativa de políticas le da a las políticas de orquestación exactamente la forma de artefacto que queremos, determinista, auditable, versionada, pero *escrita por humanos*, no aprendida [arXiv:2603.27299]; el meta-ruteo composicional aprende un ruteador interpretable offline, sobre features textuales y no estructurales [arXiv:2608.00106]. Ninguno aprende su propia política de control hacia un artefacto así.

Y el antecedente de ingeniería de «el flujo de control vive en el código» es DSPy, que trata al modelo como un módulo invocado desde un programa cuyo control es del programador, y compila el programa optimizando los prompts y los pesos de los módulos sin tocar la estructura [arXiv:2310.03714]. La separación es la misma que este trabajo hereda. La diferencia es qué se optimiza: DSPy ajusta cada módulo contra una métrica, y este trabajo excluye el fraseo del catálogo por control nulo (§3.1) y aprende sobre qué exige la pregunta y qué puede hacer cada estructura.

La idea de que el modelo propone y una función computada desde el entorno filtra es más vieja que este trabajo y hay que acreditarla. SayCan la aplica a acciones robóticas: el modelo propone, una función de affordance medida decide [arXiv:2204.01691]. LLM+P hace que el modelo traduzca a PDDL y un planificador clásico decida [arXiv:2304.11477]. CoALA ordena las arquitecturas cognitivas para agentes de lenguaje sobre el linaje de Soar que §2.1 usa [arXiv:2309.02427], así que ese puente no es de este paper. NeMo Guardrails con Colang [arXiv:2310.10501] define flujos de diálogo programables alrededor del modelo, en los que el modelo genera el paso siguiente del flujo, y LMQL [arXiv:2212.06094] restringe la salida del modelo con un lenguaje de consulta; los dos son el antecedente de tipar la salida antes de usarla, que es lo que la línea 10 del Algoritmo 2 hace. AgentSpec [arXiv:2503.18666] y GuardAgent [arXiv:2406.09187] son pisos deterministas sobre acciones de agentes, anteriores a ProvenanceGuard. Y del lado de aprender política desde episodios propios hacia un artefacto ejecutable, Voyager consolida habilidades como código desde la experiencia [arXiv:2305.16291] y Agent Workflow Memory induce flujos reutilizables desde trayectorias [arXiv:2409.07429]. Lo que queda propio de §3.5 frente a los dos últimos es que lo consolidado sea la tabla de decisión sobre una clave `COMPUTED`, con guarda de promoción sobre episodios retenidos y partición proponer, puntuar, promover por tarea.

D.5 Posicionamiento

	cobertura	decisión determinista	auditable	filtro de factibilidad	abstención	aprende sobre	clave de la política
AFlow / ADAS / GPTSwarm	offline, un workflow	no	no	no	n/c	la estructura del workflow, por benchmark	n/c
Select-then-Solve	1,0	no	no	no	no	identidad del paradigma	embedding de la tarea
FlowBank	1,0	no	no	no	no	identidad del workflow	embedding de la consulta
TRACE-Router	1,0	no	no	no	no	identidad del paradigma, por traza	features de la traza
FrugalGPT y cascadas de modelos	1,0	sí, el orden	parcial	no	no	umbral de aceptación por escalón	puntaje elicitado sobre la respuesta
SCL	modo global	sí	sí	no	no	no aprende	n/c
Este trabajo	selectiva	sí	sí	sí	sí	capacidades del brazo y ejes de la pregunta	COMPUTED por construcción (\$4.2)

Referencias

Los trabajos se identifican por su identificador persistente. Los marcados con ★ se leyeron completos en la fecha indicada; de los demás se verificó lo que este paper les atribuye, y donde el original no aporta un título corto se lo cita por identificador con la atribución que el cuerpo le hace.

Búsqueda y selección de workflows

- Select-then-Solve. arXiv:2604.06753, selección de paradigma por tarea; ★ leído completo el 2026-08-26.
- FlowBank. arXiv:2606.11290, portafolio offline, selección por consulta.
- TRACE-Router. arXiv:2607.22465, ruteo a granularidad de traza de tarea.
- Uno-Orchestra. arXiv:2605.05007, política conjunta de descomposición y despacho.

- Optimización de workflows como código, con MCTS sobre operadores: arXiv:2410.10762.
- Query Optimization for Agentic Query Execution. arXiv:2606.03152, la analogía con la optimización clásica de consultas.
- arXiv:2608.00106, ruteo sobre features textuales, con *confidence gate* declarado como trabajo futuro sin evaluar.

Cascadas y ruteo de modelos

- FrugalGPT. Chen, Zaharia y Zou (2023), *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176, la cascada de modelos con puntaje de aceptación por escalón.
- Hybrid LLM. Ding et al. (2024), *Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing*. arXiv:2404.14618.
- RouterBench. Hu et al. (2024). arXiv:2403.12031.
- RouteLLM. Ong et al. (2024), *RouteLLM: Learning to Route LLMs with Preference Data*. arXiv:2406.18665. Las cuatro deciden entre modelos con el mismo control de flujo; ninguna declara detector por tarea ni capacidades del brazo. Citadas por sus afirmaciones de portada, no leídas completas (2026-09-01).

Votación y programación con módulos de lenguaje

- Self-Consistency. Wang et al. (2022), *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv:2203.11171.
- Universal Self-Consistency. Chen et al. (2023). arXiv:2311.17311. Antecedentes directos de §6.4.2; lo que se agrega está dicho en §D.3.
- DSPy. Khattab et al. (2023), *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*. arXiv:2310.03714, antecedente de ingeniería de «el flujo de control vive en el código».
- Kautz (2022), *The Third AI Summer*, AI Magazine 43(1), la taxonomía neurosimbólica que §2.5 usa para ubicar a HADD.
- Agent2Agent, protocolo de tarjetas de agente (Google, 2025), vecino de ingeniería de §6.3 por el lado de las descripciones de habilidades.

Gobernanza simbólica y motores de creencias

- ★ Jaime y Errecalde (2026). MINERVA/HADD. *A Production Architecture for Hybrid Agents with Deterministic Decisions in Regulated Domains*. Zenodo 10.5281/zenodo.20003407 (leído 2026-08-26). Antecedente de vocabulario de §3.4. Su compuerta companion (EVR, Zenodo 19791686) no estaba disponible en Zenodo al 2026-09-01.
- SCL / Soft Symbolic Control. *Structured Cognitive Loop*. arXiv:2511.17673.
- Nous. arXiv:2606.22030, confiabilidad acotada por la procedencia del canal.
- MemIR. arXiv:2605.25869, memoria tipada por procedencia.
- Eywa. arXiv:2605.30771, promoción de hechos tras validadores.
- HEP. arXiv:2607.09195, evolución auditable de hipótesis.
- arXiv:2607.01507, diagnóstico empírico del jardín de senderos que se bifurcan.
- arXiv:2603.27299 política determinista, auditable y versionada, *escrita por humanos*.
- ProvenanceGuard. arXiv:2607.01236, pisos de procedencia sobre acciones.

- Contratos de delegación e identidad atestiguada. arXiv:2603.18043.
- arXiv:2605.26667, atribución de error por registro de operaciones intermedias.

Aprendizaje sin actualizar pesos

- Kintsugi. *Learning Policies by Repairing Executable Knowledge Bases*. arXiv:2605.09487, establece el paso de instalación filtrada por verificador que §2.4 le cede.
- EnvProbe. *Ask the World Before Acting: Budgeted Environment Probing for World-Model Calibration*. arXiv:2606.31422.
- SCM y afines, consolidación de inspiración biológica: arXiv:2604.20943, arXiv:2605.26099.
- arXiv:2606.10457, destilación de control desde trazas de expertos a reglas reinyectadas como texto de prompt.

Predicción selectiva, deferral y evaluación

- Smith y Winkler (2006), *The Optimizer's Curse: Skepticism and Postdecision Surprise in Decision Analysis*, Management Science 52(3), el sesgo del máximo que el piso de ruido de §6.2.3 descuenta.
- Chow (1970), *On optimum recognition error and reject tradeoff*.
- Mozannar y Sontag, deferral con surrogate consistente [PMLR v119]; Verma y Nalisnick, deferral one-vs-all calibrado: arXiv:2202.03673.
- arXiv:2605.07805, descomposición de incertidumbre que unifica abstención y ruteo con garantías *distribution-free*.
- eDAct. arXiv:2604.07036, deferral por umbral calibrado de incertidumbre.
- arXiv:2606.19544, confiabilidad sin validez en jueces LLM, y sobre- estimación por acuerdo crudo.
- arXiv:2512.15751.
- Survey. *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents*. arXiv:2606.04990.

Métrica y linaje clásico

- τ -bench. Yao et al. (2024), *τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv:2406.12045, origen de `pass^k`.
- τ^2 -bench. Sierra, 2025. github.com/sierra-research/tau2-bench.

Paradigmas del plantel, ruteo en RAG y antecedentes del LLM

- ReAct. Yao et al. (2022). arXiv:2210.03629.
- Reflexion. Shinn et al. (2023). arXiv:2303.11366.
- ReWOO. Xu et al. (2023). arXiv:2305.18323.
- Adaptive-RAG. Jeong et al. (2024), NAACL. arXiv:2403.14403, ruteo por complejidad entre paradigmas de recuperación.
- Self-RAG. Asai et al. (2023). arXiv:2310.11511.
- FLARE. Jiang et al. (2023), *Active Retrieval Augmented Generation*. arXiv:2305.06983.
- Self-Route. Li et al. (2024), *Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach*. arXiv:2407.16833.

- AutoMix. Aggarwal et al. (2023). arXiv:2310.12963, cascada con auto-verificación.
- SayCan. Ahn et al. (2022), *Do As I Can, Not As I Say: Grounding Language in Robotic Affordances*. arXiv:2204.01691.
- LLM+P. Liu et al. (2023). arXiv:2304.11477.
- CoALA. Summers et al. (2023). arXiv:2309.02427, arquitecturas cognitivas para agentes de lenguaje.
- NeMo Guardrails. Rebedea et al. (2023). arXiv:2310.10501.
- LMQL. Beurer-Kellner et al. (2022; PLDI 2023). arXiv:2212.06094.
- AgentSpec. Wang et al. (2025). arXiv:2503.18666.
- GuardAgent. Xiang et al. (2024). arXiv:2406.09187.
- Voyager. Wang et al. (2023). arXiv:2305.16291, habilidades consolidadas como código desde la experiencia.
- Agent Workflow Memory. Wang et al. (2024). arXiv:2409.07429.
- OpenAI (2026), *Why SWE-bench Verified no longer measures frontier coding capabilities*, openai.com, la auditoría del 59,4% de §5.1.1. Los identificadores y atribuciones de este bloque se verificaron contra el abstract de arXiv el 2026-09-01 ([historico/verificacion-citas-2026-09-01.md](#)); cinco se verificaron contra el cuerpo del PDF.

Predicción selectiva, varianza y comparaciones múltiples, adiciones

- El-Yaniv y Wiener (2010), *On the foundations of noise-free selective classification*, JMLR 11.
- Geifman, Uziel y El-Yaniv (2018), *Bias-reduced uncertainty estimation for deep neural classifiers*, arXiv:1805.08206, el AURC.
- Madras, Pitassi y Zemel (2018), *Predict responsibly*, NeurIPS.
- Mao, Mohri y Zhong (2023), *Principled approaches for learning to defer with multiple experts*, arXiv:2310.14774.
- Verma, Barrejón y Nalisnick (2023), *Learning to defer to multiple experts*, AISTATS.
- Wen et al. (2024), *Know your limits: a survey of abstention in large language models*, arXiv:2407.18418.
- Ouyang et al. (2023), *LLM is like a box of chocolates: the non-determinism of ChatGPT in code generation*, arXiv:2308.02828.
- Atil et al. (2024), *LLM stability*, arXiv:2408.04667.
- He (2025), *Defeating nondeterminism in LLM inference*, Thinking Machines.
- Bouthillier et al. (2021), *Accounting for variance in machine learning benchmarks*, arXiv:2103.03098.
- Westfall y Young (1993), *Resampling-based multiple testing*, Wiley, el procedimiento maxT de §6.2.2.

El linaje de §2.1. Alchourrón, Gärdenfors y Makinson (1985), *On the logic of theory change*. Doyle (1979), *A truth maintenance system*. de Kleer (1986), *An assumption-based TMS*. Buneman, Khanna y Tan (2001), *Why and where: a characterization of data provenance*. Green, Karvounarakis y Tannen (2007), *Provenance semirings*. Rao y Georgeff (1995), *BDI agents: from theory to practice*. Laird, Newell y Rosenbloom (1987), *SOAR: an architecture for general intelligence*. Hebb (1949), *The Organization of Behavior*. Dung (1995), *On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games*.

Marco normativo. Reglamento (UE) 2024/1689 de Inteligencia Artificial, arts. 12 y 14. Reglamento (UE) 2016/679 (RGPD), art. 22.